

claude-windows / 96963dab-d38f-488c-af44-530ddbefe1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1.jsonl`
- SHA-256: `ac46453e4fe44a5adf8752270c3f6b77b3e18cfc7e2577d21c63425cd6a13e05`
- Source modified: `2026-07-08T07:10:58+00:00`
- Imported at: `2026-07-08T15:59:19+00:00`
- project: `C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer`
- session_id: `96963dab-d38f-488c-af44-530ddbefe1`
```

Transcript

1. user (2026-07-08T05:41:46.739Z)

<local-command-caveat>Caveat: The messages below were generated by the user while running local commands. DO NOT respond to these messages or otherwise consider them in your response unless the user explicitly asks you to.</local-command-caveat>

2. user (2026-07-08T05:41:46.739Z)

```
<command-name>/model</command-name>
  <command-message>model</command-message>
  <command-args>claude-fable-5</command-args>
```

3. user (2026-07-08T05:41:46.739Z)

<local-command-stdout>Set model to claude-fable-5</local-command-stdout>

4. user (2026-07-08T05:41:47.874Z)

Work GitHub issue #524 (Khelsutra/badminton-highlight-indexer): "epic: collapse the pipeline onto the L4 worker; shrink control-plane to a thin router". Read it first (`gh issue view 524 -R Khelsutra/badminton-highlight-indexer`), plus #521 and its data-movement comment, #520, and the evidence in `docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/DEPLOY\_REPORT\_cloud-serving\_validation-latency\_2026-07-07.md` §9. This is an INVESTIGATION-FIRST task ? design before the risky rearchitecture.

Repo / environment

```
- Real code: `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (the sibling `C:\Users\avidu\Projects\badminton-highlight-indexer` is a STUB ? ignore).
- Windows/PowerShell primary. `gcloud`?googleapis + `gh`/`git`?GitHub work; general web egress is blocked. Project `gen-lang-client-0306026826`, region `asia-southeast1`. Live control-plane rev `rally-control-plane-00019` (image `c6393d27`); L4 is the `rally-worker` Cloud Run **Job**.
```

The finding that motivates this (verified 2026-07-08)

For one run a 1.83 GiB clip is moved **3x**: (1) upload streams straight to the bucket (`backend/api/uploads.py` ? never touches CP local disk, by design; #480/#471 OOM fix; MD5 computed incrementally during the stream); (2) the control-plane re-downloads it to scratch to validate ? `\_execute\_processing` ? `storage.acquire\_local\_input(gs://?)` at `backend/orchestration.py:748`, then a redundant re-hash `compute\_video\_id` at line 751; (3) the worker downloads it again to detect (`[worker] pulled ?`). The ~30 s pre-validation gap is that CP re-download, and it's unlogged.

Deliverables (in order)

1. **Design doc** under `docs/COMPUTE\_DECOUPLED\_SERVING/` that RESOLVES the compute-topology question, because "upload directly to the L4" needs an inbound endpoint on the GPU box and the current L4 is a Cloud Run **Job** (batch, no inbound HTTP). Evaluate

cost/latency/reliability/ops for:

- **A. Cloud Run Service with GPU** (scale-to-zero vs min-instances) hosting upload+pipeline;
- **B. GCE L4 VM** (persistent, reliable PD ? matches the owner's "reliable disk" point);
- **C. Hybrid** ? control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.

MUST quantify the key tension: uploads take **minutes** (1.83 GiB ? 2m55s at the user's uplink), so uploading ONTO the L4 leaves an **expensive GPU idle** during the whole upload, whereas today upload hits the cheap CP/bucket and the L4 only runs ~4 min for detect. Weigh round-trip savings vs idle-GPU cost; consider a non-GPU co-located sidecar/disk as a middle ground. Define the **minimal control-plane surface** (Cloudflare-Access auth, dispatch, DB writes, SPA + job status) and the **async copy-to-bucket durability model** (when the mirror runs ? "during segmentation" ? failure/resume semantics, and the exposure window where data is only on L4 local disk). Lay out a **phased migration** built on #521's config-driven phase-placement so each move is a config flip with a bucket-mediated fallback.

2. **Land the two low-risk near-term wins** (guarded by config, reversible):

- **Validation on the L4**: when configured, validation runs on the worker (which already downloads the clip for detect) so the control-plane never pulls the full file ? kills the ~30 s gap. Build on #521's `phase_placement` config (coordinate ? see below).`

- **Drop the redundant re-hash**: `/api/process` should trust the upload's streamed `video_id` (backend/api/uploads.py` `upload_complete` returns it) instead of re-hashing at `orchestration.py:751`. Keep a fallback that hashes when the id is absent (e.g. a direct-URI submit that didn't go through the chunked upload). The download for validation stays UNLESS validation moved off the CP (win #1).`

3. Phase the risky upload-rearchitecture behind the design doc ? do NOT ship upload-to-L4 before the topology/cost question is answered and reviewed.

Coordinate with the in-flight #521 work

A separate session is implementing #521 (compile?L4 + `phase_placement` config). Do not duplicate the config knob ? branch from the LATEST origin/master`, and if #521's PR has merged, build on its phase_placement`; if not, design win #1 to slot into the same knob and note the dependency in your PR. Prefer landing the design doc + win #2 (re-hash, independent of #521) first if #521 is still open.`

Gates + branching safety (concurrent shared checkout ? follow every time)

- Full suite green (`python run_all_tests.py`/`pytest` ? not just changed files), ruff check .`, mypy`, python -m tools.leak_scan`, python -m tools.check_md_links`; coverage ? the repo's --cov-fail-under` floor.`

- Work in an ISOLATED clone/worktree; never touch the shared working tree or sibling branches (there may be active WIP e.g. `feat/validation-result-cache`, and the #521 session). git fetch origin` then git checkout -b feat/524-l4-primary origin/master`. Re-verify before commit: git log --oneline origin/master..HEAD` lists ONLY your commits. Stage explicit paths (never git add -A`). PR against `master`, reference #524 (+ #521, #520), end the body with the Claude Code attribution line.`

- Deploy is out of scope ? design + implement + test + PR only. Ask before any `gcloud run deploy` / topology change / new VM.`

5. assistant (2026-07-08T05:41:57.826Z)

I'll start by getting the repo current and reading the primary sources (issue #524, #521 + comments, #520, and the deploy report evidence).

6. user (2026-07-08T05:42:02.265Z)

codex/issue-523-sd-reel-plan...origin/master

```
M SESSION_HANDOFF.md
?? score_golden_gate.py
?? score_local.py
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(#9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
```

7. user (2026-07-08T05:42:03.728Z)

```
{"body": "***Epic / investigation.** Related: #521 (compile?L4 + config-driven phase placement), #520 (step-level tracing). Motivated by the 2026-07-08 live-CUJ data-movement finding (see #521 comment).\n\n## Vision\nMake the **L4 worker the primary compute for the whole pipeline** ? upload ? validate ? segment ? stitch all on the L4's local disk ? and **shrink the control-plane to the smallest possible surface** (auth, routing/dispatch, DB, SPA serving, job status). Today the control-plane does heavy CPU work (validation ~183 s + stitch ~13 min on 2 vCPU) and the source clip is moved **3x** per run (upload?bucket, bucket?CP-scratch to validate, bucket?worker-scratch to detect) ? see the #521 comment.\n\n## Proposed direction (owner)\n1. **Upload directly to the L4** (write to L4 local disk) instead of streaming to the bucket, so the pipeline never re-downloads.\n2. **Async copy-to-bucket for durability** ? while the video is being segmented, a background `gsutil`/`storage.copy` mirrors the local file to GCS. Accepts a small durability window (data briefly only on L4 disk) in exchange for eliminating the round-trips. The L4's local disk is reliable enough that this is an acceptable trade.\n3. **Control-plane residual = thin router**:: Cloudflare-Access auth, dispatch, DB writes (`play_segments`/video records), SPA + job-status. No heavy decode/encode.\n\n## Hard question the investigation MUST resolve first (compute topology)\nThe current L4 is a Cloud Run **Job** (batch, no inbound HTTP, spun up per-dispatch). `\"Upload directly to the L4\"` needs an **inbound endpoint on the GPU box**, which a Job can't provide. Options ? evaluate cost/latency/ops for each:\n- **A. Cloud Run Service with GPU** (scale-to-zero or min-instances) hosting the upload + pipeline. Cold-start vs. warm-idle-GPU cost.\n- **B. GCE L4 VM** (persistent, reliable PD) hosting upload + pipeline. Persistent GPU cost, but a real reliable disk (matches the owner's `\"reliable disk\"` point).\n- **C. Hybrid**:: control-plane presigns/redirects the upload to a just-spun-up L4 (service/VM), pipeline runs there, async copy to bucket.\n- **Key tension to quantify**:: uploads can take **minutes** (user bandwidth ? the 2026-07-08 run uploaded 1.83 GiB in ~2m55s). Uploading *onto* the L4 means an **expensive GPU sits idle during the whole upload**, whereas today upload hits the cheap control-plane/bucket and the L4 only runs for the ~4-min detect. Does the round-trip saving beat the idle-GPU cost? A middle ground: upload to a **cheap L4-adjacent disk / non-GPU sidecar** co-located with the L4, or keep the bucket upload but co-locate compute so the download is LAN-fast. The investigation should produce the numbers.\n\n## Concrete near-term wins (low risk ? can land independently, overlap #521)\n- **Move validation onto the L4** (it already downloads the clip for detect) ? control-plane stops pulling the full file; the ~30 s silent pre-validation gap disappears.\n- **Trust the upload's streamed `video_id`** in `/api/process` (`backend/api/uploads.py` already computes the MD5 incrementally during the stream) ? drop the redundant re-hash in `backend/orchestration.py:751`.\n\n## Deliverables\n1. A **design doc** under `docs/COMPUTE_DECOUPLED_SERVING/` that resolves the topology question (A/B/C) with a cost/latency/reliability comparison, defines the minimal control-plane surface, specifies the async copy-to-bucket durability model (failure/resume semantics + the exposure window), and lays out a **phased migration** (start from the #521 config-driven phase-placement so each move is a config flip with a bucket-mediated fallback).\n2. Land the two near-term wins above (guarded by config; tests + full CI green).\n3. Phase the risky rearchitecture behind the design doc ? do not rush upload-to-L4 before the topology/cost question is answered.\n\n## Acceptance\n- Design doc merged with a clear recommendation + phased plan + reliability analysis.\n- Near-term wins landed (validation-on-L4 config path avoids the CP download; redundant re-hash removed), full suite + ruff + mypy + leak_scan + check_md_links green, coverage ? floor.\n- Everything reversible by config (bucket-mediated path stays as fallback).\n\", \"comments\": [], \"state\": \"OPEN\", \"title\": \"epic: collapse the pipeline onto the L4 worker; shrink control-plane to a thin router (upload?stitch on L4 + async bucket copy)\"}
```

8. assistant (2026-07-08T05:42:10.802Z)

Now let me read #521 (with its data-movement comment), #520, and check whether #521's PR has landed on master yet.

9. user (2026-07-08T05:42:14.978Z)

```
{"body": "## Problem\nThe **compile/stitch** phase runs **in-process on the CPU-only 2-vCPU control-plane** (`backend/orchestration.py::_execute_compile` ? `_LOCAL_COMPUTE_LANE` ffmpeg).\n**Validation also runs on the control-plane.** On the 2026-07-08 live CUJ (rev `rally-control-plane-00019`, clip `\"First CUJ.MP4\"` ? 1.83 GiB / 174.17 s / 4K-class):\n\n| Phase | Machine | Wall | \n| --- | --- | --- | \n| Validation (#512 shared-decode) | control-plane 2 vCPU | **183 s** | \n| Detection (WASB) | L4 worker | 161.6 s | \n| **Compile / stitch** (11 clips ? ~80 s reel) | \n| **control-plane 2 vCPU** | **~12m55s** | \n\nStitch was **~45?90 s per ~7 s clip** because the
```

****per-clip branding watermark forces a full 4K re-encode**** with no NVENC on 2 vCPU. So a single run pins the control-plane for ****~16 min of heavy CPU**** (validation + stitch), serialized by ``_LOCAL_COMPUTE_LANE``. At even a handful of concurrent users the control-plane saturates and the API becomes unusable ? estimated ****~5 concurrent users renders it useless****.
****Move compile/stitch to the L4 worker**** (hardware ****NVENC**** + 8 vCPU), the way detection already dispatches. The worker already fetches the source, and ``_execute_compile`` is designed to re-resolve in the worker (``_resolve_compile`` is explicitly ``run at SUBMIT **AND** in the WORKER``). Add a ``kind='compile'`` Cloud Run dispatch path.
****Make phase?machine placement configurable**** ? a per-phase target knob (``validation``, ``segment/detect``, ``compile/stitch`` ? ``control_plane | cloud_run_l4 | \u003cother\u003e``) so phases can be relocated ****by config, no code change****. Mirror the existing ``deployment.compute_target`` + ``_maybe_dispatch_remote`` forwarding pattern (`COMPUTE_DECOUPLED_SERVING`).
****Cheaper watermark**** ? pre-rendered overlay / NVENC filter path so stitch isn't a full CPU re-encode (the dominant term above).
 Acceptance
 - Compile can be dispatched to the L4 by config; control-plane stitch stays available as a config option.
 - Flipping config moves validation / detect / compile between machines ****without a code change****.
 - Measured stitch wall for this reel drops from ~13 min (2 vCPU) to sub-minute-class on L4.
 Evidence
 \n docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` \$9 (2026-07-08 live CUJ). Compile timeline: ``POST /api/compile` 04:37:26 ? 11x per-clip 4K re-encode+watermark (~67 s each) ? concat ~2 s ? reel mirrored 04:50:21 (`first-cuj-ks-highlight-reel.mp4`, 581.8 MB).`
 \n \n Filed from the 2026-07-08 live-CUJ phase breakdown.
 \n , "state": "OPEN", "title": "perf/scaling: run compile/stitch on the L4 worker; make phase?machine placement config-driven"}
 === COMMENTS ===
 --- avidullu ---

Data-movement finding from the 2026-07-08 live CUJ (extends this issue)

Tracing the ~30 s pre-validation gap (``POST /api/process`` ? ``Running validations``) showed the source clip is moved ****three times**** for one run:

1. ****Upload ? bucket**** ? streamed straight to GCS (``backend/api/uploads.py``; deliberately never touches CP local disk ? the #480/#471 OOM fix), computing the MD5 ****incrementally as it streams****.
2. ****Bucket ? control-plane scratch**** ? ``_execute_processing`` (``backend/orchestration.py:748``) calls ``storage.acquire_local_input(gs://?)`` to pull the whole clip back ****down**** so the validators can decode a **local** file, then ****re-hashes it**** (``compute_video_id``, line 751) even though the upload already produced the identical id.
3. ****Bucket ? worker scratch**** ? ``[worker] pulled ?`` for detection.

So for the 1.83 GiB clip: 1 stream up + ****2 full downloads**** + a redundant re-hash. The ~30 s gap is that CP re-download (silent ? only SPA job-polls log; see #520). Cheap wins that fall out of this issue's phase-placement work:

- ****Move validation onto the L4**** (it already downloads the clip for detect) ? the control-plane never pulls the full file, and the ~30 s gap disappears.
- ****Trust the upload's streamed `video\_id`**** in ``/api/process`` ? drop the redundant re-hash at ``orchestration.py:751``.

Broader direction (upload?stitch all on the L4, control-plane as a thin router, async copy-to-bucket for durability) tracked as the epic ****#524****.

10. user (2026-07-08T05:42:16.291Z)

``{"body": "## Problem\nObserving a run today means scraping Cloud Run stdout **after the fact** via `gcloud logging read`, then hand-correlating timestamps across **three** separate streams ? the control-plane service, the worker Job, and the in-process compiler ? and there is **no UI-side signal at all**.` Reconstructing the phase timeline for the 2026-07-08 live CUJ (upload ? validate ? dispatch ? detect ? ingest ? compile) required manually diffing log timestamps across those streams.
 \n \n Ask \nA ****debug/trace mode**** (config- or env-gated, ****off by default****) where every pipeline step ? and key UI actions ? emits a ****structured event**** to a ****low-latency sink****, so a run can be observed live and replayed:
 \n \n ****Server phases:**** upload (per-part / complete), validation (per-validator wall + verdict), dispatch (job/exec id + cold-start), worker (pull, skip-validation, WASB phase), ingest, compile (per-segment trim, concat, mirror-upload).
 \n ****UI events:**** upload start/complete, Process click, poll states, Compile click, download.
 \n ****Event shape:**** ``{ts, run_id, video_id, phase, step, machine, wall_ms, meta}``.
 \n

```

**Sink (configurable):**\n - GCS ? one JSONL per run at
`gs://\u003coutput_root\u003e/_debug/\u003cvideo_id\u003e/trace.jsonl` (cheap, durable,
replayable), and/or\n - a lightweight collector / Pub/Sub topic for **live** viewing (e.g. an
`/api/debug/trace/\u003cvideo_id\u003e` SSE stream, or push to an existing observability
service).\n- Gate behind e.g. `observability.debug_trace: true` (+ sink target); never on by
default (PII / volume).\n\n## Why now\nThe live CUJ
(`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md` §9) had to be timed **by hand** across control-plane + worker + compiler
logs; a first-class per-run trace (server **and** UI) would make the whole phase breakdown a
single artifact and enable low-latency live observation.\n\n_Filed from the 2026-07-08 live-CUJ
observability gap._\n", "comments": [], "state": "OPEN", "title": "Observability: step-level debug
tracing (server + UI) to a low-latency sink (bucket/service)"}

```

11. user (2026-07-08T05:42:25.512Z)

```

522 [MERGED] docs/validation-latency-live-cuj-2026-07-08 ? docs(runlogs): live SPA CUJ closes
the 2 owner-gated validation-latency cells (§9)
519 [MERGED] docs/validation-latency-closure-512-513 ? docs(runlogs): validation-latency closure
report (#512 + #513)
518 [MERGED] feat/validation-result-cache ? feat(validation): cache validation verdict by
video_id to skip re-validation on reupload
517 [MERGED] docs/runlog-cloud-run-exec-status-fastfail ? docs(runlogs): live smoke of the
crash-before-marker fast-fail (#515)
516 [MERGED] chore/revendor-hosted-compute-picker ? chore(ui): re-vendor SPA bundle with the
hosted compute-picker cloud default
515 [MERGED] fix/cloud-run-exec-status-fastfail ? fix(cloud-run): fail fast when a worker
execution terminates without a done-marker
513 [MERGED] fix/worker-skip-revalidation-on-dispatch ? fix(serving): worker skips redundant re-
validation on the cloud dispatch path
512 [MERGED] perf/validation-shared-decode-pass ? perf(validators): decode
scene_cut/blur/relevance frames in one shared pass
511 [MERGED] fix/hosted-local-detection-guardrail ? fix(serving): fail fast when compute=local
forces weightless in-process detection
510 [MERGED] docs/506-deploy-report-addendum ? docs(runlog): DEPLOY_REPORT addendum ? control-
plane redeployed to the forwarding image (#506)
509 [MERGED] chore/506-release-hardening ? chore(serving): forward WASB decode/tuning knobs to
worker + release hardening (#506)
508 [MERGED] feat/506-serving-nvdec-default ? feat(serving): default cloud-serving to
nvdec_resident decode (#506)
507 [MERGED] feat/506-gpu-resident-nvdec ? GPU-resident WASB decode behind a default-OFF
decode_backend flag (#506)
504 [MERGED] docs/runlog-alpha-all1v1-2026-07-06 ? docs(runlog): alpha-tester "All 1 v 1" cloud-
serving run ? timeout diagnosed + downsample fix proven
503 [MERGED] fix/wf294-detector-failure-not-silent ? fix(worker): surface detector timeout/abort
as failed, not a silent 0-rally "success"

```

12. user (2026-07-08T05:42:25.682Z)

No files found

13. user (2026-07-08T05:42:55.011Z)

```

origin/HEAD -> origin/master
origin/master
origin/docs/validation-latency-live-cuj-2026-07-08
origin/feat/validation-result-cache
origin/docs/runlog-cloud-run-exec-status-fastfail
origin/fix/cloud-run-exec-status-fastfail
origin/chore/revendor-hosted-compute-picker
origin/fix/hosted-local-detection-guardrail
===

```

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (§9) (#522)

14. user (2026-07-08T05:42:55.185Z)

```
1      # Session Handoff
2
3      ## You are here
4
5      This checkout is on `master` at `bf97700` after a successful `git pull --ff-only`
6      on 2026-07-08. The top-level workspace folder is not a Git repo; the relevant
7      engine repo is `badminton-highlight-indexer/`. There are pre-existing untracked
8      local files `score_golden_gate.py` and `score_local.py`; do not stage or delete
9      them.
10
11     A planning pass for the optional cheap SD reel request is complete. Tracking bug:
12     https://github.com/Khelsutra/badminton-highlight-indexer/issues/523
13
14     ## Next steps / open threads
15
16     1. Engine PR for #523: add an opt-in compile rendition field (`original` default,
17        `sd` optional), persist it through `jobs.params`, and make `_execute_compile`
18        call `VideoCompiler.compile_highlights(...)` exactly once using the requested
19        profile.
20     2. SPA PR in sibling `khelsutra/web`: expose the SD option near Build, send the
21        compile rendition only when selected, and use a distinct `-sd.mp4` filename.
22     3. Bundle follow-up: rebuild/import the SPA snapshot into `backend/ui` after the
23        source SPA PR lands, if bundling remains separate.
24     4. Link each PR back to #523 and add verification notes/artifacts there.
25
26     ## Ramp-up kit
27
28     - `backend/main.py::CompileRequest` and `backend/main.py::compile_highlights`
29     - `backend/orchestration.py::_resolve_compile` and `::_execute_compile`
30     - `backend/infrastructure/database_jobs.py::create_compile_job`
31     - `backend/pipeline/stitcher/compiler.py::VideoCompiler`
32     - `tests/test_api_endpoints.py` compile tests
33     - `tests/test_compiler.py`
34     - Sibling repo `../khelsutra/web/src/api/client.ts`
35     - Sibling repo `../khelsutra/web/src/App.tsx`
36     - Sibling repo `../khelsutra/web/src/components/SelectionFooter.tsx`
37
38     ## Key decisions
39
40     - SD is a replacement compile mode, not an additional output: if selected, build
41       and deliver only the SD reel.
42     - Backward compatibility matters: old compile payloads should behave as original
43       quality.
44     - Use the existing async compile job shape; no DB migration is needed because
45       compile-specific params already live in `jobs.params`.
46     - Apply the SD profile before any per-rally ffmpeg trim so temporary clips are
47       consistently encoded and the final concat can remain stream-copy.
48     - Prefer typed, tunable stitching SD settings over hard-coded encode constants.
49
```

15. user (2026-07-08T05:44:49.575Z)

Workflow launched in background. Task ID: wf1xxnet5

Summary: Map upload/orchestration/compute/storage/config/validation subsystems + docs conventions for the #524 L4-primary design

Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-

indexer\96963dab-d38f-488c-af44-530ddbefe1\subagents\workflows\wf_00bebd3c-82f

Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1\workflows\scripts\map-524-pipeline-topology-wf_00bebd3c-82f.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-

indexer\96963dab-d38f-488c-af44-530ddbfeed1\workflows\scripts\map-524-pipeline-topology-wf_00bebd3c-82f.js"} to iterate without resending the script.)
Run ID: wf_00bebd3c-82f
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claude\projects\C--Users\avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\workflows\scripts\map-524-pipeline-topology-wf_00bebd3c-82f.js", resumeFromRunId: "wf_00bebd3c-82f"}) ? completed agents return cached results (cached results may themselves be empty ? inspect journal.jsonl before assuming there is something to recover).

You will be notified when it completes. Use /workflows to watch live progress.

16. assistant (2026-07-08T05:44:58.748Z)

While the mapping workflow runs, let me read the core evidence and pinpoint code myself, and set up the isolated worktree.

17. user (2026-07-08T05:45:01.278Z)

```
690         provenance["job"],
691         provenance["region"],
692         result.outputs_uri,
693         video_id,
694     )
695     return (
696         {
697             "video_id": video_id,
698             "status": "success",
699             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
700             '{seg_name}').",
701             "results": [r.model_dump() for r in val_results],
702             "play_segments": segments,
703             "compute": provenance,
704         },
705         True,
706     )
707
708     def _execute_processing(
709         config=None, db=None, req=None, progress=None
710     ) -> Dict[str, Any]:
711         """The full hash ? validate ? segment ? report pipeline for one video.
712
713         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714         module-level ``global_config``/``db_instance`` (backward-compatible with existing
715         tests that pass only ``req``). Route handlers always pass explicit params; older
716         test code that calls ``_execute_processing(req)`` still works.
717
718         ``progress`` is an optional callable(stage: str) used to surface coarse job
719         progress.
720
721         Raises on unexpected internal errors ? the caller records those as a job failure
722         """
723         import backend.main as _main # call-time seam: main's globals/model stay patchable
724
725         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
726         # shift positional args ? req arrives in position 0 (config), progress in position 1
727         (db) =
728         if config is not None and not isinstance(config, AppConfig):
729             # Old signature: _execute_processing(req, progress=None)
730             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
731                 req = db
732                 db = None
733             elif req is None and isinstance(config, _main.ProcessRequest):
734                 req = config
735                 config = None
```

```

733     # Resolve config/db from module globals when not passed explicitly
734     if config is None:
735         config = _main.global_config
736     if db is None:
737         db = _main.db_instance
738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
743     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
744     # req.video_path (the source URI) is kept for the DB record + report; only the file-
745     # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""
747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
776     # "in_process" once the local segmenter is reached, set on the response as
777     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
778     # fail).
779     compute_actual: Optional[str] = None
780     response: Optional[Dict[str, Any]] = None
781     try:
782         if (
783             existing_video
784             and existing_video.get("status") == "processed"
785             and not req.force
786         ):
787             cached_segments = db.query_play_segments(video_id, {})
788             if cached_segments:
789                 logger.info(
790                     "[API] Reusing cached segmentation for video_id=%s; force=true
791                     bypasses cache.",
792                     video_id,

```

```

791         )
792         response = _cached_process_payload(
793             video_id,
794             existing_video.get("validation_results", []),
795             cached_segments,
796             getattr(req, "compute", None),
797         )
798         return response
799
800     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801     #     UNLESS this exact content already has a cached verdict under the current
validator config.
802     notify("validating")
803     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808     # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809     # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810     # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811     # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812     # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813     # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814     val_fingerprint = registry.config_fingerprint(config)
815     # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816     # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817     # failing the job. The cache is a pure speedup ? never a new failure mode.
818     reused_validation = False
819     if not req.force:

```

18. user (2026-07-08T05:45:01.537Z)

```

1:# DEPLOY REPORT ? cloud-serving, validation-latency closure (#512 shared-decode + #513 worker
skip-revalidation), 2026-07-07
5:## 1. TL;DR
18:## 2. Resources
24:## 3. Exact commands
26:# --- Provenance: prove the live image carries both fixes ---
32:# --- #513 proof: worker A/B on the LIVE image (Video 1 already staged in the serving bucket)
---
34:# Run A ? WITH --skip-validation (exactly what orchestration._maybe_dispatch_remote emits on
dispatch):
37:# Run B ? WITHOUT --skip-validation (worker re-validates with the NEW #512 validators ? still
blur-rejects):
41:# --- #512 timing: bracket "pulled ? -> in.mp4" ? "validation FAILED/SKIPPED" ? "Phase 2
(WASB) completed" ---
45:# --- #512 off-box A/B (isolated clones; run_all decodes the SAME sample indices in both
trees) ---
51:## 4. Run results
53:### 4a. Video 1 (5312x2988 10-bit HEVC, 3019 frames) ? the clip that motivated #512/#513

```

70:### 4b. Off-box #512 A/B ? codec/resolution breadth (16-core local box, cv2 4.13.0, warm, `run\_all`)
84:### 4c. Control-plane projection (2-vCPU)
87:### 5. Rollout state
92:### 6. Honest limits / not-verified
99:### 7. Cost
102:### 8. Closure verdict
105:### 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)

19. user (2026-07-08T05:45:07.693Z)

105 ## 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)
106
107 The two ? cells in \$4a (the **live control-plane validation wall** and the **end-to-end reel**) were owner-gated on the Cloudflare-Access `POST /api/process`. On **2026-07-08** the owner drove one real SPA upload (compute=**Cloud**) end-to-end, so both are now measured on live serving infra. Mirrors the gpu-resident report's "\$5 Update" append ? the body above is unchanged; this section records what the live run added.
108
109 **Two honest deltas from the recipe** (state moved after this report's 2026-07-07 image cutover):
110 1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **live control-plane-00019-96f**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed 2026-07-07T17:06Z ? ~51 min after the `00018/8045f54e` cutover \$2 documents; the worker Job is on the same `:latest=c6393d27` digest (both surfaces match). It's an undocumented forward-bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the `00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
111 2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"** ? **1.83 GiB** (1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? **heavier** than Video 1's 1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different clip has a different absolute wall).
112
113 **Full phase breakdown (UTC 2026-07-08, rev `00019`):**
114
115 | # | Phase | Window | Wall | Machine | Notes |
116 |---|---|---|---|---|---|
117 | 1 | Upload (client?server, 80-part multipart) | 04:25:56 ? 04:28:51 | **~2m55s** | client uplink | 1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra |
118 | 2 | Accept ? validation start (stage) | 04:28:52 ? 04:29:22 | ~30 s | control-plane | stage upload to serving bucket |
119 | 3 | **Validation (#512 shared-decode)** ? | 04:29:22 ? 04:32:25 | **183 s (3m03s)** | control-plane 2 vCPU | one-pass scene_cut/blur/relevance; PASSED |
120 | 4 | Dispatch ? worker ready (cold start) | 04:32:25 ? 04:33:20 | ~55 s | Cloud Run Job | image pull + GCSFuse mount + torch/cuda init |
121 | 5 | **Worker skip-validation (#513)** ? | 04:33:20 ? 04:33:24 | ~4 s | L4 | `validation SKIPPED`, 0 re-validation |
122 | 6 | Detection / segment (WASB) | 04:33:24 ? 04:36:20 | **161.6 s** | L4 (cuda) | 15 candidate windows, 2561 visible pts |
123 | 7 | Ingest + finalize | 04:36:20 ? 04:36:24 | ~4 s | control-plane | intervals?DB, `status=success` |
124 | ? | (user reviews rallies, clicks Compile) | 04:36:24 ? 04:37:26 | ~62 s idle | ? | human-in-the-loop |
125 | 8 | **Compile / stitch** ? slow | 04:37:26 ? 04:50:21 | **~12m55s** | control-plane 2 vCPU | 11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s |
126
127 Server pipeline (process?success, excl. upload+idle): **7m32s**. Full CUJ (upload?reel): **~24m25s** wall.
128
129 **Cell 1 ? control-plane validation AFTER (live):** **183 s (3m03s)** on rev `00019`, PASSED ? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected **2?3 min** band on a clip **heavier** than Video 1 (pre-#512 per-sample seeks were the ~6m52s-class baseline). Evidence: `Running validations 04:29:22.789` ? Video passed checks

```

04:32:25.780`.
130
131    **Cell 2 ? end-to-end reel (live):** `/api/compile` stitched **11 of the 15** detected
rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khelestra-
rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved
04:50:10 **"Dynamic highlight reel saved (581.8 MB)"**, mirrored 04:50:21, served via
`/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch
? per-clip trim+watermark ? concat ? mirror-upload, all exercised.
132
133    **Bonus ? #513 proof upgraded** from the $6 dispatch-faithful *direct exec* to a genuine
**control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted
`--skip-validation` on a live SPA run and the worker honored it (0 re-validation).
134
135    **? Finding ? stitch is the scaling bottleneck (follow-ups filed).** The compile ran
**~13 min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a
full 4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized
by `_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't
scale past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make
phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a
low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).
136
137    **Cost (heuristic):** one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55**
(rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ?
**~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **<
$0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but
the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.
138
139    **Net:** both owner-gated cells are now live-measured on rev `00019` ? validation wall
**183 s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass
validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The
Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).
140

```

20. user (2026-07-08T05:45:28.754Z)

```

Exit code 128
Preparing worktree (new branch 'feat/524-l4-primary')
branch 'feat/524-l4-primary' set up to track 'origin/master'.
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md: Filename too long
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_validation-latency_2026-07-07.md: Filename too long
fatal: Could not reset index file to revision 'HEAD'.

```

21. user (2026-07-08T05:45:33.699Z)

```

__pycache__
artifacts
backend
backend.db
CLAUDE.md
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
README.md
requirements.txt
requirements-trackers.txt
ruff.toml

```

```
run_all_tests.py
run_eval.py
run_regression.py
score_golden_gate.py
score_local.py
scratch
scripts
SESSION_HANDOFF.md
tests
tools
training
Python 3.13.13
fastapi          0.136.1
mypy             2.1.0
mypy_extensions 1.1.0
pydantic         2.13.4
pydantic_core    2.46.4
pytest          9.0.3
pytest-cov       7.1.0
ruff             0.15.16
```

22. assistant (2026-07-08T05:45:50.655Z)

Windows long-path failure ? the scratchpad path is too deep. Let me clean up and retry at a shorter session-tagged path with `core.longpaths` on.

23. user (2026-07-08T05:45:58.752Z)

Exit code 1

Deleted branch feat/524-l4-primary (was bf97700).

```
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/pr465': Permission denied
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
error: failed to delete '.git/worktrees/pr467': Permission denied
error: failed to delete '.git/worktrees/pr469': Permission denied
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete '.git/worktrees/wt-docs': Permission denied
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete '.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete '.git/worktrees/wt-review-492-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
```

```
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
```

24. assistant (2026-07-08T05:46:14.898Z)

Pre-existing stale worktree metadata from past sessions (permission-locked) ? not mine to force-delete. Let me verify my broken one is gone and retry at a short path:

25. user (2026-07-08T05:46:18.951Z)

```
---
Preparing worktree (new branch 'feat/524-l4-primary')
branch 'feat/524-l4-primary' set up to track 'origin/master'.
HEAD is now at bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-
latency cells ($9) (#522)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
feat/524-l4-primary
```

26. assistant (2026-07-08T05:46:39.870Z)

Worktree ready. Let me set up task tracking for this multi-phase epic, then probe the Billing Catalog API for real GPU pricing (design-doc input).

27. user (2026-07-08T05:46:47.017Z)

```
{
  "services": [
    {
      "name": "services/0017-8C5E-5B91",
      "serviceId": "0017-8C5E-5B91",
      "displayName": "OpenLogic CentOS 7.8 (v20200922) - Security Hardened",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/002A-FAF7-9793",
      "serviceId": "002A-FAF7-9793",
      "displayName": "Single node file server",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/0042-8437-FF55",
      "serviceId": "0042-8437-FF55",
      "displayName": "Adverity",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/005A-0B51-C56E",
      "serviceId": "005A-0B51-C56E",
      "displayName": "Metabase Data Visualization & BI Platform",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/0069-3716-5463",
      "serviceId": "0069-3716-5463",
      "displayName": "Qubole Open Data Lake Platform",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/008F-652C-F842",
      "serviceId": "008F-652C-F842",
      "displayName": "Nventify Imagizer",
      "businessEntityName": "businessEntities/GCP"
    }
  ]
}
```

```

    },
    {
      "name": "services/0094-29B1-96E7",
      "serviceId": "0094-29B1-96E7",
      "displayName": "Live Stream API",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/0095-7665-C074",
      "serviceId": "0095-7665-C074",
      "displayName": "CIS (Center for Internet Security) CIS Microsoft Windows Server 2008 R2
Benchmark v3.0.1 Level 1",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/00C7-CE6D-B98F",
      "serviceId": "00C7-CE6D-B98F",
      "displayName": "TigerGraph Inc. TigerGraph Developer Edition 2-6-3",
      "businessEntityName": "businessEntities/GCP"
    },
    {
      "name": "services/00E4-BD19-6904",
      "serviceId": "00E4-BD19-6904",
      "displayName": "Commvault Backup & Recovery  BYOL",

```

28. assistant (2026-07-08T05:46:59.210Z)

Billing API works. Let me create the task list and kick off the SKU pricing fetch in the background.

29. user (2026-07-08T05:47:01.090Z)

Task #1 created successfully: Map #524 subsystems (workflow wf1xxnet5)

30. user (2026-07-08T05:47:02.952Z)

Task #2 created successfully: Gather GCP pricing for topology cost model

31. user (2026-07-08T05:47:05.342Z)

Task #3 created successfully: Write design doc: L4-primary topology (A/B/C)

32. user (2026-07-08T05:47:06.592Z)

Task #4 created successfully: Win #2: trust streamed video_id, drop re-hash

33. user (2026-07-08T05:47:15.095Z)

Task #5 created successfully: Win #1: validation-on-L4 via phase_placement knob

34. user (2026-07-08T05:47:17.360Z)

Task #6 created successfully: Run gates: full suite, ruff, mypy, leak_scan, md_links, coverage

35. user (2026-07-08T05:47:18.863Z)

Task #7 created successfully: Adversarial review workflow on the diff, then PR

36. user (2026-07-08T05:47:24.366Z)

Command running in background with ID: b71jz7y77. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\tasks\b71jz7y77.output. You will be

notified when it completes. To check interim output, use Read on that file path.

37. user (2026-07-08T05:47:33.571Z)

```
94:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
258:class ProcessRequest(BaseModel):
277:    # AnnotatePanel(). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
    verbatim.
278:    video_id: str
289:    # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
297:    video_id: str
305:    video_id: str
439:    "current video" label actually read: ``video_id`` (the MD5 join key = `id`), a
    friendly
448:        "video_id": row.get("id"),
461:    (``video_id`` + a friendly `match_name` derived from the filename) so the library
    shows real
482:    does NOT track a video_id?reel association today (a per-video listing would be a
    phantom), so
512:    """"Stream a compiled reel by its bare filename (no video_id ? reels are flat in
    output_dir).
549:def _source_cache_lock(video_id: str) -> threading.Lock:
551:    return _source_cache_locks.setdefault(video_id, threading.Lock())
554:def _source_cache_path(video_id: str, input_ref, cfg) -> str:
559:    name = safe_output_filename(f"{video_id}{ext}")
563:def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:    cache_path = _source_cache_path(video_id, input_ref, cfg)
568:    lock = _source_cache_lock(video_id)
591:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
596:    if video_id in _proxy_inflight:
598:        _proxy_inflight.add(video_id)
605:        src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608:        logger.warning("proxy: warm failed for %s: %s", video_id, e)
611:        _proxy_inflight.discard(video_id)
614:@app.get("/api/source/{video_id}")
615:def get_source(video_id: str, request: Request):
621:    rec = request.app.state.db.get_video(video_id)
628:    proxy_name = safe_output_filename(f"{video_id}.mp4")
655:    local = _cached_remote_source(video_id, input_ref, cfg)
667:    target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
677:    ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?
    so it joins
681:    Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
    as a bare
691:    name = safe_output_filename(f"{req.video_id}.rallies.csv")
693:    raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
707:    "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
720:def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721:    """"The durable, MD5-keyed destination inside the chosen medium:
    ``<medium>/<video_id><ext>``.
722:    return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
753:    MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
```

[Showing results with pagination = limit: 40]

38. user (2026-07-08T05:47:34.597Z)

```
14: returns the staged URI (inside the #465 input jail by construction ? it is under
15: ``input_root``) plus the streamed md5 as ``video_id``.
43:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
77:    return upload_dir, max_bytes, part_bytes, exts
88:    return {
104:        return None
108:        return None
110:    return {"root": input_root, "config": cfg}
```

```

117:         return
132:         return
146:     return unicodedata.normalize("NFKC", filename).strip().rstrip(" ")
185:         "hasher": hashlib.md5(),
191:         return {
210:     return {
279:     return {
287: def upload_complete(upload_id: str, req: UploadCompleteRequest):
288:     """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
300:         # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301:         # part IS compute_video_id's digest (same bytes, same algorithm), so /api/process
314:         return {
316:             "video_id": st["hasher"].hexdigest(),
329:     video_id = compute_video_id(final_path)
330:     return {
332:         "video_id": video_id,
346:     return {"status": "aborted", "upload_id": upload_id}

```

39. user (2026-07-08T05:47:53.340Z)

```

258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269         compute: Optional[str] = None
270         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272         force: bool = False
273
274
275     class AnnotationsRequest(BaseModel):
276         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278         video_id: str
279         csv: str
280
281
282     class AssetRequest(BaseModel):
283         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285         #       local working copy for the run), OR
286         #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
287         #       registered as-is, no copy (D14b).
288         #   medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by

```

```

289         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290         uploaded_path: Optional[str] = None
291         source_uri: Optional[str] = None
292         medium_uri: str
293         sport: Optional[str] = None
294
295
296     class QueryRequest(BaseModel):
297         video_id: str
298         server: Optional[str] = None
299         receiver: Optional[str] = None
300         duration_min: Optional[float] = None
301         event_type: Optional[str] = None
302
303
304     class CompileRequest(BaseModel):
305         video_id: str
306         segment_ids: List[int]
307         output_filename: str

```

40. user (2026-07-08T05:47:55.516Z)

```

assets
index.html
THIRD_PARTY_NOTICES.md
ui_version.json

```

41. user (2026-07-08T05:48:02.356Z)

```

index-BjemzRyy.css
index-kJhH4dVE.js
noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
noto-sans-kannada-kannada-wght-normal-Duwb-P8d.woff2
noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
video_path:l

```

42. user (2026-07-08T05:48:16.255Z)

```

{video_path:l, locale:r, compute:o}
===
2

```

43. user (2026-07-08T05:48:19.923Z)

```

    611 skus_cloudrun.jsonl
    31588 skus_gce.jsonl
    32199 total
NVIDIA RTX Pro 6000 GPU with zonal redundancy in asia-southeast1 | second | $0.000682960
NVIDIA L4 GPU with zonal redundancy in asia-southeast1 | second | $0.000349080
NVIDIA L4 GPU with no zonal redundancy in asia-southeast1 | second | $0.000224040
NVIDIA RTX Pro 6000 GPU with no zonal redundancy in asia-southeast1 | second | $0.000438260

```

44. user (2026-07-08T05:48:32.874Z)

```

=== CLOUD RUN cpu/mem (asia-southeast1) ===
Worker Pools Memory in asia-southeast1 [Compute] | gibibyte second | $0.000001482
Services CPU Tier 2 (Request-based billing) [Compute] | second | $0.000033600
Instances Memory in asia-southeast1 [Compute] | gibibyte second | $0.000002316
Services Min Instance CPU Tier 2 (Request-based billing) [Compute] | second | $0.000003500
Services CPU (Instance-based billing) in asia-southeast1 [Compute] | second | $0.000021600

```

Services Min Instance Memory Tier 2 (Request-based billing) [Compute] | gibibyte second | \$0.000003500
Services Memory Tier 2 (Request-based billing) [Compute] | gibibyte second | \$0.000003500
Services Memory (Instance-based billing) in asia-southeast1 [Compute] | gibibyte second | \$0.000002400
Jobs CPU in asia-southeast1 [Compute] | second | \$0.000021600
Jobs Memory in asia-southeast1 [Compute] | gibibyte second | \$0.000002400
Instances CPU in asia-southeast1 [Compute] | second | \$0.000000324
Worker Pools CPU in asia-southeast1 [Compute] | second | \$0.000013492

45. user (2026-07-08T05:48:36.804Z)

=== GCE G2 / L4 / PD (asia-southeast1) ===
Balanced PD Capacity in Singapore | gibibyte month | \$0.110000000
Balanced PD Instant Snapshot data storage in Singapore | gibibyte month | \$0.110000000
Committed Use Discount Premium for G2 Custom Instance Core running in Singapore | hour | \$0.000693629
Committed Use Discount Premium for G2 Custom Instance Ram running in Singapore | gibibyte hour | \$0.000081260
DWS Defined Duration G2 Core running in Singapore | hour | \$0.030827960
DWS Defined Duration G2 Custom Core running in Singapore | hour | \$0.032369350
DWS Defined Duration G2 Custom Ram running in Singapore | gibibyte hour | \$0.003792173
DWS Defined Duration G2 Ram running in Singapore | gibibyte hour | \$0.003611593
G2 Custom Instance Core running in Singapore | hour | \$0.032369360
G2 Custom Instance Ram running in Singapore | gibibyte hour | \$0.003792173
G2 Instance Core running in Singapore | hour | \$0.030827960
G2 Instance Ram running in Singapore | gibibyte hour | \$0.003611593
Nvidia L4 GPU attached to DWS Defined Duration VMs running in Singapore | hour | \$0.690921600
Nvidia L4 GPU attached to Spot Preemptible VMs running in Singapore | hour | \$0.414600000
Nvidia L4 GPU running in Singapore | hour | \$0.690921600
Regional Balanced PD Capacity in Singapore | gibibyte month | \$0.220000000
Regional Balanced PD Instant Snapshot data storage in Singapore | gibibyte month | \$0.220000000
Regional SSD backed PD Capacity in Singapore | gibibyte month | \$0.374000000
Regional Storage PD Capacity in Singapore | gibibyte month | \$0.088000000
Spot Preemptible G2 Custom Instance Core running in Singapore | hour | \$0.019420000
Spot Preemptible G2 Custom Instance Ram running in Singapore | gibibyte hour | \$0.002275000
Spot Preemptible G2 Instance Core running in Singapore | hour | \$0.018500000
Spot Preemptible G2 Instance Ram running in Singapore | gibibyte hour | \$0.002167000
SSD backed PD Capacity in Singapore | gibibyte month | \$0.187000000
Storage PD Capacity in Singapore | gibibyte month | \$0.044000000

46. user (2026-07-08T05:50:04.561Z)

Updated task #1 status

47. user (2026-07-08T05:50:04.709Z)

Updated task #2 status

48. user (2026-07-08T05:50:19.197Z)

CR job rate (L4+8cpu+32Gi): \$0.000474/s = \$1.705/hr
today detect exec ~222s: \$0.105
pipeline-on-L4 ~314s: \$0.149
+ idle during 175s upload: \$0.083
GCE g2-standard-4: \$0.872/hr on-demand (\$637/mo), \$0.523/hr spot (\$382/mo)
CR GPU svc warm (4cpu/16Gi): \$1.256/hr = \$917/mo
breakeven GCE-vs-CR duty cycle: 51%

49. user (2026-07-08T05:50:35.514Z)

deploy/gcp/README.md:180:./deploy/digitalocean/bootstrap.sh --gpu
venv + torch cu124 (confirm cuda True)

50. user (2026-07-08T05:50:36.190Z)

```
423: def _maybe_dispatch_remote(
424-     cfg,
425-     db,
426-     req: "ProcessRequest",
427-     video_id: str,
428-     seg_name: str,
429-     val_results,
430-     play_wm: int,
431-     cand_wm: int,
432-     notify,
433-) -> Optional[Tuple[Dict[str, Any], bool]]:
434-     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435-     ``deployment.compute_target
436-     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437-     intervals into
438-     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439-     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether the
440-     cloud job
441-     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the observability
442-     report
443-     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-process
444-     segmenter
445-     (**DEFAULT-OFF** : ``get_compute_backend`` returns ``None`` for every non-``cloud_run``
446-     target, so
447-     the in-process path stays byte-identical)."""
448-     from backend.compute import ComputeJobSpec, get_compute_backend
449-
450-     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
451-         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
452-         play_wm);
453-         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
454-         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
455-         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run" only
456-         when the
457-         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
458-         ingest
459-         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
460-         records the
461-         # intended backend, `requested` the picker choice, `dispatched` whether the remote
462-         job ran.
463-         return (
464-             {
465-                 "video_id": video_id,
466-                 "status": "failed",
467-                 "message": msg,
468-                 "results": [r.model_dump() for r in val_results],
469-                 "play_segments": [],
470-                 "compute": {
471-                     "path": "cloud_run" if ran else "not_run",
472-                     "requested": req.compute,
473-                     "target": "cloud_run",
474-                     "dispatched": ran,
475-                 },
476-             },
477-             ran,
478-         )
479-
480-     # SPA compute-picker (item 3): a per-request choice overrides the server's configured
481-     default.
482-     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded
483-     by
484-     # _cloud_available so we fail clearly rather than silently running local); None /
```

```

unknown ? default.
473-     choice = (req.compute or "").strip().lower()
474-     if choice and choice not in ("local", "cloud"):
475-         logger.warning(
476-             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477-             req.compute,
478-         )
479-         choice = ""
480-     if choice == "local":
481-         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482-         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed trajectory
detector
483-         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with a
cryptic
484-         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485-         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486-         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs in-
process
487-         # below exactly as before (the weights-present check is the escape hatch).
488-         if (
489-             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490-             and _cloud_available(cfg)
491-             and not _local_detector_weights_present(cfg, seg_name)
492-         ):
493-             return _failed(
494-                 f"this hosted server can't run detection locally ? the '{seg_name}' model
weights "
495-                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496-                 False,
497-             )
498-         return None # explicit "run on my machine" ? in-process regardless of the server's
target
499-     if choice == "cloud":
500-         if not _cloud_available(cfg):
501-             return _failed(
502-                 "cloud-serving was requested but this server isn't configured for it "
503-                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504-                 False,
505-             )
506-         compute = get_compute_backend(cfg, target_override="cloud_run")
507-     else:
508-         compute = get_compute_backend(
509-             cfg
510-         ) # server default (default-OFF unless cloud_run)
511-     if compute is None:
512-         return None # default-OFF ? run the in-process segmenter below (unchanged)
513-

```

51. assistant (2026-07-08T05:50:36.721Z)

You've reached your Fable 5 limit. Run /usage-credits to continue or switch models with /model.

52. user (2026-07-08T05:46:08.525Z)

<local-command-caveat>Caveat: The messages below were generated by the user while running local commands. DO NOT respond to these messages or otherwise consider them in your response unless the user explicitly asks you to.</local-command-caveat>

53. user (2026-07-08T05:46:08.525Z)

```
<command-name>/model</command-name>
  <command-message>model</command-message>
  <command-args>claude-opus-4-8</command-args>
```

54. user (2026-07-08T05:46:08.525Z)

```
<local-command-stdout>Set model to claude-opus-4-8</local-command-stdout>
```

55. user (2026-07-08T05:52:34.264Z)

```
<task-notification>
<task-id>wf1xxnet5</task-id>
<tool-use-id>toolu_01WQZH8mAskVxdmG3mhguirs</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\tasks\wf1xxnet5.output</output-
file>
<status>completed</status>
<summary>Dynamic workflow "Map upload/orchestration/compute/storage/config/validation subsystems
+ docs conventions for the #524 L4-primary design" completed</summary>
<result>{"map":["summary":"UPLOAD FLOW subsystem: the browser chunked-upload router
(backend/api/uploads.py) streams sequential ?95MB parts either into a local partial file
(appliance) or directly through a storage.open_writer into
gs://&lt;input_root&gt;/uploads/&lt;upload_id&gt;/&lt;filename&gt; (hosted, #480/#471 OOM fix ?
bytes never assemble on control-plane disk) while an incremental hashlib.md5 accumulates;
upload_complete returns {path, video_id(=streamed md5 hex), size_bytes}. The SPA (khelsutra/web)
then DISCARDS that video_id (only logs it) and submits POST /api/process with just {video_path,
locale, compute} ? ProcessRequest has no video_id field. The submit path re-derives the md5
cheaply via one GCS metadata stat (#484), probes the DB/bucket cache, atomically claims the job,
and on a miss the drain worker (_execute_processing) downloads the ENTIRE gs:// video back to CP
scratch, re-hashes it with compute_video_id, and runs the full validator suite ON THE CP before
dispatching DETECT to the L4 Cloud Run Job with --skip-validation (#513). That CP-side fetch+re-
hash+validate is exactly what #524 (a)/(b) targets: the streamed upload md5 is already
trustworthy, and validation could ride the L4 worker which re-fetches and re-hashes the same
bytes anyway.", "key_files":["path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/api/uploads.py", "role":"Chunked-upload router: init/part/complete/abort; the
#480/#471 stream-to-bucket design", "key_lines":"1-25: module docstring = the #480/#471 design
(parts stream through open_writer, instance memory bounds a CHUNK not the video); 52-58:
UploadInitRequest{filename,total_size_bytes}/UploadCompleteRequest{total_parts}; 61-77:
_upload_cfg reads live backend.main.global_config.security; 94-110: _staging_target ? remote
staging iff storage.input_backend!='local' AND input_root non-empty (same config /api/process
resolves against, so the staged URI is submittable verbatim); 149-214: upload_init ? ext/size
checks, remote: stage_uri=f\"{input_root}/uploads/{upload_id}/{name}\" + storage.open_writer +
hashlib.md5() in _uploads state (168-195); NotImplementedError?silent local fallback (177-179);
local: .partial-&lt;id&gt; under security.upload_dir + require_free_bytes 507 guard (196-209);
217-283: upload_part ? strict sequential index (409 at 225-229), remote path buffers the WHOLE
part in a bytearray (237-250) then one run_in_threadpool(writer.write)+hasher.update (251-254);
413-overflow aborts (270-274); 286-335: upload_complete ? remote: writer.close() finalizes,
returns {path: stage_uri, video_id: hasher.hexdigest(), size_bytes, next} (299-319; 300-303
states the streamed md5 IS compute_video_id's digest); local: os.replace + full compute_video_id
re-read (320-335); 113-137: _abort_upload closes writer + deletes staged
object\"}, {"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/main.py\", \"role\":\"/api/process endpoint + ProcessRequest model + CLI single-shot
path\", \"key_lines\":\"258-272: ProcessRequest fields = video_path, player_pool, max_frames,
segmenter, locale, compute('local'|'cloud'|None), force ? NO video_id field; 904-974:
process_video ? edge-auth (917), validate_input_path with
allowed_root=security.allowed_video_root + allowed_uri_root=storage.input_root (921-939, bucket-
URI tenant jail), resolve_input_ref (935), local-existence fast-fail on resolved key only (946),
segmenter check (953), jobs_api.submit_process_job (964), drain kick via
_get_executor().submit(_drain_due_jobs) (971-973); 1492-1593: CLI mode mirrors the same
acquire_local_input+compute_video_id+validate flow
(1506-1516)\"}, {"path\":\"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/api/jobs.py\", \"role\":\"The whole submit decision: #484 submit-time md5 + cache
probe + #448/#481 atomic dedup claim\", \"key_lines\":\"80-153: submit_process_job ? order is (1)
```

submit_time_video_id (98), (2) probe_process_cache unless force (99-101), (3) remote-input scratch 507 guard only on probe miss (102-103), (4) atomic claim (104), (5) claim winner realizes cache hit ? mark_job_done instantly, no GPU (111-128); 29-68: claim_process_job ? dedup keyed on (owner_id, video_path) NOT video_id (37-40); video_id persisted at CREATE so a restarted CP can re-associate bucket outputs (38-40, CP-rebuild §4.2); 71-77: _fetch_scratch_guard (tempdir 507)", {"path": "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py", "role": "Pipeline execution: the redundant re-hash, CP-side validation, cloud dispatch", "key_lines": "220-239: submit_time_video_id ? gcs-only, ONE metadata stat, md5Hash?hex, None for local/composite, never raises; 287-317: probe_process_cache ? DB then {output_root}/outputs/{video_id}/report.json; 320-395: cached_process_result ? per-video_id lock re-hydrate (bucket=truth, DB=cache); 366-368: legacy videos/&md5>/ source-recovery prefix; 741-763: THE RE-HASH ? notify('hashing') ? storage.acquire_local_input downloads the full gs:// video to CP scratch (748-750) ? video_id=compute_video_id(local_video) (751), even when the job row already carries the submit-time md5; 802-873: VALIDATION ON THE CP ? #518 verdict cache keyed (video_id, config_fingerprint) (814-843), registry.run_all_with_context (846), set_validation_cache (851-856), failure short-circuits before dispatch (858-873); 906-913: _maybe_dispatch_remote called AFTER validation passes; 423-705: _maybe_dispatch_remote ? compute picker (473-512, 'local' weights fast-fail 480-498), local-upload staging to gs://{input_root}/{video_id}/{basename} + req.video_path repoint + add_video upsert (523-579), out_root required (580-586), --set forwarding of indexing.skip_ai_handoff + wasb knobs (598-619), ComputeJobSpec(skip_validation=True) #513 (620-632), video_id divergence warn-and-keep-local (647-655), ingest+provenance (657-705); 920-923: in-process segmenter under _LOCAL_COMPUTE_LANE", {"path": "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py", "role": "Async drain loop (DB is the clock) + #471 restart-safe remote-job resume", "key_lines": "261-284: _drain_due_jobs claims via db.claim_due_job; 145-193: _run_claimed_job ? _execute_processing, then set_job_video_id from result (177-178), mark_job_done; 122-142: _job_to_request rebuilds ProcessRequest from the job row (video_path only ? no video_id is threaded into execution); 56-69: _cloud_dispatch_possible; 86-101: configure_drain / deployment.max_concurrent_remote_jobs; 334-490: resume_interrupted_remote_jobs + _resume_one_remote_job ? polls outputs/&md5>/report.json using the persisted submit-time video_id, never re-dispatches (proof the md5-at-submit contract already carries recovery weight)", {"path": "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py", "role": "CloudRunCompute dispatch shim: argv contract, done-marker poll, #515 fast-fail", "key_lines": "57-60: container forced to compute_target=local_gpu + detector_impl=native (never re-dispatches); 244-318: run_infer ? done_uri=f\"{out_root}/dispatch/{token}.done\" (252), argv=['infer', '--video-uri', '?', '--out-uri', '?', '--done-uri', '?'] +-segmenter +-skip-validation +-set k=v (253-269), bucket-poll with on_wait heartbeat (281-290), post-timeout rescue + cancel (291-295), outputs_uri=f\"{out_root}/outputs/{video_id}/\" from the marker's video_id (296-307); 88-101: execution_terminal_state advisory ? terminated-without-marker ? RemoteExecutionFailed fast-fail (#515); marker checked FIRST, stays sole success signal (320-345)", {"path": "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py", "role": "The L4 worker CLI (python -m backend.worker infer) ? where #524(a) validation would land", "key_lines": "244-299: cmd_infer ? anti-recursion dispatch guard (254-256), PULL storage.fetch to scratch/in.mp4 (265-267), IDENTITY video_id=compute_video_id(local_video) (271) ? ... (truncated 206895 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\\96963dab-d38f-488c-af44-530ddbeefed1\\tasks\\wf1xxnet5.output)</result>
<diagnostics>Per-agent results: C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\\96963dab-d38f-488c-af44-530ddbeefed1\\subagents\\workflows\\wf_00bebd3c-82f\\journal.jsonl ? one {"type": "result", ...} line per completed agent with its full return value.
If the result above is empty or unexpected, Read this file BEFORE diagnosing ? do not assume agents returned non-empty results.
To re-run with edited post-processing: Workflow({scriptPath: 'C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\\96963dab-d38f-488c-af44-530ddbeefed1\\workflows\\scripts\\map-524-pipeline-topology-wf_00bebd3c-82f.js', resumeFromRunId: 'wf_00bebd3c-82f'}) ? agents whose (prompt, opts) are unchanged replay from cache.</diagnostics>
<failures>[gaps:critic] failed: You've reached your Fable 5 limit. Run /usage-credits to continue or switch models with /model.</failures>
<usage><agent_count>9</agent_count><agents_done>8</agents_done><agents_error>1</agents_error><agents_skipped>0</agents_skipped><agents_empty_result>0</agents_empty_result><subagent_tokens>9403

```
54</subagent_tokens><tool_uses>262</tool_uses><duration_ms>464689</duration_ms></usage>
</task-notification>
```

56. assistant (2026-07-08T05:52:38.480Z)

8/8 mapping agents succeeded (the critic hit a model limit, but I have enough). Let me read the full map from the journal.

57. user (2026-07-08T05:52:43.125Z)

17 journal.jsonl

```
=====
?
=====
SUMMARY: The compute subsystem is the seam that decides WHERE the DETECT core runs.
`backend/compute/` defines a `ComputeBackend` ABC + a lazy factory (`get_compute_backend`) that
returns None (run in-process) for every `deployment.compute_target` except "cloud_run", in which
case it returns `CloudRunCompute` ? a dispatch shim that starts a Cloud Run GPU Job execution of
the SAME worker CLI (`python -m backend.worker infer ?`) with per-execution container-arg
overrides, then bucket-polls a tiny JSON done-marker for completion (the marker, not the Cloud
Run API, is the sole success signal; the Executions API is only an advisory fast-fail probe,
#515). The worker entrypoint (`backend/worker/__main__.py`, the rally-worker Job's ENTRYPOINT)
pulls the input from GCS, re-hashes it to md5 (`video_id`), optionally validates (skipped when
the CP passes `--skip-validation`, #513), runs the segmenter, pushes
`outputs/<video_id>/{intervals.csv,report.json}` and finally the done-marker. The control-plane
side (`backend/orchestration.py::maybe_dispatch_remote`) validates FIRST on the CP, dispatches
with skip_validation=True, polls, then ingests intervals.csv into its SQLite DB. There is NO
phase_placement knob today ? validation always runs on the CP (CPU) and only DETECT runs on the
L4; the worker CAN validate (default-on for the bare CLI) but its validation verdict/reasons are
never serialized back (report.json/marker carry no validator results or error message ? only
status+returncode), which i

=====
?
=====
SUMMARY: EVIDENCE + TESTS map for #524. Evidence = §9 (lines 105-139) of
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md: the 2026-07-08 live SPA CUJ on Cloud Run rev 00019 that measured every
phase of the upload?validate?dispatch?detect?ingest?compile pipeline and identified control-
plane stitch (~12m55s) + validation (183s) as the CPU bottlenecks #524 wants moved to the L4.
Tests = a fully-offline, fully-mocked pytest suite (no conftest.py anywhere ? each module builds
its own fixtures), run via run_all_tests.py (thin pytest wrapper), gated in
.github/workflows/ci.yml by ruff/mypy/leak_scan/check_md_links/import-smoke/full-suite-
with-70%-coverage-floor/golden-crossos/license-scan/build-artifact. Every area #524 touches
(uploads.py, orchestration processing+dispatch, compute/cloud_run, worker, validators, config
models) has a dedicated large test module with explicit contract docstrings.

=====
?
=====
SUMMARY: CONFIG SYSTEM (backend/config/): a typed Pydantic AppConfig with load precedence
"built-in model defaults < deployment-profile preset < config.json < env vars < worker --set"
(presets.py:7-9). The ONLY placement knob today is deployment.compute_target (Literal
""|local_gpu|cloud_run|cpu_mock) ? consumed solely by get_compute_backend(), which returns a
CloudRunCompute shim only for "cloud_run" and None (in-process) otherwise; it moves ONLY the
detect phase. Validation and compile always run on the control plane. Config reaches the L4
rally-worker Job three ways: (1) image-baked env (RALLY_DETECTOR_IMPL=native, WASB_*), (2) per-
execution CLI args built by CloudRunCompute (--set k=v overrides, --segmenter, --skip-
validation, plus two FORCED disarm overrides deployment.compute_target=local_gpu +
indexing.detector_impl=native), and (3) Job env vars that beat config (WASB_DECODE_BACKEND ? the
no-redeploy rollback lever). The worker Job has NO DEPLOYMENT_PROFILE so presets never apply
there; the control plane resolves config and forwards knobs as --set. The live control plane is
configured by deploy/cloudrun/gen_service_yaml.py which base64-bakes a config.json into the
```

service manifest wrapper (RALLY_APP_HOME=/tmp/apphome) + env DEPLOYMENT_PROFILE=cloud-serving/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT. A #521 phase_placement knob slots in as a new field on DeploymentConfig with canonical values pinned in presets.py and a parity test, exactly like compute_target.

=====

?

=====

SUMMARY: backend/storage/ is the URI-driven, fully SYNCHRONOUS pluggable storage layer (COMPUTE_DECOUPLED_SERVING M2+). A thin facade (backend/storage/__init__.py) lazily constructs one of five StorageBackend implementations (local, s3, gcs, gdrive=mounted-Drive-over-local-FS, gdrive-api) from StorageConfig, keyed by URI scheme. It is the seam every pipeline stage crosses: browser uploads stream INTO the bucket via open_writer (with an incremental MD5 that already yields video_id for free ? #480/#484 groundwork for #524(b)); the control plane resolves inputs via resolve_input_ref and materializes them via acquire_local_input (identity for local, download-to-fresh-scratch for remote); the cloud dispatch stages local uploads UP with a blocking storage.put; the L4 worker pulls with storage.fetch and pushes outputs/done-marker with per-file storage.put; compile re-fetches the source and mirrors the reel with a best-effort blocking put. There is NO async/background copy primitive anywhere ? the only 'durable while bytes arrive' mechanism is GCS open_writer (streaming BlobWriter), which the upload path already exploits; an 'async copy-to-bucket during segmentation' model would need a new background-put primitive (or reuse of the existing upload-time streaming) plus completion/failure semantics.

=====

?

=====

SUMMARY: VALIDATION is a CPU-only, pluggable gate that runs on the control-plane (CP) inside _execute_processing BEFORE segmentation. A global ValidationRegistry holds 6 validators registered in dependency order: format_check (ffprobe container probe, seeds context['ffprobe_meta']), resolution_fps_check, pts_continuity_check, then three frame-decoding FrameSamplingValidators ? scene_cut_check, blur_check, relevance_check ? which since #512 share ONE cv2 decode pass (frame_sampler.py). The verdict is a List[ValidationResult] persisted in three places: the videos.validation_results JSON column, the #518 validation_cache SQLite table (keyed by video_id = MD5 of file bytes + a config fingerprint), and the job result JSON ('results' key) the SPA polls. A FAIL sets videos.status='failed' and returns a status='failed' job result with a joined failure message; segmentation never runs. On the cloud_run dispatch path the CP is the authoritative gate: it validates locally, then sends ComputeJobSpec(skip_validation=True) so the worker's --skip-validation flag (#513) skips its own re-validation (the worker's config is NOT fully forwarded and once rejected a CP-passed clip). Validation needs the WHOLE local file (full-file ffprobe demux + frame samples up to the last frames) but zero GPU ? which is exactly why #524 wants a phase_placement knob to move it onto the L4 worker where the bytes already land.

=====

?

=====

SUMMARY: The DOCS+CONVENTIONS subsystem governing any new design work under docs/COMPUTE_DECOUPLED_SERVING/. The main design doc (README.md, ADR-014's project doc) locks 17 decisions (D1?D17) and tracks milestones M0?M12; TESTING_STRATEGY.md defines the 5-layer test/CI conventions; runlogs/ is the durable record of real cloud runs (naming + 8 required sections). PROJECT_DOC_TEMPLATE.md defines the mandatory structure/status-header for any new tracked design doc; DOC_STATUS.md defines lifecycle (DRAFT?IN PROGRESS?DONE?archived) + the \$2 archival due-diligence checklist. ADR-012 (Cloud Run+GPU scale-to-zero = THE serving model), ADR-013 (GCP-only compute), and ADR-014 (pure 3-stage core, profile-selected compute+storage, stitch-where-metered) are the ratified constraints that bound the #524 topology options. Critically for #524: the live system today stitches AND validates on the 2-vCPU control-plane (validation-latency runlog \$9), which DEVIATES from ratified D4/ADR-014 decision 4 ('in cloud-serving, STITCH runs on the Cloud Run job so it is metered') ? so a phase_placement/L4-primary design largely REALIZES an already-ratified decision rather than needing a new ADR, but it is bounded by D16(a) (Cloud Run JOBS, not Services-with-GPU), D7 (scale-to-zero, no warm pool), D16(b) (libx264 encode for parity; NVENC only behind a deferred config knob), D9 (Cloudflare Access edge auth), and D5 (parity enforced at WINDOW+STITCH byte-level).

=====

?
=====

SUMMARY: The orchestration processing path is the control-plane (CP) pipeline for one video: a queued job row is drained by backend/api/job_worker.py, reconstructed into a ProcessRequest, and run through backend/orchestration.py::_execute_processing = fetch-to-local ? full-file MD5 re-hash ? validator suite (with a video_id-keyed verdict cache, #518) ? segmenter dispatch (in-process under a 1-slot local-compute lane, or a Cloud Run L4 Job via _maybe_dispatch_remote with skip_validation=True) ? ingest intervals.csv back into SQLite via the same add_play_segment writes ? destroy-after-confirm finalize ? observability report + scratch cleanup.

_execute_compile is the sibling worker handler for reel stitching; _resolve_compile runs both at submit (fast 4xx) and in the worker (re-resolve). For issue #524: (a) validation today runs ONLY on the CP (the worker's own validation is explicitly skipped via --skip-validation after the #513 threshold-divergence incident); (b) the re-hash at orchestration.py:751 recomputes the MD5 the upload endpoint already streamed (uploads.py:316) and that the job row may already carry (persisted at claim, #484); (c) upload?stitch currently streams browser parts into the bucket, then the CP re-downloads the whole video to scratch just to hash+validate before dispatching the GPU job that downloads it a third time. There is no phase_placement knob anywhere in the repo yet (grep: 0 hits).

=====

?
=====

SUMMARY: UPLOAD FLOW subsystem: the browser chunked-upload router (backend/api/uploads.py) streams sequential ?95MB parts either into a local partial file (appliance) or directly through a storage.open_writer into gs://<input_root>/uploads/<upload_id>/<filename> (hosted, #480/#471 OOM fix ? bytes never assemble on control-plane disk) while an incremental hashlib.md5 accumulates; upload_complete returns {path, video_id(=streamed md5 hex), size_bytes}. The SPA (khelsutra/web) then DISCARDS that video_id (only logs it) and submits POST /api/process with just {video_path, locale, compute} ? ProcessRequest has no video_id field. The submit path re-derives the md5 cheaply via one GCS metadata stat (#484), probes the DB/bucket cache, atomically claims the job, and on a miss the drain worker (_execute_processing) downloads the ENTIRE gs:// video back to CP scratch, re-hashes it with compute_video_id, and runs the full validator suite ON THE CP before dispatching DETECT to the L4 Cloud Run Job with --skip-validation (#513). That CP-side fetch+re-hash+validate is exactly what #524 (a)/(b) targets: the streamed upload md5 is already trustworthy, and validation could ride the L4 worker which re-fetches and re-hashes the same bytes anyway.

Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

58. assistant (2026-07-08T05:53:06.785Z)

Rich map. Let me pull the detailed `key\_files`/`contracts`/`gotchas` for the four agents whose code I'll actually modify (orchestration, config, compute-worker, validation), plus read the real files for the re-hash and video_id threading.

59. user (2026-07-08T05:53:15.428Z)

Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

60. user (2026-07-08T05:53:22.493Z)

(Bash completed with no output)

61. assistant (2026-07-08T05:53:29.499Z)

```
import json
p = 'C:/Users/avidu/.claude/projects/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/96963dab-d38f-488c-af44-530ddbeefed1/subagents/workflows/wf_00bebd3c-82f/journal.jsonl'
want = {'map:orchestration', 'map:config', 'map:compute-worker', 'map:validation'}
out = []
for line in open(p, encoding='utf-8'):
    r = json.loads(line)
    if r.get('type') != 'result':
        continue
    lbl = r.get('label', '')
```

```

if lbl not in want:
    continue
val = r.get('value') or {}
out.append('\n' + '#' * 72 + '\n### ' + lbl + '\n' + '#' * 72)
out.append('-- KEY_FILES --')
for kf in val.get('key_files', []):
    out.append(f" {kf.get('path','')} :: {kf.get('role','')}")
    if kf.get('key_lines'):
        out.append(f"    lines: {kf['key_lines']}")
out.append('-- CONFIG_KNOBS --')
for k in val.get('config_knobs', []):
    out.append(' * ' + k)
out.append('-- CONTRACTS --')
for k in val.get('contracts', []):
    out.append(' * ' + k)
out.append('-- GOTCHAS --')
for k in val.get('gotchas', []):
    out.append(' * ' + k)
print('\n'.join(out))

```

62. user (2026-07-08T05:53:29.536Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\extract.py (file state is current in your context ? no need to Read it back)

63. user (2026-07-08T05:53:33.421Z)

(Bash completed with no output)

64. user (2026-07-08T05:53:42.368Z)

```

total lines: 17
0 keys= ['type', 'key', 'agentId'] type= started label= None
1 keys= ['type', 'key', 'agentId'] type= started label= None
2 keys= ['type', 'key', 'agentId'] type= started label= None
3 keys= ['type', 'key', 'agentId'] type= started label= None
4 keys= ['type', 'key', 'agentId'] type= started label= None
5 keys= ['type', 'key', 'agentId'] type= started label= None
6 keys= ['type', 'key', 'agentId'] type= started label= None
7 keys= ['type', 'key', 'agentId'] type= started label= None
8 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
9 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
10 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
11 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
12 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
13 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
14 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
15 keys= ['type', 'key', 'agentId', 'result'] type= result label= None
16 keys= ['type', 'key', 'agentId'] type= started label= None

```

65. user (2026-07-08T05:53:48.932Z)

```

v2:5b6da07d9dc8cd8614dd0a31d63526e281c0ea16e8231d01f08ad12f8f87b26a -> dict
v2:2af6f93938443ef9df9513eec36c8222da55a1262b35bc3712e2f0b4a891d854 -> dict
v2:61a175ad18cd6d946dde9c72a92b6d1026a5ac1934e0c9c53fca60228b17f733 -> dict
v2:ace3c4f9313b2dc1800acdb8d771d9f1968989ea7f5c501ea9ad47bf12a98119 -> dict
v2:25d3842b64e7cedf54676b2096b893eec11671ca5f0252b77cc8bb5d9f691372 -> dict
v2:a643decd71f660ee8b87783782128ff975ff7da74659dd20c718f954349ddc2e -> dict
v2:e747da219caad0e961388477eff877d3e8fe30e34009bcab8d4748191d13fb28 -> dict
v2:895e83ab80fdb3aba4bbaa7af5f528acedcc8b85ed4b62b957492e8a32a89bac -> dict

```

66. user (2026-07-08T05:54:01.349Z)

#####

COMPUTE-WORKER

#####

KEY_FILES:

- base.py :: ComputeBackend ABC + the job/result dataclasses + the #515 exception
 - 20-31: RemoteExecutionFailed (terminal-without-marker, #515); 34-51: ComputeJobSpec (video_uri, out_uri, segmenter, config_overrides tuple, skip_validation bool default False ? the #513 field, EXACT name `skip\_validation`); 54-68: ComputeResult (returncode 0/2/3 mirrors worker, 4 = poll timeout/unreadable marker; outputs_uri, video_id, report dict, execution name for provenance); 71-87: run_infer(
 - __init__.py :: Factory: the default-OFF seam selecting local-inline vs cloud dispatch
 - 42-97: get_compute_backend ? returns None unless target=="cloud_run" (67-68); target_override for the per-request SPA picker (62-66); 78-84: poll budget derived from deployment.remote_poll_max_wait_sec when caller passed DEFAULT_POLICY; 85-88: env coordinates CLOUD_RUN_JOB (default "rally-worker"), CLOUD_RUN_REGION (default "asia-south1"), GOOGLE_CLOUD_PROJECT
 - cloud_run.py :: CloudRunCompute: dispatch shim + bucket-poll + #515 fast-fail
 - 57-60: _DEFAULT_CONTAINER_OVERRIDES (--set deployment.compute_target=local_gpu + indexing.detector_impl=native ? anti-recursion); 63-101: CloudRunJobClient ABC (run_job/cancel_execution/execution_terminal_state advisory probe); 104-189: GoogleCloudRunJobClient (real SDK, lazy-import; container args override per execution at 135-142); 192-211: _classify_execution_state (pure; completion_time!=None
 - __main__.py :: The worker entrypoint that runs INSIDE the rally-worker Cloud Run Job (also the bare CLI)
 - 244-256: cmd_infer head ? load_config + --set overrides + startup checks; get_compute_backend guard (254-256) so a cloud target dispatches instead of running (174-214: _dispatch_remote maps RemoteExecutionFailed?rc 5, PolicyTimeout?rc 4); 264-268: PULL via storage.fetch ? '[worker] pulled' log (268); 271: re-hash compute_video_id(local_video); 282-291: #513 skip ? `if getattr(args, 'skip\_validation`
 - orchestration.py :: Control-plane: validate ? dispatch ? poll ? ingest; also the #524(b) re-hash
 - 423-705: _maybe_dispatch_remote ? picker local/cloud (473-512); local-upload staging to <input_root>/<video_id>/<basename> + db repoint (523-579); output_root required (580-586); config forwarding = ONLY indexing.skip_ai_handoff + indexing.wasb.{decode_backend,batch_size,prefetch_batches} (598-619; validation.* NOT forwarded); ComputeJobSpec(..., skip_validation=True) (626-632); run_infer + heartb
 - uploads.py :: Chunked upload that ALREADY streams the md5 (the video_id #524(b) wants to trust)
 - 168-195: init ? remote staging writer to <input_root>/uploads/<upload_id>/<name> + hashlib.md5() hasher; 252-254: hasher.update per part; 299-319: complete returns {path: stage_uri, video_id: streamed md5, size_bytes} ? 'the md5 accumulated per part IS compute_video_id's digest'; 320-335: local-staging fallback hashes the assembled file
 - jobs.py :: Submit hook: cheap-md5 cache probe before queueing
 - 80-146: submit_process_job ? submit_time_video_id (98) ? probe_process_cache (99-101) ? atomic claim/dedup (104-110) ? realize cache hit instantly (111-128) ? else queue; job row stores video_id when known
 - job_worker.py :: Async job drain ? how failures surface to the job today
 - 145-193: _run_claimed_job ? job.status is EXECUTION state only: 'done' even when the pipeline verdict is failed (verdict lives in result['status'], 149-152); mark_job_failed ONLY on a raised exception (191-193); 122-142: _job_to_request rebuilds ProcessRequest incl. compute picker; 261-278: _drain_due_jobs loop
 - execution_policy.py :: Bounded poll primitives used by the bucket-poll
 - 45-75: ExecutionPolicy defaults (poll_every_n_sec=10, max_wait_sec=1800 ? overridden to deployment.remote_poll_max_wait_sec=3900 by the factory); 41-42: PolicyTimeout; 152-201: poll_until (check() None=keep waiting; on_tick heartbeat swallowed on error)
 - models.py :: DeploymentConfig ? the config heartbeat #524(a)'s phase_placement would join
 - 454-483: profile (468), compute_target Literal '|local_gpu|cloud_run|cpu_mock (471), max_concurrent_remote_jobs default 4 (477), remote_poll_max_wait_sec default 3900.0 deliberately > the job's --task-timeout 3600 (478-483). NO phase_placement knob exists today
 - Dockerfile :: rally-worker Job image
 - 115-119: env WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS + RALLY_DETECTOR_IMPL=native + PYTHONUNBUFFERED=1 (live logs); 121-123: ENTRYPOINT ['python3','-m','backend.worker'] ? dispatcher supplies ALL per-execution args; no DEPLOYMENT_PROFILE set, so the worker never applies the cloud-serving preset itself
 - README.md :: Job creation contract
 - 32-36: gcloud run jobs create rally-worker ? --max-retries 0 --task-timeout 3600; 104: CP

```

env exports CLOUD_RUN_JOB/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT
CONFIG_KNOBS: [
  "deployment.profile (Literal '|'truly-local'|cloud-serving'|cpu-mock', default '|') \u2014
backend/config/models.py:468; preset 'cloud-serving' sets compute_target='cloud_run'
(backend/config/presets.py:52)",
  "deployment.compute_target (Literal '|'local_gpu'|cloud_run'|cpu_mock', default '|') \u2014
models.py:471; the ONLY value that activates the CloudRunCompute backend is 'cloud_run'
(backend/compute/__init__.py:67)",
  "deployment.remote_poll_max_wait_sec (float, default 3900.0) \u2014 models.py:483; consumed at
backend/compute/__init__.py:78-84 to stretch DEFAULT_POLICY.max_wait_sec (base poll cadence
poll_every_n_sec=10.0, execution_policy.py:50)",
  "deployment.max_concurrent_remote_jobs (int, default 4, ge=1 le=64) \u2014 models.py:477; drain
parallelism cap for concurrent cloud dispatches",
  "storage.output_root \u2014 required non-empty for any cloud dispatch
(orchestration.py:580-586); the bucket the worker writes outputs+marker to",
  "storage.input_root / storage.input_backend \u2014 where uploads stream/stage; a cloud
input_root is required to stage a CP-local upload for dispatch (orchestration.py:533-546)",
  "indexing.skip_ai_handoff, indexing.wasb.decode_backend
CONTRACTS: [
  "Dispatch encoding = container-args override on the existing Cloud Run Job
(run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)), NO env vars, NO GCS manifest:
`infer --video-uri <gs://\u2026> --out-uri <gs://output_root> --done-uri
<gs://output_root>/_dispatch/<uuid4-hex>.done [--segmenter <name>] [--skip-validation] --set
<k=v> \u2026` (cloud_run.py:253-269; Dockerfile ENTRYPOINT `python3 -m backend.worker`)."
  "#513 skip-revalidation flag: ComputeJobSpec.skip_validation (bool, default False,
backend/compute/base.py:51) \u2014 CLI flag `--skip-validation` (appended at
cloud_run.py:264-267; parsed at worker __main__.py:753-760; consumed via
getattr(args, 'skip_validation', False) at 282). Set True by the CP at orchestration.py:631.",
  "Done-marker (bucket object at <out_root>/_dispatch/<token>.done, written LAST by the worker):
JSON {\"video_id\": <md5>, \"returncode\": int, \"segment_count\": int, \"status\":
\"success\"|\"failed\"} (__main__.py:229-234). Missing returncode reads as 1; presence of the
object terminates the poll.",
  "Results manifest report.json (at <out_root>/outputs/<video_id>/report.json): {\"video_id\",
\"status\": \"success\"|\"failed\", \"segment_c
GOTCHAS: [
  "#513 origin incident (2026-07-07, comments at worker __main__.py:273-281 +
orchestration.py:620-625): CP validated a 5312x2988 clip at validation.min_blur_threshold=8.0,
worker re-validated at its default 20.0 and REJECTED after a successful GPU run. Root cause: the
worker Job has NO DEPLOYMENT_PROFILE and gets only the explicitly forwarded --set overrides
(orchestration.py:598-619) \u2014 validation.* config is NOT forwarded today. If #524 moves
validation ONTO the worker, this asymmetry reverses: worker-side validation would run at model-
default thresholds unless the CP forwards validation.* overrides (or the phase move ships config
forwarding).",
  "#515 deployment coupling: an OLD worker image argparse-rejects an UNKNOWN flag and dies before
writing any marker \u2014 RemoteExecutionFailed (rc 5 path / 'cloud dispatch error'). Adding any
new worker CLI flag for #524 (e.g. a validate phase flag) requires the image rollout BEFORE the
CP starts sending it, or every dispatch fast-fails. The fast-fail itself is advisory-only:
GoogleCloudRunJobClient.execution_terminal_state degrades to None on any error, so the marker
stays the sole success signal (cloud_run.py:88-101, 358-373).",
  "The done-marker write is BEST-EFFORT (never raises, __main__.py:240-241): a failed marker push
on an otherwise-Succeeded execution surfaces as RemoteExecutionFailed ('Succeeded with NO
completion marker'). A missing 'returncode' key in a present marker defaults to 1 = failure
(cloud_run.py:297-299); a present-but-empty marker still terminates the poll (430).",
  "NO validation verdict/error chan

```

```
#####
```

CONFIG

```
#####
```

KEY_FILES:

```

- models.py :: All typed config models; DeploymentConfig is the deployment section
  454-483: DeploymentConfig ? profile (468, Literal '|'truly-local|cloud-serving|cpu-mock),
compute_target (471, Literal '|'local_gpu|cloud_run|cpu_mock, default '|'),
max_concurrent_remote_jobs (477, default 4), remote_poll_max_wait_sec (483, default 3900.0).
202-295: IndexingConfig ? detector_impl (210, 'auto', env RALLY_DETECTOR_IMPL overrides),
skip_ai_handoff (289), default_segmenter (203, 'yolo

```

- presets.py :: Profile presets + canonical enum values + env auto-detect (suggest-only, D15)
- 27-28: PROFILES + COMPUTE_TARGETS tuples (canonical values, mirrored by the models.py Literal, drift pinned by tests/test_config_deployment.py:77-97). 32-36: COMPUTE_TARGET_FOR_PROFILE 1:1 map. 43-80: PROFILE_PRESETS ? 'cloud-serving' (51-73) sets compute_target=cloud_run, detector_impl=native, skip_ai_handoff=False, wasb{batch_size:64, prefetch_batches:4, decode_backend:nvdec_resident}, storage.b
- loader.py :: load_config: defaults < preset < config.json merge; profile resolution
- 53-79: _resolve_explicit_profile ? DEPLOYMENT_PROFILE env WINS over config.json deployment.profile; unknown env value warns+falls through. 128-236: load_config ? config file is JSON at backend/utils/app_home.py::default_config_path() (RALLY_APP_HOME-rooted, else ./config.json); 193-219: preset deep-merge + re-stamp of profile and canonical compute_target (an active profile with blanked compute_target)
- startup.py :: Per-profile boot coherence checks (M5) ? where a placement/target mismatch check belongs
- 240-297: cloud-serving branch ? STORAGE_INCOHERENT fatal (242-251), AI_HANDOFF_KEY_MISSING fatal (261-280), CREDITS_UNCONFIRMED/CLOUD_CPU_DEVICE warns. 195-197: no profile ? strict no-op. 314-344: enforce_startup_checks (server calls with include_exposure=True at main.py:1241 and sys.exit(1) on fatal; worker via worker/__main__.py:153-171 returns rc 2).
- __init__.py :: get_compute_backend ? THE consumption point of deployment.compute_target
- 62-68: target resolution (target_override param for per-request SPA 'cloud' choice; only 'cloud_run' returns a backend, everything else None=in-process). 78-83: remote_poll_max_wait_sec plumbed from config into ExecutionPolicy ? the cleanest precedent of a deployment.* field consumed at the seam. 85-88: Cloud Run coordinates from env: CLOUD_RUN_JOB (default 'rally-worker'), CLOUD_RUN_REGION (default)
- cloud_run.py :: CloudRunCompute ? how per-job config physically reaches the worker container
- 57-60: _DEFAULT_CONTAINER_OVERRIDES = ('deployment.compute_target=local_gpu', 'indexing.detector_impl=native') ? forced --set args that disarm recursive re-dispatch; a phase_placement knob needs the same disarm treatment. 253-269: argv build ? infer --video-uri/--out-uri/--done-uri [--segmenter] [--skip-validation] then --set for (*spec.config_overrides, *self._container_overrides). 135-141: real
- base.py :: ComputeJobSpec/ComputeResult contracts
- 34-51: ComputeJobSpec ? config_overrides: tuple[str,...] of raw '--set k=v' strings (43-44); skip_validation: bool = False (45-51, the #513 field ? closest precedent for a phase-move flag). 54-68: ComputeResult rc semantics (0 ok / 2 validation-or-config / 3 segmenter / 4 poll-timeout; CLI adds 5 for RemoteExecutionFailed).
- __main__.py :: Worker CLI ? how the container consumes config + overrides
- 245-246: cmd_infer loads config (defaults-only in the container; no config.json in the image) then _apply_overrides(config, args.set). 48-71: _coerce + _apply_overrides ? dotted-path setattr on the typed config (NOTE: nested models lack validate_assignment, so --set values on nested fields bypass Literal validation). 250-256: worker itself re-dispatches when compute_target=cloud_run (disarmed by t
- orchestration.py :: Control-plane phase sequencing + the --set forwarding pattern
- 741-856: _execute_processing ? hashing (748-751), then VALIDATION on the control plane (802-856, with the #518 verdict cache keyed by registry.config_fingerprint(config) at 814), then dispatch decision. 423-512: _maybe_dispatch_remote ? per-request compute='local'/'cloud' override handling; 506-512 get_compute_backend. 598-631: THE forwarding precedent ? control plane forwards resolved indexing.sk
- job_worker.py :: Drain concurrency + restart recovery read compute_target
- 56-69: _cloud_dispatch_possible (compute_target==cloud_run OR CLOUD_RUN_JOB+GOOGLE_CLOUD_PROJECT env + storage.output_root). 72-90: _resolve_drain_workers/configure_drain ? deployment.max_concurrent_remote_jobs consumed (#466); called at server startup main.py:1635. 359-361: resume path re-reads compute_target.
- gen_service_yaml.py :: Live control-plane deploy: bakes config.json into the service manifest
- 62-87: build_config ? the baked config.json: deployment{profile:cloud-serving, compute_target:cloud_run}, indexing{skip_ai_handoff:true, default_segmenter:wasb_hybrid (#479)}, storage{gcs, output_root=gs://khelsutra-rally-serving, input_root=<bucket>/videos}, security pins (max_part_mb=24 #480). 92-102: wrapper base64-decodes it to /tmp/apphome/config.json + exports RALLY_APP_HOME. 129-137: env bl
- Dockerfile :: Worker Job image ? env-baked config
- 115-119: ENV WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS + RALLY_DETECTOR_IMPL=native baked in. 121-123: ENTRYPOINT python3 -m backend.worker (dispatcher supplies all per-execution args). No config.json in the image; no DEPLOYMENT_PROFILE.
- README.md :: Deploy runbook ? knob-flow doc + version-skew rules
- 60-65: 'How the knob reaches the worker' ? preset is single source of truth, forwarded as --set. 67-78: WASB_DECODE_BACKEND Job env = no-redeploy override/rollback lever (env beats

forwarded --set). 79-90: version-skew warning ? a NEW worker CLI flag argparse-aborts an OLD image before the done-marker (roll worker-first; #513/#515). 32-39: rally-worker Job create flags (L4, --task-timeout 3600, GC

- native_wasb_runner.py :: Env-over-config precedence at the leaf (worker side)

108-133: NativeWasbConfig.from_indexing_cfg ?

env(WASB_REPO_DIR/WASB_PYTHON/WASB_WEIGHTS/WASB_DEVICE/WASB_SCRATCH/WASB_DECODE_BACKEND) each beat the corresponding indexing.wasb.* config value.

- test_config_deployment.py :: Parity pin: model Literal == presets tuples

77-97: asserts set(PROFILE_PRESETS)==set(PROFILES) and the compute_target Literal == {'', *COMPUTE_TARGETS} ? a new placement enum must add the same parity assertions.

CONFIG_KNOBS: [

- "deployment.profile = '' (Literal ''|truly-local|cloud-serving|cpu-mock; models.py:468; DEPLOYMENT_PROFILE env wins)",
- "deployment.compute_target = '' (Literal ''|local_gpu|cloud_run|cpu_mock; models.py:471 \u2014 the ONLY where-does-detect-run knob today)",
- "deployment.max_concurrent_remote_jobs = 4 (models.py:477; drain concurrency for cloud_run, #466)",
- "deployment.remote_poll_max_wait_sec = 3900.0 (models.py:483; must outlast the Job's --task-timeout 3600)",
- "indexing.detector_impl = 'auto' (models.py:210; auto=WSL, native=Linux GPU, stub=CPU mock; RALLY_DETECTOR_IMPL env overrides \u2014 trajectory_hybrid.py:93)",
- "indexing.skip_ai_handoff = False (models.py:289; cloud-serving preset False, live baked control-plane config pins true)",
- "indexing.default_segmenter = 'yolo_hybrid' (models.py:203; live control plane pins wasb_hybrid, #479)",
- "indexing.wasb.decode_backend = 'cpu' (models.py:131; cloud-serving preset 'nvdec_resident'; WASB_DECODE_BACKEND env beats all)",
- "indexing.wasb.batch_size / prefetch_batches = None (models.py:118/121; preset 64/4; forwarded as --set only when set)",
- "indexing.wasb.device = 'cuda' (models.py:91; WASB_DEVICE env overrides)",
- "stora

CONTRACTS: [

- "Config file: a single JSON config.json at default_config_path() = \$RALLY_APP_HOME/config.json when set, else ./config.json (backend/utils/app_home.py:30-56). No YAML, no .env-based config values (only env VAR overrides at consumption sites).",
- "Env vars (control plane): DEPLOYMENT_PROFILE (selects preset, beats config.json), CLOUD_RUN_JOB (default 'rally-worker' \u2014 must NOT be set on a Cloud Run service manifest, reserved name), CLOUD_RUN_REGION (default 'asia-south1' in code; live is asia-southeast1), GOOGLE_CLOUD_PROJECT (required for real dispatch), RALLY_ALLOWED_HOSTS, RALLY_APP_HOME.",
- "Env vars (worker Job): RALLY_DETECTOR_IMPL=native, WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS baked in Dockerfile:115-119 (WASB_WEIGHTS re-pointed to the GCS FUSE mount at job-create, README:37); WASB_DECODE_BACKEND/WASB_DEVICE/WASB_SCRATCH optional overrides that beat any --set/config value (native_wasb_runner.py:117-132).",
- "Worker CLI contract (the per-execution container args): `infer --video-uri <uri> --out-uri <prefix> [--done-uri <uri>] [--segmenter <name>] [--skip-validation] [--config path] [--scratch dir] [--max-frames n] (--set a.b.c=v)\*` (worker/__main__.py:720-761). --set is

GOTCHAS: [

- "The worker Job runs with NO DEPLOYMENT_PROFILE, so PROFILE_PRESETS never apply in the container \u2014 every preset-resolved knob the worker must honor has to be explicitly forwarded as --set by the control plane (orchestration.py:598-619) or set as Job env. Forgetting this silently reverts the worker to model defaults (the #506/#507 lesson: decode_backend fell back to cpu/batch-8 until forwarding landed).",
- "#513 incident (2026-07-07): control-plane validated a clip at min_blur_threshold=8.0, then the worker RE-validated with its own default 20.0 and rejected it AFTER the paid GPU dispatch \u2014 fixed by ComputeJobSpec.skip_validation + --skip-validation (worker/__main__.py:273-291). Any plan moving validation ONTO the L4 (#521/#524) must forward the control plane's resolved validation.* config to the worker (the --set precedent), or the same divergence reappears in reverse; it also interacts with the #518 validation-verdict cache, whose fingerprint folds validation.* config + sport + validator set (orchestration.py:814, database get/set_validation_cache).",
- "Version skew is one-directional: a NEW worker CLI flag makes an OLD worker image argparse-abort (rc 2) BEFORE writing the done-marker; pre-#515 that meant a ~65-min opaque poll timeout, now RemoteExecutionFailed fails fast (#515, cloud_run.py:320-356). Rule: roll the worker image FIRST, control plane second (deploy/cloudrun/README.md:79-90). A new phase-placement dispatch flag inherits this constraint.",

"_apply_overrides does setattr on NESTED config models which do NOT have validate_assignment
(model_config

#####

VALIDATION

#####

KEY_FILES:

- base.py :: Registry + ABCs + the #518 config fingerprint
 - 23: _VALIDATION_FINGERPRINT_SCHEMA=1 (manual bump for logic changes); 26-30: ValidationResult{validator_name,passed,message,details}; 33-58: VideoValidator ABC (validate(video_path,config,context)); 61-118: FrameSamplingValidator (declares sample_indices + online _init_state/_consume/_finalize; uses shared pass via context or standalone fallback); 121-186: ValidationRegistry ? run_all_with_context
- __init__.py :: Validator SET + order (folded into fingerprint)
 - 10-15: register order = format_check, resolution_fps_check, pts_continuity_check, scene_cut_check, blur_check, relevance_check
- frame_sampler.py :: #512 shared single-pass decoder for the 3 frame validators
 - 49-50: FRAME_META_KEY/FRAME_STATES_KEY context keys; 56-73: hybrid grab-vs-peek gap threshold (max(2s of frames, 64)); 76-113: run_frame_sampling ? one VideoCapture, union of sample indices, streams frames into per-validator accumulators (1 frame in RAM at a time); 116-143: _decode_and_dispatch monotonic-forward pass (returns early on EOF). Module docstring: 6m47s ? ~8.6s on the 1.4GiB reference c
- format.py :: format_check ? ffprobe container/streams; seeds shared context
 - 26-31: hard-fails if path not on local disk; 33-51: ffprobe -show_format -show_streams (timeout via backend/utils/ffmpeg); 61: context['ffprobe_meta']=info (consumed by resolution_fps + report media block); 74-80: requires 'mp4' in format_name
- pts_continuity.py :: pts_continuity_check ? full-file packet demux
 - 25-36: ffprobe -show_entries packet=pts_time,flags over the ENTIRE file (reads every packet header, no decode); 97-112: fails on backward jumps or keyframe gap > config.validation.pts_max_gap_seconds
- scene_cut.py :: scene_cut_check ? dense samples, first ~50s only
 - 10-13: cut threshold 0.6, cap 100 samples at 2fps; 25-37: sample_indices; 95-104: fails if cuts/min > validation.max_scene_cuts_per_minute
- blur.py :: blur_check ? 15 frames evenly across the WHOLE video
 - 21-24: sample_indices span to end of file; 67-76: avg Laplacian variance < validation.min_blur_threshold ? fail (default 20.0; the #513 incident knob)
- relevance.py :: relevance_check ? 10 frames whole-video, sport-keyed court color
 - 22-25: sample_indices; 27-63: _init_state resolves HSV bounds with precedence config.validation.relevance > sport_registry.get(config.sport).get_relevance_config() > hardcoded defaults (why `sport` is in the fingerprint); 111-117: avg green ratio < min_court_ratio ? fail
- orchestration.py :: CP pipeline: hash ? validate (+#518 cache) ? dispatch/segment
 - 69-77: _validation_failure_message joins 'name: message'; 242-258: _cached_process_payload replays stored videos.validation_results; 220-239: submit_time_video_id (gs:// metadata md5 only, None for local/composite); 423-705: _maybe_dispatch_remote ? _failed replays val_results (458), stages local upload to storage.input_root as <root>/<video_id>/<basename> (523-579), re-UPSERts videos with staged_
- database.py :: Schema: videos column + #518 v10 validation_cache table
 - 196-204: videos(id TEXT PK, filepath, processed_at, status, validation_results TEXT); 471-496: _migrate_to_v10 CREATE TABLE validation_cache(video_id TEXT PRIMARY KEY, fingerprint TEXT NOT NULL, passed INTEGER NOT NULL, results TEXT NOT NULL, created_at TIMESTAMP) ? standalone, NO FK to videos (writable before the videos UPSERT), one latest verdict per video
- database_media.py :: CRUD for verdict persistence
 - 17-40: add_video UPSERT (never REPLACE ? a 'failed' re-validation must not cascade-delete prior segments); 42-68: set_validation_cache UPSERT; 70-91: get_validation_cache ? {fingerprint, passed, results}; 360-368: clear_video_data explicitly DELETES the validation_cache row (no FK cascade)
- __main__.py :: L4 worker: pull ? hash ? (validate|skip) ? segment ? push
 - 244-256: cmd_infer loads ITS OWN config.json + --set overrides (validation config NOT forwarded by the CP today); 265-271: storage.fetch to scratch then compute_video_id re-hash; 273-299: validation ? skipped when --skip-validation (282-291) else validator_registry.run_all_with_context; 293-299: results go into a THROWAWAY scratch DB (never reaches the CP DB); 301-338: _fail(rc=2 for validation) p
- base.py :: Dispatch contract
 - 34-51: ComputeJobSpec(video_uri, out_uri, segmenter, config_overrides tuple of raw '--set

```

k=v', skip_validation=False + full rationale comment); 54-68: ComputeResult(returncode
0/2=validation-or-config/3=segmenter/4=timeout, outputs_uri, video_id from done-marker,
execution)
- cloud_run.py :: Spec ? container argv
  250-269: builds `python -m backend.worker infer --video-uri ? --out-uri ? --done-uri ?`;
264-267: appends --skip-validation when spec.skip_validation; 268-269: --set for each override
- uploads.py :: #480 streamed upload identity (the hash #524(b) wants to trust)
  185+254: incremental hashlib.md5() updated per part; 299-319: bucket-staging finalize
returns {path: stage_uri, video_id: st['hasher'].hexdigest()} ? same bytes/algorithm as
compute_video_id, no re-read; 320-335: local finalize computes compute_video_id(final_path) and
returns it. NOTE: ProcessRequest (backend/main.py:258-272) has NO video_id field, so the
returned id is currently discarded by /ap
- jobs.py :: Submit-time cache probe + job claim
  86-113: submit flow ? submit_time_video_id (gs metadata md5, empty for local uploads) ?
probe_process_cache ? claim_process_job(video_id=?) ? cached_process_result instant completion
(no drain)
- job_worker.py :: How a rejected video surfaces
  146-190: run_one_job ? job.status is EXECUTION state ('done' even for a validation FAIL);
the pipeline verdict lives in result['status']='failed' + result['message'] ('Video failed
validation: ?') + result['results'] (the ValidationResult list); 177-179: set_job_video_id +
mark_job_done. SPA polls the job and renders result.status/message
- models.py :: Validation config (every knob in the fingerprint)
  18-21: ValidationRelevanceConfig(court_green_lower=[35,40,40],
court_green_upper=[85,255,255], court_pixels_min_ratio=0.05); 24-37:
ValidationConfig(min_resolution_width=1280, min_resolution_height=720, min_fps=29.9,
min_blur_threshold=20.0, max_scene_cuts_per_minute=0.5, pts_max_gap_seconds=10.0); 471:
deployment.compute_target Literal['','local_gpu','cloud_run','cpu_mock']; 423/433: storage.outp
- harvest.py :: Verdict's third consumer: the observability report
  33-44: video_meta_from_context reads val_context['ffprobe_meta'] (cached-verdict replay
leaves it {}) ? media block degrades to size-only, deliberate); 83-89: _validation_stage records
total/passed/failures
CONFIG_KNOBS: [
  "validation.min_resolution_width = 1280 (config/models.py:25)",
  "validation.min_resolution_height = 720 (models.py:26)",
  "validation.min_fps = 29.9 (models.py:27)",
  "validation.min_blur_threshold = 20.0 (models.py:28) \u2014 the #513 incident knob (CP ran 8.0,
worker default 20.0)",
  "validation.max_scene_cuts_per_minute = 0.5 (models.py:29)",
  "validation.pts_max_gap_seconds = 10.0 (models.py:30-37)",
  "validation.relevance.court_green_lower = [35,40,40] / court_green_upper = [85,255,255] /
court_pixels_min_ratio = 0.05 (models.py:18-21; sport profile fallback in relevance.py:27-63)",
  "sport (top-level AppConfig field) \u2014 folded into the fingerprint because relevance keys
its court profile off it (base.py:203-217)",
  "deployment.compute_target: ''|local_gpu|cloud_run|cpu_mock (models.py:471) \u2014 gates
_maybe_dispatch_remote",
  "storage.input_root / storage.output_root (models.py:433/423) \u2014 staging + worker output;
both required for the cloud path",
  "NO phase_placement knob exists anywhere yet (repo-wide grep is empty) \u2014 #524(a)
introduces it",
  "ffprobe timeout via backend/utils/ffmpeg.py::ffprobe_timeout_sec (used by
format/pts/resolution validators)"

```

67. assistant (2026-07-08T05:55:14.130Z)

Comprehensive map in hand. Now let me read the actual code for the two wins and confirm #521's status (open PR/branch?), so I design win #1 to slot into its knob without duplication.

68. user (2026-07-08T05:55:18.488Z)

```

525 feat/521-compile-on-l4 ? perf(serving): run compile/stitch on the L4 worker; config-driven
phase?machine placement (#521)
=== open branches mentioning 521/phase/compile ===
  origin/feat/521-compile-on-l4

```

69. user (2026-07-08T05:55:18.510Z)

```
216         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None
227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242 def _cached_process_payload(
243     video_id: str, validation_results, segments, requested_compute
244 ) -> Dict[str, Any]:
245     """The /api/process result for a video whose detection already exists ? ONE shape
for
246     both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247     return {
248         "video_id": video_id,
249         "status": "success",
250         "message": (
251             f"Reused {len(segments)} cached play segments. "
252             "Pass force=true to reprocess this video."
253         ),
254         "results": validation_results or [],
255         "play_segments": segments,
256         "cached": True,
257         "compute": {"path": "cached", "requested": requested_compute},
258     }
259
260
```

70. assistant (2026-07-08T05:55:27.799Z)

#521 has an open branch `feat/521-compile-on-l4`. Let me inspect its exact `phase\_placement` shape so my design + win #1 slot into it without duplication.

71. user (2026-07-08T05:55:30.862Z)

```
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
=== FILES CHANGED ===
backend/compute/__init__.py          | 2 +
backend/compute/base.py              | 34 +++
```

```

backend/compute/cloud_run.py | 100 +++++-
backend/config/__init__.py | 16 ++
backend/config/models.py | 52 ++++
backend/config/placement.py | 59 ++++
backend/config/presets.py | 10 +-
backend/orchestration.py | 322 ++++++-----
backend/pipeline/stitcher/compiler.py | 94 +++++-
backend/worker/__main__.py | 139 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 169 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 1 +
tests/test_api_cloud_dispatch.py | 70 +++++
tests/test_compile_dispatch.py | 204 ++++++
tests/test_compiler.py | 64 +++++
tests/test_compute_backend.py | 111 ++++++
tests/test_config_deployment.py | 5 +
tests/test_phase_placement.py | 138 ++++++
tests/test_worker.py | 175 ++++++
19 files changed, 1710 insertions(+), 55 deletions(-)

```

72. assistant (2026-07-08T05:55:36.612Z)

This is exactly the knob I must not duplicate. Let me read #521's `placement.py`, its config model additions, and its `PHASE\_PLACEMENT.md` so my design + win #1 slot in precisely.

73. user (2026-07-08T05:55:38.741Z)

```
"""Phase ? machine placement resolution (#521).
```

A **phase placement** answers one question per pipeline phase ? ***where does the heavy compute for this phase run?*** ? with a config knob (``deployment.phase_placement``) instead of code. The three placeable phases and their values:

```

validation ? control_plane | cloud_run_l4
segment     ? control_plane | cloud_run_l4 (a.k.a. "detect")
compile     ? control_plane | cloud_run_l4 (a.k.a. "stitch")

```

```

`control_plane` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
`cloud_run_l4`
dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This
module
is the SINGLE source of truth so `backend.orchestration` (segment + compile) and the worker
never
re-derive the mapping.

```

Default = today's behaviour (pure / additive). When a phase's knob is unset (``None``) the resolver reproduces the pre-#521 placement:

```

validation ? control_plane (the authoritative gate,
#512/#513)
segment     ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else control_plane
compile     ? control_plane (the in-process ffmpeg
stitch)

```

so a config with no ``phase_placement`` block behaves byte-identically to before this module existed.

```
from __future__ import annotations
```

```
from typing import Any
```

Phase identities (kept as constants so callers don't stringly-type the phase names).

```
PHASE_VALIDATION = "validation"
```

```

PHASE_SEGMENT = "segment"
PHASE_COMPILE = "compile"
PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)

```

Placement targets.

```

PLACE_CONTROL_PLANE = "control_plane"
PLACE_CLOUD_L4 = "cloud_run_l4"
PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)

```

```

def resolve_phase_placement(cfg: Any, phase: str) -> str:
    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?

```

Precedence: an explicit ``deployment.phase\_placement.<phase>`` wins; otherwise the legacy default

(mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent ``deployment`` / ``phase\_placement`` degrades to the legacy default.

"""

```

deployment = getattr(cfg, "deployment", None)
placement = getattr(deployment, "phase_placement", None)
explicit = getattr(placement, phase, None) if placement is not None else None
if explicit:
    return str(explicit)
# Legacy defaults ? reproduce today's placement exactly.
if phase == PHASE_SEGMENT:
    compute_target = (getattr(deployment, "compute_target", "") or "").strip()
    return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
# validation + compile default to the control-plane (their pre-#521 home).
return PLACE_CONTROL_PLANE

```

74. user (2026-07-08T05:55:39.806Z)

Phase ? machine placement + compile-on-L4 (#521)

```

> **Status:** ? **SHIPPED (code)** . **Owner:** Avi . **Created:** 2026-07-08 . **Issue:**
[#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
> **Relation:** extends [COMPUTE_DECOUPLED_SERVING](./README.md) (the `ComputeBackend` dispatch
seam) and **refines D16(b)** (NVENC-encode-deferred-behind-a-config-knob) now that the
2026-07-08 live CUJ supplies the cost data D16(b) said to revisit on.
> **Evidence:** `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(2026-07-08 live CUJ, rev `rally-control-plane-00019`).

```

0. TL;DR

The compile/stitch phase used to run **in-process** on the CPU-only 2-vCPU control-plane, where the 2026-07-08 live CUJ measured **~12m55s** for an 11-clip / ~80 s reel from a 1.83 GiB 4K source (~45?90 s per ~7 s clip ? the per-clip branding watermark forces a full 4K re-encode with **no NVENC**). Validation (183 s) runs there too; both serialize on `_LOCAL_COMPUTE_LANE``, so one run pins the control-plane for ~16 min ? the API can't scale past a handful of concurrent users.

#521 makes **where** each pipeline phase runs a config edit, not a code change, and moves the stitch to the **L4 worker** (NVENC + 8 vCPU) the way detection already dispatches:

1. `deployment.phase_placement`` ? a per-phase knob (``validation` / `segment` / `compile``) with values ``control_plane | cloud_run_l4``. Unset ? **today's placement, byte-identical**.
2. **compile ? L4** ? a ``kind='compile'`` Cloud Run dispatch mirroring the detect path: the control-plane resolves the reel's segments, stages a compile-spec to the bucket, dispatches

```
`python -m backend.worker compile` on the L4, bucket-polls a done-marker, and returns the
same
`{status, filepath, download_url}` shape the SPA already expects.
3. **NVENC on the L4** ? `deployment.cloud_stitch_encoder` (default `h264_nvenc`) is forwarded
onto
    the worker's `stitching.encoder`; libx264 stays the everywhere-safe fallback (auto-probed).

The cloud-serving preset opts compile onto the L4 (`phase_placement.compile=cloud_run_l4`).
```

1. The placement model

`resolve\_phase\_placement(cfg, phase)` (in `backend/config/placement.py`) is the ****single source of truth**** ? orchestration and the worker never re-derive the mapping. Precedence: an explicit `deployment.phase\_placement.<phase>` wins; otherwise the ****legacy default**** reproduces pre-#521 behaviour exactly:

```
| phase | knob unset (legacy default) | meaning |
|---|---|---|
| `validation` | `control_plane` | the authoritative gate (#512/#513) stays on the control-plane
|
| `segment` (detect) | `cloud_run_l4` **iff** `compute_target==cloud_run`, else `control_plane`
| today's DETECT dispatch |
| `compile` (stitch) | `control_plane` | the in-process ffmpeg stitch |
```

So a config with ****no**** `phase\_placement` block behaves exactly as before this existed. To relocate a phase, set its knob:

```
```jsonc
// Move the stitch to the L4 (what the cloud-serving preset does):
{ "deployment": { "compute_target": "cloud_run",
 "phase_placement": { "compile": "cloud_run_l4" } } }

// Force detection back onto the control-plane on a cloud box (e.g. a debugging run):
{ "deployment": { "compute_target": "cloud_run",
 "phase_placement": { "segment": "control_plane" } } }

// Move validation off the control-plane too (must pair with a cloud segment ? see §4):
{ "deployment": { "compute_target": "cloud_run",
 "phase_placement": { "validation": "cloud_run_l4" } } }
...

```

`control\_plane` runs the phase in-process on the (CPU-only) control-plane service;  
`cloud\_run\_l4`  
dispatches it to the GPU/NVENC Cloud Run worker Job.

---

## 2. compile ? L4 dispatch (the flow)

`orchestration.\_execute\_compile` consults `resolve\_phase\_placement(cfg, "compile")`. When it resolves to `cloud\_run\_l4` **\*\*and\*\*** cloud dispatch is available (`\_cloud\_available`), it calls `\_dispatch\_compile\_remote`; otherwise it stitches in-process via the shared `\_stitch\_reel` core (unchanged).

`\_dispatch\_compile\_remote` (mirrors `\_maybe\_dispatch\_remote`):

1. **\*\*Resolve\*\*** the reel's segments HERE ? the control-plane owns the DB (`\_resolve\_compile` ? `(video, kept, out\_name)`), applying the `stitching.min\_shot\_count` floor as before.
2. **\*\*Stage a compile-spec\*\*** to `/compile/<token>/spec.json` ? the resolved

`time\_pairs`, the control-plane's resolved `stitching` config (watermark + padding, with `encoder` swapped to `cloud\_stitch\_encoder`), the source `video\_uri`, `video\_id`, `output\_name`, `locale`.

The worker therefore needs **no control-plane DB** ? everything rides the spec.

- Dispatch** a `compile` Cloud Run Job execution (`CompileJobSpec` ? `CloudRunCompute.run\_compile`)
  - running `python -m backend.worker compile --spec-uri ? --done-uri ? --set storage.output\_root=?`,
  - forwarding the bucket coordinates (the worker Job has no `DEPLOYMENT\_PROFILE`).
- Bucket-poll** the done-marker at `/compile/<token>.done` ? the SAME `\_await\_marker` machinery `run\_infer` uses (crash-fast on a terminal-without-marker execution, rescue-or-cancel on a poll timeout). Distinct `\_compile/` prefix so a compile and a detection for one bucket never collide on a marker.
- The worker (`cmd\_compile`) reads the spec, stitches the reel via the **shared `\_stitch\_reel` core** (no forked pipeline), mirror-uploads it to `/highlights/<video\_id>/<name>`, and writes the marker (rc + `download\_url` + `reel\_uri`).
- The control-plane returns `{status, filepath, download\_url, video\_id}` ? the reel is already in the bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker rc?0 ? `status='failed'` with the worker's message (clean SPA verdict, never a crash).

**Fallback:** if compile is placed on the L4 but the box isn't actually cloud-wired (no output bucket / Cloud Run env), `\_dispatch\_compile\_remote` logs and stitches in-process ? a slow reel beats no reel.

---

### 3. NVENC on the L4 ? and the D5/D16(b) parity note

The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding watermark forces (~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521 adds an encoder knob:

- `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) | `h264\_nvenc` | `hevc\_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a 1-frame lavfi test encode) and **falls back to libx264** if this ffmpeg build can't run it, so a misconfig / non-GPU box never breaks. All clips share the one chosen encoder ? the concat stays `-c copy`.
- `deployment.cloud\_stitch\_encoder` (default `h264\_nvenc`) ? what the compile-dispatch forwards onto the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4 stitch on CPU.
- The control-plane's **own** local stitch always uses `stitching.encoder` (libx264 by default) ? the CP has no GPU.

**Parity note (honest):** [README.md](./README.md) **D5/D16(b)** kept stitch **encode = libx264** to protect a byte-exact stitch parity guarantee, and explicitly **deferred NVENC** encode behind a config knob, "revisit only if M7 cost data justifies relaxing parity." The 2026-07-08 CUJ (~13 min stitch, a hard scaling wall past ~5 concurrent users) **is** that cost data. #521 lands exactly the deferred shape ? NVENC **behind a config knob**, libx264 the default, config-reversible per phase.

Enabling  
 NVENC relaxes byte-identity **\*\*at the reel ENCODE layer only\*\*** (a cosmetic, shareable artifact);  
 the  
**\*\*WINDOW/rally-level parity\*\*** (the analytical output) is untouched ? detection and windowing are  
 unchanged. If bit-exact reels are ever required, set ``cloud_stitch_encoder=libx264``.  
  
 > Follow-up (tracked, not in #521's first cut): a **\*\*pre-rendered / overlay-filter watermark\*\*** so  
 the  
 > stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).  
  
 ---

#### 4. Validation placement + the "nobody validates" guard

``phase_placement.validation`` moves the CPU-bound validator suite off the 2-vCPU control-plane  
 too,  
 via the existing ``--skip-validation`` lever (#513):

- **\*\*`control\_plane`\*\*** (default): the control-plane runs the validators and is the authoritative  
 gate;  
 the worker **\*\*skips\*\*** its redundant re-validation (``skip_validation=True``) ? today's #512/#513  
 behaviour, byte-identical.
- **\*\*`cloud\_run\_l4`\*\***: when detection **\*\*actually\*\*** dispatches to the L4, the control-plane  
**\*\*delegates\*\***  
 ? it does NOT run the validator suite (``_execute_processing`` skips it), and the dispatch  
 carries  
**\*\*no\*\*** ``--skip-validation``, so the **\*\*worker\*\*** validates and rejects a bad clip via a non-zero  
 rc  
 (surfaced as a failed job).

**\*\*Footgun guard:\*\*** a clip can only be validated on the worker if the worker also **\*\*detects\*\*** it  
 there.  
 Two protections:

1. **\*\*Config-load guard\*\*** (``DeploymentConfig`` model-validator): ``validation=cloud_run_l4``  
 requires the  
 effective segment placement to also be ``cloud_run_l4`` (explicit, or via  
``compute_target=cloud_run``)  
 ? else config load fails with an actionable error.
2. **\*\*Runtime guard\*\*** (``_will_detect_on_cloud``): if a per-request ``compute='local'`` forces  
 detection  
 in-process (no worker), the control-plane **\*\*still validates\*\*** ? never leave a clip  
 unvalidated  
 because nobody ran the gate.

---

#### 5. Files & tests

Area	Code	Tests
Placement resolver + config	<code>`backend/config/placement.py`</code> , <code>`backend/config/models.py`</code> <code>(`PhasePlacementConfig`, `DeploymentConfig.phase_placement` / `cloud_stitch_encoder`, guard)</code>	<code>`tests/test_phase_placement.py`</code> , <code>`tests/test_config_deployment.py`</code>
Preset	<code>`backend/config/presets.py`</code> ( <code>`cloud-serving`</code> ? <code>`phase_placement.compile=cloud_run_l4`</code> )	<code>`tests/test_phase_placement.py`</code>
compile dispatch	<code>`backend/orchestration.py`</code> ( <code>`_execute_compile`</code> , <code>`_dispatch_compile_remote`</code> , <code>`_stitch_reel`</code> , <code>`_will_detect_on_cloud`</code> )	<code>`tests/test_compile_dispatch.py`</code>
compute backend	<code>`backend/compute/base.py`</code> ( <code>`CompileJobSpec`</code> , <code>`run_compile`</code> ), <code>`backend/compute/cloud_run.py`</code> ( <code>`run_compile`</code> , <code>`_await_marker`</code> )	<code>`tests/test_compute_backend.py`</code>
worker	<code>`backend/worker/__main__.py`</code> ( <code>`compile`</code> subcommand, <code>`cmd_compile`</code> )	<code>`tests/test_worker.py`</code>
NVENC encoder	<code>`backend/pipeline/stitcher/compiler.py`</code> ( <code>`encoder`</code> , <code>`_select_encoder`</code> ,	

```
`_probe_encoder_available`) | `tests/test_compiler.py` |
| validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip flag,
`_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
```

All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the staged spec, writes the reel + done-marker into a local "bucket"), mirroring `tests/test\_api\_cloud\_dispatch.py`.

#### 75. assistant (2026-07-08T05:56:04.941Z)

**\*\*Critical discovery:\*\*** #521's open branch **\*\*already implements win #1\*\*** (validation-on-L4 via `phase\_placement.validation=cloud\_run\_l4`, with config-load + runtime footgun guards). Re-implementing it would duplicate the knob and the delegation logic ? exactly what the task forbids. Let me verify whether #521 still downloads/re-hashes on the CP when validation is delegated (that determines whether win #2 + download-avoidance is the complementary piece that kills the ~30s gap).

#### 76. user (2026-07-08T05:56:08.324Z)

```
18:+ PHASE_VALIDATION,
22:+ resolve_phase_placement,
31:+def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
34:+ decide whether to DELEGATE validation to the worker (`phase_placement.validation`). A
per-request
44:+ resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
64:+ resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
80:+ # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
82:+ # here), so the worker SKIPS its redundant re-validation (#513). If validation is placed
on the
83:+ # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ? see
84:+ # _execute_processing), so the worker must NOT skip: it becomes the gate.
85:+ worker_skips_validation = (
86:+ resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
94:+ skip_validation=worker_skips_validation,
104:+ # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
105:+ # actually dispatch there, the control-plane DELEGATES validation to the worker ?
it does NOT
106:+ # run the validator suite here (the whole point is to move the CPU-bound validation
OFF the
107:+ # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
110:+ # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
112:+ resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
113:+ or not _will_detect_on_cloud(config, getattr(req, "compute", None))
117:+ "[API] validation delegated to the L4 worker "
118:+ "(deployment.phase_placement.validation=cloud_run_l4) ? the control-plane
skips its "
139:+ if validate_on_control_plane and not reused_validation:
144:+ if validate_on_control_plane and not reused_validation:
408:+ resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
```

#### 77. assistant (2026-07-08T05:56:54.764Z)

Confirmed: #521 delegates the *\*validator suite\** to the worker, but does **\*\*not\*\*** touch the top-of-`\_execute\_processing` `acquire\_local\_input` + `compute\_video\_id` block ? so even with validation delegated, the CP still downloads the full 1.83 GiB and re-hashes it. That top block is exactly win #2's territory and is independent of #521.

**\*\*Scope is now locked:\*\*** #521 already owns win #1 (validation-on-L4 + the `phase\_placement` knob + guards). Duplicating it is forbidden. My PR = **\*\*design doc + win #2 (re-hash)\*\***, with win #1

credited to #521 and the full ~30s-gap kill (CP-download-avoidance, which composes #521 + win #2) specified as the follow-up. Let me read the exact win #2 code paths.

## 78. user (2026-07-08T05:56:55.781Z)

```
1 """Job-list endpoint (the processing-queue view) as its own router ? kept OUT of the
backend.main
2 god-module per the P24 route ceiling (see tests/test_main_module_budget.py; pattern:
backend/api/uploads.py).
3
4 GET /api/jobs is tenant-scoped and status-filtered so a client can discover in-flight
work it did NOT
5 submit (a page reload, a new tab, another device) and show its status ? which also stops
a tester who
6 can't see their running job from re-submitting the same video into a second, PAID GPU
run (#448). The
7 per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
8
9 Reads the live DB + config off ``request.app.state`` (set in backend.main.create_app),
so it needs no
10 module-global wiring and stays trivially testable via the standard
TestClient(create_app(cfg, db)).
11 """
12
13 import tempfile
14 from typing import Any, List, Optional, Tuple
15
16 from fastapi import APIRouter, HTTPException, Request
17 from fastapi.responses import JSONResponse
18
19 from backend.api import (
20 auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
21)
22 from backend.utils.freespace import (
23 require_free_bytes, # HTTP-507 scratch guard (P2, D11)
24)
25
26 router = APIRouter()
27
28
29 def claim_process_job(
30 db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32 """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33 ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34 double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
35 ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
36 just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
37 ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
38 hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39 the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40 can re-associate the job with its bucket outputs (CP-rebuild §4.2); "" ? the drain
fills it."""
41 claimed = db.claim_or_create_process_job(
42 job_id,
43 req.video_path,
```

```

44 segmenter=req.segmenter,
45 player_pool=req.player_pool,
46 max_frames=req.max_frames,
47 owner_id=owner_id,
48 locale=req.locale,
49 compute=req.compute,
50 force=req.force,
51 video_id=video_id,
52)
53 if claimed is None:
54 return None
55 if claimed["created"]:
56 return (True, None)
57 return (
58 False,
59 JSONResponse(
60 status_code=202,
61 content={
62 "job_id": claimed["job_id"],
63 "status": claimed["status"],
64 "poll": f"/api/jobs/{claimed['job_id']}",
65 "deduplicated": True,
66 },
67),
68)
69
70
71 def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:
72 """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
73 (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
74 copy."""
75 from backend import orchestration # call-time seam (mirrors submit_process_job)
76 need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
77 None))
78 require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
79
80 def submit_process_job(
81 db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82) -> Optional[Tuple[bool, JSONResponse]]:
83 """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
84 one hook.
85
86 Order matters (#492 review):
87 1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
88 effects.
89 2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
90 fetches to local scratch, so low temp space must not reject it (review blocker
91 2).
92 3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
93 re-hydrate side effect, so the first request owns the cache completion (blocker
94 1).
95 4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
96 serialized
97 per video_id) and completes the job instantly: no drain, no GPU dispatch.
98
99 Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
100 ``None`` ?
101
102 DB failure (the caller 500s)."""
103 from backend import orchestration # call-time seam: tests patch orchestration attrs
104
105 video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
106 probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
107 db, cfg, video_id

```

```

101)
102 if input_ref.backend != "local" and not probed:
103 _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
104 outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
105 if outcome is None:
106 return None
107 created, dedup_response = outcome
108 if not created:
109 assert dedup_response is not None # claim contract: not-created ? dedup
response
110 return (False, dedup_response)
111 if probed:
112 cached = orchestration.cached_process_result(
113 db, cfg, video_id, req.video_path, requested_compute=req.compute
114)
115 if cached is not None:
116 db.mark_job_done(job_id, cached)
117 return (
118 False,
119 JSONResponse(
120 status_code=202,
121 content={
122 "job_id": job_id,
123 "status": "done",
124 "poll": f"/api/jobs/{job_id}",
125 "cached": True,
126 },
127),
128)
129 # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
130 # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
131 # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
132 # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133 if input_ref.backend != "local":
134 try:
135 _fetch_scratch_guard(cfg, input_ref)
136 except HTTPException:
137 db.mark_job_failed(
138 job_id,
139 "cache re-hydrate failed and local scratch cannot hold a fetch of "
140 "the source; free disk space or retry later",
141)
142 raise
143 return (
144 True,
145 JSONResponse(
146 status_code=202,
147 content={
148 "job_id": job_id,
149 "status": "queued",
150 "poll": f"/api/jobs/{job_id}",
151 },
152),
153)
154
155 # Execution states a job can be in. ACTIVE = still doing (or about to do) work; the
queue view defaults
156 # to these. done/failed are terminal and only surface under ?status=all (or an explicit
single status).
157 ACTIVE_JOB_STATES = ("queued", "running", "waiting")
158 _LISTABLE_JOB_STATES = frozenset(ACTIVE_JOB_STATES) | {"done", "failed"}
159

```

```

160
161 @router.get("/api/jobs")
162 def list_jobs(request: Request, status: str = "active"):
163 """The caller's jobs (tenant-scoped), newest first. ``status=active`` (default) ?
164 queued|running|waiting; ``status=all`` ? recent jobs of any state; a single explicit
state is also
165 accepted (anything unknown degrades to the active queue ? never a raw 500). Light
projection: the
166 heavy per-job ``play_segments`` payload is dropped; ``result_status`` carries the
terminal verdict."""
167 try:
168 owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
169 except edge_auth.EdgeAuthError as e:
170 raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
171 if status == "all":
172 statuses: Optional[List[str]] = None
173 elif status in _LISTABLE_JOB_STATES:
174 statuses = [status]
175 else: # "active" and any unknown value ? the active queue
176 statuses = list(ACTIVE_JOB_STATES)
177 rows = request.app.state.db.list_jobs(owner_id, statuses=statuses, limit=50)
178 return {
179 "jobs": [
180 {
181 "job_id": j["id"],
182 "kind": j.get("kind") or "process",
183 "status": j["status"],
184 "progress": j.get("progress"),
185 "video_id": j.get("video_id"),
186 "source": j.get("video_path"),
187 "compute": j.get("compute"),
188 "result_status": j.get("result_status"),
189 "error": j.get("error"),
190 "created_at": j.get("created_at"),
191 "started_at": j.get("started_at"),
192 "finished_at": j.get("finished_at"),
193 }
194 for j in rows
195]
196 }
197

```

## 79. assistant (2026-07-08T05:57:00.877Z)

Now the job-drain side ? how the request is rebuilt and where the persisted `video\_id` can be threaded into `\_execute\_processing`.

## 80. user (2026-07-08T05:57:02.023Z)

```

118 self.delay_sec = max(0.0, float(delay_sec))
119 self.progress = progress
120
121
122 def _job_to_request(job: Dict[str, Any]):
123 """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124 re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125 the original submit. The row stores exactly the ProcessRequest fields."""
126 import backend.main as _main
127
128 raw_params = job.get("params")
129 params = (
130 raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131)
132 return _main.ProcessRequest(
133 video_path=job["video_path"],

```

```

134 player_pool=job.get("player_pool"),
135 max_frames=job.get("max_frames"),
136 segmenter=job.get("segmenter"),
137 # v8 compute-picker: the worker runs _execute_processing ?
maybe_dispatch_remote, which
138 # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139 # the job row for analytics). NULL ? server default = pre-picker behaviour.
140 compute=job.get("compute"),
141 force=bool(params.get("force")),
142)
143
144
145 def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146 """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147 terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149 Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150 the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151 result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152 job self-scheduled and will be re-claimed when `next_poll_at` is due.
153 """
154 import backend.main as _main
155
156 job_id = job["id"]
157 # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158 # original /api/process pipeline). Both record their terminal state the same way;
only process
159 # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160 is_compile = job.get("kind") == "compile"
161 try:
162 if is_compile:
163 result = _main._execute_compile(
164 config,
165 db,
166 job,
167 progress=lambda stage: db.set_job_progress(job_id, stage),
168)
169 else:
170 req = _main._job_to_request(job)
171 result = _main._execute_processing(
172 config,
173 db,
174 req,
175 progress=lambda stage: db.set_job_progress(job_id, stage),
176)
177 if result.get("video_id"):
178 db.set_job_video_id(job_id, result["video_id"])
179 db.mark_job_done(job_id, result)
180 if not is_compile:
181 _maybe_record_job_cost(
182 job, result, config, db
183) # M8 cost ledger (default-OFF; process jobs only)
184 _maybe_run_flywheel(
185 job, result, config
186) # M11 data-flywheel (process jobs only)
187 except _main.JobWaiting as w:
188 # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189 logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190 db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)

```

```

191 except Exception as e:
192 logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193 db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196 def _maybe_record_job_cost(
197 job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198) -> None:
199 """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200 ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201 (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
202 pricebook, and
203 persist it.

```

## 81. assistant (2026-07-08T05:57:40.396Z)

I now have the complete win #2 picture. The trusted id source is the job row's `video\_id` (persisted at claim via `submit\_time\_video\_id`, #484 ? GCS-authoritative, **not** client-supplied), which `\_job\_to\_request` currently drops. Let me confirm the config placement and check nothing else re-derives `video\_id` from the local file.

## 82. user (2026-07-08T05:57:41.843Z)

```

200
201
202 class IndexingConfig(BaseModel):
203 default_segmenter: str = "yolo_hybrid"
204 use_gpu: bool = True
205 # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206 # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207 # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208 # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209 # RALLY_DETECTOR_IMPL env var overrides this. See
210 pipeline/detectors/{stub,native_wasb}_runner.py.
211 detector_impl: str = "auto"
212 motion_threshold: float = 0.08
213 min_segment_duration: float = 3.0
214 dead_time_merge_gap: float = 2.0
215 chunk_duration: float = 90.0
216 chunk_overlap: float = 10.0
217 detector_model: str = "fasterrcnn_resnet50_fpn"
218 detector_score_threshold: float = 0.5
219 yolo_velocity_threshold: float = 0.15
220 yolo_frame_skip: int = 3
221 yolo_tracker: str = "bytetrack.yaml"
222 yolo_prefetch_workers: int = 2
223 min_action_window_duration: float = 2.0
224 # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
225 # = a window
226 # longer than this many seconds is recursively split at its largest internal
227 # inactivity gap (?
228 # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
229 # swallow
230 # several rallies + the dead time between them. Never merges, only cuts (recall
231 # can't drop).
232 # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
233 # Measured n=13
234 # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
235 # p=0.0005; cap=6
236 # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
237 # 0.651?0.161). ***
238 max_window_duration: float = 6.0
239 window_min_split_gap: float = (
240 0.5 # min internal gap (s) that qualifies as a split point
241)

```

```

234 # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
235 # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
236 # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
237 # Only cuts (recall can't drop). OFF by default until measured/promoted.
238 window_hard_cap: bool = False
239 # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240 # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241 # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242 # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243 # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244 # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
245 # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246 window_inactivity_end: float = 1.5
247 inactivity_rally_thresh: float = 0.20
248 inactivity_min_density: float = 0.30
249 # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250 # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251 # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252 # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253 # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254 # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255 # regression), [?end] 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
256 # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257 # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258 inactivity_temporal_density: bool = True
259 # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
260 # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261 # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262 # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263 # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264 # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265 # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266 # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267 window_blob_fallback: bool = False
268 # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269 # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270 # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active

```

```

271 # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
272 window_blob_factor: float = 4.0
273 log_yolo_candidates: bool = True
274 store_yolo_candidates: bool = True
275 ai_handoff_padding: float = 2.0
276 ai_max_workers: int = 5
277 # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278 # chunks of a high-bitrate (4K) source blow past the provider upload cap
279 # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280 # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281 # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282 # Matches gemini_refine's --clip-scale-width default.
283 ai_chunk_scale_width: int = 1280
284 # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285 # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286 # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287 # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288 # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289 skip_ai_handoff: bool = False
290 # Optional debug knob: trim every video to the first N seconds before processing.
291 debug_duration: Optional[float] = None
292
293 tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294 wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)

```

### 83. assistant (2026-07-08T05:59:25.112Z)

The internal-parameter approach is cleaner and safer than a client-facing field: the job-row `video\_id` is **server-derived** (`submit\_time\_video\_id`, #484 ? authoritative GCS md5), so threading it via an internal param can't be spoofed by a client. Let me confirm the hash format and that the submit-time cache path already trusts this same value (which makes win #2 consistent with existing behavior).

### 84. user (2026-07-08T05:59:29.618Z)

```

1 """Canonical file-identity hashing for the indexer.
2
3 `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4 (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5 defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6 The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7 any future change to one copy would silently diverge the DB key.
8
9 This module is the single source of truth. Import `compute_video_id` everywhere a
10 video needs to be keyed.
11 """
12
13 from __future__ import annotations
14
15 import hashlib
16
17 # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18 # was centralized. The chunk size does not affect the resulting digest ? it only
19 # bounds memory use while streaming large files.
20 _CHUNK_SIZE = 65536
21
22
23 def compute_video_id(filepath: str) -> str:
24 """Return the MD5 hex digest of the whole file ? the canonical video_id.

```

```

25
26 Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27 into memory at once.
28 """
29 hasher = hashlib.md5()
30 with open(filepath, "rb") as f:
31 for chunk in iter(lambda: f.read(_CHUNK_SIZE), b''):
32 hasher.update(chunk)
33 return hasher.hexdigest()
34

```

#### 85. assistant (2026-07-08T05:59:39.809Z)

`compute\_video\_id` = 32-char md5 hex. Consistency confirmed: the submit-time cache path (jobs.py:98-101) already trusts this same `submit\_time\_video\_id` to complete jobs with zero re-hash ? win #2 just extends that established trust to the cache-miss drain path. Let me re-scope the tasks and read the doc conventions.

#### 86. user (2026-07-08T05:59:40.324Z)

Updated task #1 status

#### 87. user (2026-07-08T05:59:44.375Z)

Updated task #5 subject, description, status

#### 88. user (2026-07-08T05:59:44.784Z)

```

1 <!--
2 PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3
4 A "tracked project doc" is how this repo carries substantial design/project work: it can
be
5 ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables,
then
6 archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
7 docs/GOLDEN_REGRESSION_FIXTURES.md.
8
9 Rules of thumb:
10 - §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable
PR.
11 - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs
sign-off),
12 or [design] (proposed). The doc reflects real shipped state, never aspirational.
13 - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
14 - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model
for
15 research- or security/privacy-heavy work). Delete sections that do not apply.
16 - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state
work).
17 -->
18
19 # <Project / Plan Title>
20
21 > **Status:** `DRAFT` ? `<owner-gated? | in progress>` . **Owner:** `<name>` .
Created: `<YYYY-MM-DD>` . **Last updated:** `<YYYY-MM-DD>`
22 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
23 > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
24 > **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
25 > **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not
legal/financial advice where applicable.
26

```

```

27 ---
28
29 ## 0. TL;DR
30 <2?5 sentences: the goal, the approach, and the single most important caveat. Call out
any finding that is already actionable on its own.>
31
32 ## 1. Problem & goal
33 <Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a
reader must read to get current.>
34
35 ## 2. Decisions locked
36 <Table ? capture answered questions so they are not relitigated.>
37
38 | # | Decision | Source / date | Implication |
39 |---|-----|-----|-----|
40 | D1 | ? | ? | ? |
41
42 ## 3. Foundation ? research / prior art *(OPTIONAL)*
43 <Include when the plan rests on external facts: cited research, legal/compliance,
benchmarks, prior-art survey. Mark researched-vs-verified; list any sign-off gates this
creates.>
44
45 ## 4. Design / architecture
46 <The actual plan. Data contracts, diagrams, and a REUSE MAP ? name the existing
modules/code this builds on so it is clear what is new vs reused.>
47
48 ## 5. Threat model / risk table *(OPTIONAL ? security / privacy / compliance work)*
49 <Who/what is protected vs NOT; residual risks stated plainly.>
50
51 ## 6. Honest limits ? what this does NOT do
52 <The false-confidence guardrail: what a green suite / a DONE status does NOT prove.>
53
54 ## 7. Deliverables & progress tracker ? **source of truth**
55 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
56
57 | ID | Deliverable | Depends on | Gated? | Status | PR |
58 |---|-----|-----|-----|-----|----|
59 | P0 | ? | ? | No | ? | ? |
60
61 ## 8. Open questions ? owner / external *(split blocking-gates from nice-to-knows; name
who decides)*
62 1. ?
63
64 ## 9. Definition of done
65 <Explicit acceptance checklist. When all ticked ? flip `Status ? DONE` and archive.>
66 - [] ?
67
68 ## 10. References
69 **Internal code anchors `[verified]`:** ``. **Internal docs:** ``.
External: ``.
70
71 ---
72
73 ### Changelog
74 - `` ? ``.
75

```

**89. user (2026-07-08T05:59:50.595Z)**

```

=== COMPUTE_DECOUPLED_SERVING dir ===
architecture.svg
README.md
runlogs
TESTING_STRATEGY.md

```

=== README index tail (doc listing) ===

1:# Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage  
6:> **Relation to existing docs:** the productionization successor to the archived  
`DECOUPLED\_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from  
[PLATFORM\_ARCHITECTURE.md](../PLATFORM\_ARCHITECTURE.md) §3; the **truly-local** profile is the  
L0 tier of [VIDEO\_LOCALITY\_MODEL.md](../VIDEO\_LOCALITY\_MODEL.md) and the `LocalDesktop` backend  
of [DESKTOP\_APP\_PLAN.md](../DESKTOP\_APP\_PLAN.md).  
14:## 0. TL;DR  
17:## 1. Problem & goal  
24:## 2. Decisions locked  
26:| # | Decision | Source | Implication |  
27:|---|-----|-----|-----|  
28:| D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch**  
inside it. | design `[verified]` | All profile difference lives in config + the two  
registries. |  
29:| D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` +  
`compute\_target`; `StorageBackend` (what-bytes) by `input\_backend`/`storage.backend`. | ADR-014  
| Realizes PLATFORM\_ARCHITECTURE's `local`/`hosted`/`byo\_worker`. |  
30:| D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo\_worker`  
reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |  
31:| D4 | In **cloud-serving**, STITCH runs on Cloud Run (co-located with detect) so CPU/wall +  
egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real  
cost. |  
32:| D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect**  
is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the rally/window  
level, never CSV bytes. |  
33:| D6 | **Default-OFF**, smallest-safe-first, one PR per milestone; any core-caller change  
(e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-  
convention]] | Zero-regression discipline. |  
34:| D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an  
async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that  
~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane  
only on a UX complaint. |  
35:| D8 | **Pricebook** = a versioned DB table; cost computed in **USD** (matches GCP billing =  
one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-  
versioned when GCP prices change. | owner 2026-06-21 | M8. |  
36:| D9 | **Edge auth** = Cloudflare Access (reuse the site's existing CF). Lock the Cloud Run  
**ingress** to the CF proxy + **verify the `Cf-Access` JWT signature** (so a direct hit to the  
Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |  
37:| D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training rights**  
(uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid  
feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes,  
training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data +  
**auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public  
signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] /  
PERSONALIZATION flywheel. |  
38:| D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears  
the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model  
runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ?  
CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |  
39:| D12 | **Data-flywheel harness is IN SCOPE** (owner add, Q5): capture the (consented, adult)  
uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video  
workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones**  
to the golden TRAINING set (human-reviewed labels) so the owned model climbs toward the D11  
floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) +  
`data\_pipeline` + the eval/experiment harness. |  
40:| D13 | **Profile is user-switchable ANYTIME** ? a headline FEATURE. The same user runs  
**local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure  
core gives **equivalent** output either way. **Advertise it** ("your machine \*or\* our cloud ?  
your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the  
**rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile,  
switching is **between** jobs. |  
41:| D14 | **NORTHSTAR** ? operate toward this: a **mobile, app-like** flow ? point at a  
**Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel  
lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **gdrive-api**

(OAuth) StorageBackend\*\* (read source + write reel back) ? distinct from the local \*\*mount\*\* backend; \*\*resolves S8 #7\*\*, new \*\*M12\*\*. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |

42:| D15 | \*\*Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.\*\* A GPU owner may still choose cloud; \*\*cloud (= spend) is never entered silently\*\* ; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends S4.3 (auto-detect = a proposed default, not an auto-decision). |

43:| D16 | \*\*M6/M7 implementation SHAPE + the cheap S8 picks (owner 2026-06-21):\*\* (a) \*\*Cloud Run JOBS\*\* (not Services-with-GPU) for the detect+stitch unit, \*\*bucket-poll for completion\*\* (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) \*\*stitch ENCODE = libx264\*\* (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? \*\*NVDEC for DECODE only\*\* (a speed lever that never touches output bytes); \*\*NVENC encode is DEFERRED behind a config knob\*\*, revisit only if M7 cost data justifies relaxing parity (S8 Q8); (c) \*\*input cap = hard-reject `<2GB`\*\* for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? S8 Q9). | owner 2026-06-21 | M6/M7. Resolves S8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

44:| D17 | \*\*Real cloud verification BUDGET = \$100 / ?10k cap (owner 2026-06-21):\*\* the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY\_REPORT), \*\*provided the cap is respected\*\* ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and \*\*DELETE every resource after\*\* ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |

46:## 3. Foundation ? evolution / ADR lineage ? \*trace the idea end-to-end\*

49:| ADR / doc | Contributed | Subtlety carried into this project |

50:|---|---|---|

51:| \*\*ADR-005\*\* | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the \*\*container image\*\* (ffmpeg, fonts, weights) stay commercial-clean. |

52:| \*\*ADR-008\*\* | Owned-model topology; \*\*train==serve featurizer single-sourcing\*\* | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to \*\*local/cloud\*\*. |

53:| \*\*ADR-009\*\* | KhelSutra Guru; \*\*local-first delivery\*\* | The \*\*truly-local profile\*\* is the direct continuation ? privacy/offline self-host stays first-class. |

54:| \*\*ADR-010\*\* | DECOUPLED: D0?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |

55:| \*\*ADR-011\*\* | DECOUPLED scope = M0?M2+M3.5; \*\*deferred M3 (serving) + M4 (billing/cost-guard)\*\*; override the \*\*"rent nothing / not a service"\*\* posture | \*\*This project realizes M3/M4.\*\* Carries S06's owner-gated gates: \*\*DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3\*\*. |

56:| \*\*ADR-012\*\* | GPU-native; TPU deferred; \*\*Cloud Run+GPU scale-to-zero = serving model\*\* ; \*\*NVDEC\*\* named as the decode lever | The serving substrate + the 7-step seed scope this expands. |

57:| \*\*ADR-013\*\* | Drop D0; \*\*GCP sole compute stack\*\* ; \$200 D0 credits ? non-compute only | The provider for cloud-serving; D0 Spaces remains only a possible S3 \*storage\* target. |

58:| \*\*? ADR-014 (new)\*\* | \*\*Compute-decoupled serving:\*\* one pure core, profile-selected compute+storage; stitch-where-metered | This doc. \*\*Refines\*\* ADR-011/012, does \*\*not\*\* supersede them. |

59:| PLATFORM\_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |

60:| VIDEO\_LOCALITY\_MODEL.md / DESKTOP\_APP\_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |

61:| 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with \*\*no CPU fallback\*\* ? a portability gap M4 closes. |

62:| DEPLOY\_REPORT\_GCP\_2026-06-20 (archived) | First real L4 run; \*\*cross-GPU sub-pixel parity finding\*\* | Why D5 (parity at window level, not CSV bytes). |

64:## 4. Design / architecture

## 90. assistant (2026-07-08T06:01:30.106Z)

I have the complete picture: cost model, topology constraints (D4/D7/D16a from ADR-014), #521's foundation, and the central quantified insight ? \*\*uploading onto the L4 trades a ~free, seconds-fast intra-region LAN pull for ~175s of idle GPU per run\*\* (a bad trade), while the ~30s

gap is actually killed by \*not\* downloading to the CP at all\* (composing #521's validation-delegation + win #2's md5-trust). Let me write the design doc.

91. assistant (2026-07-08T06:04:14.731Z)

## L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the control

```
> **Status:** `DRAFT` ? **owner-review** (recommends *against* the epic's proposed direction #1;
see §0/§8) . **Owner:** Avi . **Created:** 2026-07-08 . **Last updated:** 2026-07-08
> **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524) (epic)
. **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
(`phase_placement`) . **Motivated by:** [#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that surfaced the data-movement finding)
> **Relation to existing docs:** extends [COMPUTE_DECOUPLED_SERVING/README.md](./README.md)
(ADR-014) and [PHASE_PLACEMENT.md](./PHASE_PLACEMENT.md) (#521); it does **not** supersede them
? it answers the *topology* question they left open and sequences the migration on top of
`phase_placement`.
> **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog),
`[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]`
(proposed). Not legal/financial advice.
```

---

## 0. TL;DR

The 2026-07-08 live CUJ moved one 1.83 GiB clip **3x** (upload?bucket, bucket?control-plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for **~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by **uploading directly onto the L4** and async-copying to the bucket. **This design recommends against that direction** and quantifies why: a same-region bucket?compute transfer inside GCP is **LAN-fast** and not billed as internet egress **[researched]**, so the round-trip that upload-onto-L4 saves is worth **~seconds** and **~\$0**, while keeping an L4 up to **receive** a **minutes-long** upload burns **~175 s** of idle GPU per run (~\$0.08?0.16) **[design]** and **opens** a durability exposure window instead of closing one.

The cheaper, more durable, already-ratified path is the opposite: **keep the upload streaming to the cheap bucket** (the #480 design ? the durable copy exists **before** the L4 ever starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ? which is exactly what **#521's `phase\_placement`** already wires as a config flip. The residual "30 s pre-validation gap" is then killed not by moving the **upload** but by the control-plane **never downloading the file at all** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3x to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero idle-GPU cost** and **zero exposure window**.

**Actionable now (this PR):** the design doc + **win #2** (drop the redundant control-plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.

## 1. Problem & goal

**Why now.** Tracing the live CUJ (`runlogs/DEPLOY\_REPORT\_cloud-serving\_validation-latency\_2026-07-07.md` §9 **[verified]**) produced two findings:

- Data is moved 3x per run** **[verified]**: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? storage.acquire_local_input(gs://?)`` at `backend/orchestration.py:748``, then a **redundant re-hash** `compute_video_id`` at `:751``; (c) the worker downloads it a third time to detect (`[worker] pulled ?``). The unlogged **~30 s pre-validation gap** is finding (b).

2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by `_LOCAL_COMPUTE_LANE`` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

**The concrete outcome we want.** The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls the full file**. Every move must be **reversible by config** with a **bucket-mediated fallback**, and land as small PRs on top of `origin/master``.

**The hard question this doc must resolve (compute topology).** "Upload directly to the L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch, no inbound HTTP ? `backend/compute/cloud_run.py``, dispatched per-request, `[verified]`). So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a hybrid** presign/redirect. §4 answers it with numbers.

**Read to get current:** [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521 ? the `phase_placement`` knob this sequences on), [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

## 2. Decisions locked

#	Decision	Source / date	Implication
L1	<b>Keep upload ? bucket</b> (streaming, #480). The durable copy exists <b>before</b> any compute runs; the bucket <b>is</b> the "reliable co-located disk" the epic wants ? no VM needed for durability.   this doc <code>[design]</code> + #480 <code>[verified]</code>   No exposure window; upload cost stays on the cheap I/O path.		
L2	<b>Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job</b> via <code>phase_placement`</code> (#521), not onto a persistent GPU. Honors <b>D7</b> (scale-to-zero) + <b>D16(a)</b> (Jobs, not Services).   #521 <code>[verified]</code> + D7/D16   The "L4-primary" vision is realized by <b>config flips</b> , not a rearchitecture.		
L3	<b>Kill the ~30 s gap by not downloading to the control-plane</b> , not by uploading onto the L4. <b>Compose #521's validation-delegation</b> (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash).   this doc <code>[design]</code>   The CP stops pulling the full file when validation is delegated; movement ? 3x?2x.		
L4	<b>Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.</b> Breakeven vs scale-to-zero is a <b>~51 % on-demand / ~31 % spot duty cycle</b> (? 2,600?4,300 runs/mo sustained) <code>[design]</code> ; we are at pilot volume. Revisit only behind a <b>new ADR</b> at sustained load.   §4.2 cost model <code>[design]</code>   Preserves per-run billing (D7/D8); avoids ~\$382?637/mo idle.		
L5	<b>Reject "upload onto the L4."</b> It saves a ~free, seconds-fast intra-region LAN pull at the cost of ~175 s idle GPU/run <b>and</b> opens a durability window.   §4.3 <code>[design]</code>   The epic's proposed direction #1 is not adopted; owner sign-off requested (§8).		
L6	<b>Win #2 (drop the re-hash) is config-guarded + reversible</b> ( <code>deployment.trust_submit_video_id`</code> , default on) and produces a <b>byte-identical video_id</b> (the GCS <code>md5Hash`</code> over the same bytes = <code>compute_video_id`</code> ).   win #2 <code>[verified]</code>   Zero-regression; one-flag rollback.		

\*(IDs are `L#`` to avoid colliding with README.md's `D#``. These refine ? never override ? D4/D7/D16.)\*

## 3. Foundation ? the ratified constraints this must live inside

The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the serving substrate. The options in §4 are scored **against** these, and any option that breaks one needs a new ADR, not just this doc:

Ratified decision	What it pins	Consequence for #524
<b>D4</b>	In cloud-serving, <b>stitch runs on Cloud Run</b> (co-located, metered).   The live system's CP-stitch is a <b>deviation</b> from D4; #521 <b>realizes</b> D4. #524 is mostly <b>completing</b> an already-ratified decision, not inventing one.	
<b>D7</b>	<b>Scale-to-zero, no warm pool</b> (a ~\$500/mo idle GPU "would break per-video	

billing"). | Any persistent-GPU option (B, Service-with-min-instances) **\*\*contradicts D7\*\*** ? new-ADR territory (\$4.2). |  
 | **\*\*D16(a)\*\*** | **\*\*Cloud Run Jobs\*\*** (not Services-with-GPU), **\*\*bucket-poll\*\*** for completion. |  
 Option A (Service-with-GPU inbound upload) **\*\*contradicts D16(a)\*\***. The Job model is load-bearing (reuses `JobWaiting`/`claim\_due\_job`). |  
 | **\*\*D16(b)\*\*** | Stitch encode = libx264 for parity; **\*\*NVENC deferred behind a knob\*\***. | #521 lands exactly that knob (`cloud\_stitch\_encoder`), using this CUJ as the cost data D16(b) said to revisit on. |  
 | **\*\*D16(c)\*\*** | Input cap hard-reject ` $< 2 \text{ GB}$ `. | The 1.83 GiB CUJ clip is **\*at\*** the cap; the topology must stay sane at 2 GB (upload minutes, pull seconds). |

**\*\*Key reframing this foundation gives us:\*\*** the owner's "reliable persistent disk" requirement is **\*already met\*** by the **\*\*regional GCS bucket\*\*** ? a durable, co-located, cheap "disk" the upload already writes to directly. The persistent-VM instinct solves a problem (durable local disk) that the bucket-first design doesn't have.

## 4. Design / architecture ? resolving the topology question

### 4.1 The measured pipeline (evidence base) `[verified]`

From runlog \$9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s / 4K-class):

Phase	Wall	Machine	GPU needed?	Notes
Upload (client ? bucket)	<b>**~2m55s**</b>	client uplink	no	1.83 GiB @ ~84 Mbps ? <b>**owner bandwidth**</b> , not infra
Accept ? validate start	~30 s	control-plane	no	<b>**the gap**</b> = CP re-download to scratch (`acquire_local_input` + re-hash)
Validation (#512 one-pass)	<b>**183 s**</b>	control-plane 2 vCPU	no	(CPU-bound)   scene_cut/blur/relevance shared decode
Dispatch ? worker ready	~55 s	Cloud Run Job	?	image pull + GCSFuse + torch/cuda init (cold start)
Detection (WASB)	<b>**161.6 s**</b>	L4 (cuda)	<b>**yes**</b>	the only genuinely GPU-bound phase
Ingest + finalize	~4 s	control-plane	no	intervals ? DB
Compile / stitch	<b>**~12m55s**</b>	control-plane 2 vCPU	no	(NVENC would need GPU)   11x 4K re-encode+watermark

**\*\*Two facts that drive the whole design:\*\***

- **\*\*Only \*detection\* is GPU-bound.\*\*** Validation is CPU/decode-bound; stitch is encode-bound (NVENC **\*wants\*** a GPU but libx264 doesn't **\*need\*** one). So "collapse onto the L4" is about **\*\*co-locating CPU work next to the GPU on a box that's already up and already holding the bytes\*\***, not about the GPU per se.
- **\*\*Upload dwarfs compute\*\*** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a GPU allocated **\*during upload\*** roughly **\*\*doubles\*\*** the GPU-allocated time for zero compute.

### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08) `[verified]` rates / `[design]` per-run

Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-lang-client-0306026826`):

Resource	Rate	Source
Cloud Run <b>**L4 GPU**</b> (no zonal redundancy)	<b>**\$0.00022404 / s**</b>	\$0.806/hr   Catalog <b>**[verified]**</b>
Cloud Run vCPU (instance/jobs)	\$0.0000216 / s	Catalog <b>**[verified]**</b>
Cloud Run memory	\$0.0000024 / GiB·s	Catalog <b>**[verified]**</b>
? <b>**L4 Job @ 8 vCPU / 32 GiB (running)**</b>	<b>**? \$1.705 / hr**</b>	(\$0.000474/s)   derived <b>**[design]**</b>
GCE <b>**g2-standard-4**</b> (4 vCPU/16 GiB + L4), on-demand	<b>**\$0.872 / hr**</b>	= <b>**~\$637 / mo**</b>   Catalog <b>**[verified]**</b>
GCE g2-standard-4, <b>**spot/preemptible**</b>	\$0.523 / hr	= ~\$382 / mo   Catalog <b>**[verified]**</b>
Cloud Run <b>**GPU Service, warm**</b> (min-instances=1, 4 vCPU/16 GiB)	? \$1.256 / hr	= <b>**~\$917 / mo**</b> idle   derived <b>**[design]**</b>

| Balanced PD | \$0.11 / GiB.mo | Catalog `[verified]` |

**\*\*Per-run GPU-allocated cost (the term that varies by topology):\*\***

Topology	GPU-allocated time / run	GPU cost / run	? vs today	Concurrency wall?
**\*\*Today\*\*** (detect-only on L4; validate+stitch on CP)	~222 s (cold+detect+io)	**\*\*~\$0.105\*\***		
?	**\*\*Yes\*\*** ? ~16 min CP CPU/run			
**\*\*#521 + win #2\*\*** (validate+detect+stitch on L4; upload?bucket)	~312 s (validate 60 + detect 162 + stitch 60 + io 30)	**\*\*~\$0.148\*\***	**\*\*+\$0.04\*\***	**\*\*No\*\*** ? CP is thin
**\*\*Upload-onto-L4\*\*** (GPU up during upload too)	~487 s (+175 s idle upload)	**\*\*~\$0.231\*\***		
**\*\*+\$0.13\*\***	No, but idle-GPU tax			
**\*\*Persistent GCE VM\*\*** (on-demand)	billed 24/7 regardless	**\*\*\$637/mo flat\*\***	breaks per-run billing	No

**\*\*Breakeven (persistent vs scale-to-zero):\*\*** a persistent on-demand g2 (\$0.872/hr) beats the scale-to-zero Job (\$1.705/hr \*while running\*) only at a **\*\*duty cycle ? \$0.872/\$1.705 ? 51 %\*\*** (spot: ? 31 %). At ~312 s/run that is **\*\*~2,600?4,300 runs/month sustained, 24/7\*\*** `[design]`. Pilot volume is \*handfuls/day\*. **\*\*Scale-to-zero wins by ~2 orders of magnitude\*\*** and preserves per-run billing (D7/D8). A persistent GPU is also **\*\*idle >95 % of the time\*\*** at our volume ? the ~\$382?637/mo would be almost pure waste.

### 4.3 The three topology options, scored

**\*\*Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.\*\***

- **\*Latency:** scale-to-zero ? same ~55 s cold start as the Job; **\*\*min-instances=1\*\*** removes cold start but pins a **\*\*~\$917/mo warm idle GPU\*\***.
- **\*Cost:** to accept the upload, the GPU Service instance must be **\*\*up for the whole 175 s upload\*\*** ? the idle-GPU tax of \$4.2 on **\*every\*** request, plus the warm-idle bill if **min-instances>0**.
- **\*Ops/parity:** **\*\*contradicts D16(a)\*\*** (Jobs, not Services) and **\*\*D7\*\*** (scale-to-zero) ? abandons the ``JobWaiting`/`claim_due_job`/bucket-poll` machinery that's shipped and tested. A Service also needs request-timeout/concurrency tuning the batch Job sidesteps.
- **\*\*Verdict: reject.\*\*** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.

**\*\*Option B ? GCE L4 VM (persistent, reliable PD).\*\***

- **\*Reliability:** real, attached, reliable PD ? the owner's stated appeal. But **\*\*the bucket already gives us durable co-located storage\*\*** (L1), so the PD solves a non-problem.
- **\*Cost:** **\*\*\$382?637/mo flat\*\***, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **\*\*preemptible\*\*** ? directly against the "reliable" motivation.
- **\*Ops:** we now **\*\*own a VM\*\*** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.
- **\*\*Verdict: reject at current volume; owner-gated future ADR\*\*** if sustained load crosses the \$4.2 breakeven.

**\*\*Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.\*\***

- **\*The fatal timing problem:** compute needs the **\*\*whole\*\*** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up **\*to receive\*** the upload means the GPU is **\*\*allocated and idle for the entire 175 s upload\*\*** ? the \$4.2 idle tax, same as A. Upload and compute **\*\*cannot overlap\*\***.
- **\*Durability:** introduces an **\*\*exposure window\*\*** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **\*\*loses the source\*\*** unless the client re-uploads (see §4.5).
- **\*\*Verdict: reject.\*\*** It pays the idle-GPU tax **\*and\*** takes on a durability window, to save a transfer that is ~free and seconds-fast.

**\*\*Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).\*\***

- Keep **\*\*upload ? bucket\*\*** (cheap, bounded-memory, durable-immediately). Keep the **\*\*scale-to-zero L4 Job\*\***. Move **\*\*validate + stitch\*\*** onto the Job via ``phase_placement`` (#521). Make the CP **\*\*stop downloading\*\*** (delegate validation + trust the md5, win #2). The Job pulls the file **\*\*once\*\*** from the same-region bucket at LAN speed.
- **\*Data movement:** **\*\*1 client upload + 1 intra-region LAN pull\*\*** ? the floor. (Detection and validation/stitch all run in **\*\*one\*\*** Job execution, so it's one pull, not two.)

- **\*Cost:** +\$0.04/run GPU vs today, **\*\*~16 min CP CPU/run\*\*** (the actual scaling win). No idle tax, no warm bill, no VM.
- **\*Durability:** **\*\*no exposure window\*\*** ? the bucket copy exists before the Job starts.
- **\*Ops/parity:** **\*\*honors D4/D7/D16\*\***; reuses the shipped dispatch/poll path; every phase is a **\*\*config flip with an in-process fallback\*\*** (#521's ``_dispatch*_remote`` already logs-and-falls-back to in-process when the box isn't cloud-wired).

> **\*\*The one-line intuition:\*\*** the bucket **\*is\*** the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.

#### 4.4 The minimal control-plane surface (the "thin router")

After A?, the control-plane does **\*\*only\*\*** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):

```
| Keep on the control-plane | Why it must stay | Code anchor `[verified]` |
|---|---|---|
| **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |
| **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the durable-copy producer | `backend/api/uploads.py` |
| **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the async job lifecycle (`JobWaiting`/`claim_due_job`) | `backend/orchestration.py`, `backend/compute/cloud_run.py` |
| **DB writes** (jobs, videos, `play_segments`, validation verdict) | The control-plane owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database*.py` |
| **SPA serving + job status** (`/api/jobs/{id}` polling) | The UX surface | `backend/api/jobs.py`, `backend/ui/` |
| **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP just signs | `backend/main.py::get_highlights` |
| **~Validation~** | ? **L4** via `phase_placement.validation` (#521) | delegated |
| **~Stitch/compile~** | ? **L4** via `phase_placement.compile` (#521) | delegated |
| **~Full-file download + re-hash~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |
```

#### 4.5 The async copy-to-bucket durability model (spec ? needed **\*only\*** if a future upload-onto-L4 path is ever adopted)

The recommended A? design **\*\*has no async-copy step\*\*** (the bucket copy is the **\*upload\***, produced before compute). This section specs the model the epic asked for, so that if the owner overrides §8 and adopts upload-onto-L4, the durability semantics are pinned rather than improvised:

- **\*\*When the mirror runs:\*\*** kick a background ``storage.put(local ? gs://?/uploads/?)`` **\*\*immediately on upload-complete\*\*** (not "during segmentation" ? starting it later only lengthens the exposure window; there's no compute reason to delay it). Run it concurrently with `validation/detect``.
- **\*\*Failure/resume:\*\*** retry with capped exponential backoff; the mirror is **\*\*idempotent\*\*** (same object key = the md5). On worker preemption before the mirror confirms, the run is **\*\*lost\*\*** ? the job must fail with a **\*retryable\*** status so the client can re-submit (the source is still on the client). **\*\*Never\*\*** report success until the mirror is **\*\*confirmed\*\*** (object exists + size/md5 match).
- **\*\*Exposure window:\*\*** upload-complete ? mirror-confirmed, the **\*\*only\*\*** durable copy is L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds ``[design]``). A preemption in-window = re-upload. **\*\*This window is the entire reason A? is preferred\*\*** ? A?'s window is **\*zero\***.
- **\*\*Ack contract:\*\*** the client keeps the source until the API returns ``mirror_confirmed:true``; only then may it release it. (A? needs no such contract ? bucket-first.)

### 5. Phased migration ? every step a config flip with a bucket-mediated fallback

Built on #521's ``resolve_phase_placement(cfg, phase)`` so **\*\*placement is config, not code\*\***. Each phase is independently reversible (``control_plane`` ? ``cloud_run_l4``) and falls back to in-process on a non-cloud box.

```

| Phase | Change | Mechanism | Fallback | Status |
|---|---|---|---|---|
| **P0** | *Today* ? detect on L4; validate+stitch on CP | `compute_target=cloud_run` | in-
process | shipped `[verified]` |
| **P1** | **Stitch ? L4** | `phase_placement.compile=cloud_run_l4` (+ `cloud_stitch_encoder`) |
in-process stitch | **#521 (open)** |
| **P2a** | **Drop the CP re-hash** (trust submit-time md5) | `deployment.trust_submit_video_id`
(default on) | re-hash on the drain | **this PR (win #2)** |
| **P2b** | **Validation ? L4** (CP stops running validators) |
`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment) | CP validates |
#521 (open) |
| **P2 net** | **CP never downloads the file** ? the ~30 s gap dies; movement 3x2x | P2a ? P2b
together | either half alone degrades gracefully | needs #521 ***** win #2 merged |
| **P3** | Worker serializes its **validation verdict** back (per-validator results + #518 cache
warm), so a worker-side reject renders the same SPA detail as a CP reject | new
`report.json`/marker field + CP ingest | worker reject = rc?0 + message (today) | **follow-up**
(see $6) |
| **P4** | *If and only if* sustained load crosses the $4.2 breakeven | persistent worker
(VM/Service) | scale-to-zero Job | **owner-gated, new ADR** |

```

\*\*The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's knob + this PR's win #2.\*\* No rearchitecture, no new infra, fully reversible.

## 6. Honest limits ? what this does NOT do / does NOT prove

- \*\*A green suite does not prove the cloud path\*\* ? the tests are fully-mocked/offline (a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are validated \*\*in the cost model + code paths\*\*, not on live infra. A live CUJ on rev ? `00020` (owner-side, D17 budget) is required to \*confirm\* the gap is gone and the movement is 2x.
- \*\*Network-egress "free intra-region" is `[researched]`, not live-verified\*\* ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were \*not\* free, A? is \*still\* cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.
- \*\*Win #2 changes nothing when no trusted id is present\*\* ? local/appliance uploads and non-GCS direct-URI submits (`submit\_time\_video\_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).
- \*\*P3 (worker validation-verdict serialization) is a real gap #521 leaves open\*\* `[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-validator `ValidationResult` list (`result['results']` the SPA renders) and the #518 `validation\_cache` warm are \*\*not\*\* reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.
- \*\*SQLite is the residual CP scaling limit\*\* ? the thin router is stateless \*except\* the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).
- \*\*The #513 asymmetry reverses under P2b\*\* `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards `validation.\*` as `--set`. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.

## 7. Deliverables & progress tracker ? source of truth

Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from `origin/master`, no stacking.

```

| ID | Deliverable | Depends on | Gated? | Status | PR |
|----|-----|-----|-----|-----|----|
| P0 | **This design doc** (resolves topology A/B/C; recommends A?; specs durability + phased
migration) | ? | owner-review | ? (this PR) | #524 |
| P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5;
`deployment.trust_submit_video_id` (default on), reversible | ? | No | ? (this PR) | #524 |
| P2 | **Win #1** ? validation-on-L4 (`phase_placement.validation`) | #521 | No | ? **delivered
by #521** ? not duplicated here | #521 |
| P3 | **CP-download-avoidance measured live** ? confirm the ~30 s gap is gone + movement is 2x
on rev ? `00020` | #521 + P1 merged | ? owner-side (live spend, D17) | ? | ? |
| P4 | **Worker validation-verdict serialization** ($6 gap) so P2b keeps rich SPA verdict + #518

```

cache | #521 | No | ? follow-up | ? |  
| P5 | \*\*Persistent-GPU re-evaluation\*\* only if load crosses \$4.2 breakeven | volume | ? new ADR  
| ? | ? |

## 8. Open questions ? owner / external

**Blocking (owner sign-off):**

- Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it; this doc recommends against it with the \$4.2/\$4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. \*If you still want upload-onto-L4\* (e.g. for a future no-bucket edge deployment), \$4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).
- Flip P2b (validation?L4) before or after P3 (verdict serialization)?** P2b alone offloads CPU but degrades the reject-UX + skips #518 cache-warm (\$6). Recommend: land P3 first, then P2b.

**Nice-to-know:**

- Is pilot volume expected to cross the \$4.2 breakeven within the planning horizon? (Decides whether P5 is ever live.)

## 9. Definition of done

- [x] Design doc resolves the topology question (A/B/C scored) with a clear recommendation (A?) + quantified cost/latency/reliability. ``[this PR]``
- [x] Minimal control-plane surface defined (\$4.4). ``[this PR]``
- [x] Async copy-to-bucket durability model specified with failure/resume + exposure window (\$4.5). ``[this PR]``
- [x] Phased migration laid out on #521's ``phase_placement``, each step a config flip with a bucket-mediated fallback (\$5). ``[this PR]``
- [x] Win #2 landed (config-guarded, reversible, tests green). ``[this PR]``
- [ ] Win #1 confirmed delivered by #521 (not duplicated). ``[cross-PR]``
- [ ] Live CUJ confirms the gap is gone + movement is 2x (P3, owner-side). ``[gated]``
- [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.

## 10. References

**Internal code anchors ``[verified]``:** ``backend/api/uploads.py`` (streamed md5), ``backend/orchestration.py:220-239`` (``submit_time_video_id``, #484) / ``:741-763`` (re-hash) / ``_maybe_dispatch_remote``, ``backend/api/jobs.py:80-153`` (submit-time cache already trusts the md5), ``backend/api/job_worker.py:122-193`` (``_job_to_request``/``_run_claimed_job``), ``backend/compute/cloud_run.py`` (dispatch + bucket-poll), ``backend/config/placement.py`` (#521 ``resolve_phase_placement``), ``backend/worker/__main__.py`` (worker validation + ``--skip-validation``).

**Internal docs:** `[PHASE_PLACEMENT.md](./PHASE_PLACEMENT.md)` (#521), `[README.md](./README.md)` (ADR-014, D4/D7/D16), ``runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`` §9 (evidence).

**External ``[verified]`` rates:** GCP Cloud Billing Catalog API (``cloudbilling.googleapis.com``, asia-southeast1, USD, 2026-07-08). **``[researched]``:** same-region GCS?Cloud Run egress treatment (not live-fetchable here).

---

### Changelog

- ``2026-07-08`` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs + bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased migration on #521's ``phase_placement``; win #2 landed alongside.

### 92. user (2026-07-08T06:04:14.804Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md (file state is current in your context ? no need to Read it back)

### 93. assistant (2026-07-08T06:04:27.659Z)

Now I must reconcile the doc's internal links: `PHASE\_PLACEMENT.md` only exists on #521's unmerged branch, \*\*not on master\*\* ? so my relative links to it would break the `link-check` CI. Let me check exactly what `check\_md\_links` validates.

#### 94. user (2026-07-08T06:04:27.872Z)

```
1 """Markdown internal-link checker ? fail CI on a broken internal link in a LIVE doc.
2
3 Scans tracked ``*.md`` files (via ``git ls-files``) and, for each ``[text](dest)`` /
4 ``![alt](dest)``
5 link whose destination is INTERNAL (a repo path and/or a ``#anchor``), verifies that
6 - the target file exists (resolved relative to the linking file, or the repo root for
7 ``/abs``), and
8 - any ``#anchor`` matches a GitHub-style heading slug in the target markdown file.
9
10 SKIPPED: external links (``http(s)``/``mailto``/``tel``/``ftp``/protocol-relative
11 ``//``), links inside
12 fenced code blocks, and any file under ``docs/archives/****`` ? archived docs are frozen
13 snapshots whose
14 relative links are historical-by-design (see ``docs/DOC_STATUS.md`` §2). Mirrors
15 ``tools/leak_scan.py``:
16 run ``python -m tools.check_md_links`` (exit 1 on any broken internal link in a live
17 doc).
18
19 """
20
21 from __future__ import annotations
22
23 import argparse
24 import re
25 import subprocess
26 import sys
27 from pathlib import Path
28 from typing import Dict, List, Optional, Sequence, Set, Tuple
29
30 #: Source docs under these prefixes are NOT scanned ? archived = frozen snapshots whose
31 relative
32 #: links are historical-by-design (DOC_STATUS.md §2 step 5).
33 EXCLUDE_PREFIXES: Tuple[str, ...] = ("docs/archives/",)
34 #: This tool + its test legitimately embed sample/broken links ? never scan them.
35 SELF_EXCLUDE: Tuple[str, ...] = ("tools/check_md_links.py",
36 "tests/test_check_md_links.py")
37
38 _LINK_RE = re.compile(r"!?\[[^\]]*\]\(\\s*([^\)]+?)\\s*\)")
39 _HEADING_RE = re.compile(r"^(#{1,6})\s+(.?)\s*#\s*$")
40 _FENCE_RE = re.compile(r"^\\s*(?:``|~~~)")
41 #: a scheme (``http:`` , ``mailto:`` , ?) or protocol-relative ``//`` ? external, not our
42 concern.
43 _EXTERNAL_RE = re.compile(r"^(?:[a-zA-Z][a-zA-Z0-9+\\.\\-]*:|//)")
44
45 def slugify(heading: str) -> str:
46 """GitHub-style heading ? anchor slug (matches `github-sluggger`): lowercase, drop
47 punctuation
48 (keep word chars / spaces / hyphens), then replace EACH space with a hyphen ?
49 crucially NOT
50 collapsing, so a heading with removed punctuation (e.g. 'a / b' ? 'a b') yields
51 'a- b' like GitHub."""
52 s = re.sub(r"[^\\w\\s-]", "", heading.strip().lower())
53 return s.replace(" ", "-")
54
55 def heading_slugs(text: str) -> Set[str]:
56 """The set of GitHub anchor slugs for a markdown file's headings (with -1/-2 dedup
57 suffixes)."""
```

```

47 counts: Dict[str, int] = {}
48 out: Set[str] = set()
49 in_fence = False
50 for line in text.splitlines():
51 if _FENCE_RE.match(line):
52 in_fence = not in_fence
53 continue
54 if in_fence:
55 continue
56 m = _HEADING_RE.match(line)
57 if not m:
58 continue
59 base = slugify(m.group(2))
60 n = counts.get(base, 0)
61 out.add(base if n == 0 else f"{base}-{n}")
62 counts[base] = n + 1
63 return out
64
65
66 def _split_dest(dest: str) -> Tuple[str, str]:
67 """`path#anchor` (with an optional `title` suffix) ? `(path, anchor)`."""
68 dest = dest.strip().split()[0] # drop a trailing "title"
69 path, _, anchor = dest.partition("#")
70 return path, anchor
71
72
73 def check_file(repo: Path, rel: str, slug_cache: Dict[str, Set[str]]) -> List[Tuple[int,
str]]:
74 md = repo / rel
75 base = md.parent
76 findings: List[Tuple[int, str]] = []
77 in_fence = False
78 for i, line in enumerate(md.read_text(encoding="utf-8",
errors="replace").splitlines(), 1):
79 if _FENCE_RE.match(line):
80 in_fence = not in_fence
81 continue
82 if in_fence:
83 continue
84 for m in _LINK_RE.finditer(line):
85 dest = m.group(1).strip()
86 if not dest or _EXTERNAL_RE.match(dest):
87 continue
88 path, anchor = _split_dest(dest)
89 if path == "":
90 target = md # pure same-file anchor
91 elif path.startswith("/"):
92 target = repo / path.lstrip("/")
93 else:
94 target = (base / path).resolve()
95 if path != "" and not target.exists():
96 findings.append((i, f"broken link -> {dest} (no file {path})"))
97 continue
98 if anchor and target.suffix.lower() == ".md" and target.is_file():
99 key = str(target)
100 if key not in slug_cache:
101 slug_cache[key] = heading_slugs(
102 target.read_text(encoding="utf-8", errors="replace")
103)
104 if slugify(anchor) not in slug_cache[key]:
105 findings.append((i, f"broken anchor -> {dest} (no heading
#{anchor})"))
106 return findings
107
108

```

```

109 def tracked_md(repo: Path) -> List[str]:
110 out = subprocess.run(
111 ["git", "ls-files", "*.md"], cwd=str(repo), capture_output=True, text=True
112)
113 files = sorted({f.strip() for f in out.stdout.splitlines() if f.strip()})
114 return [
115 f for f in files if not f.startswith(EXCLUDE_PREFIXES) and f not in SELF_EXCLUDE
116]
117
118
119 def scan_repo(repo: Path) -> List[Tuple[str, int, str]]:
120 slug_cache: Dict[str, Set[str]] = {}
121 results: List[Tuple[str, int, str]] = []
122 for rel in tracked_md(repo):
123 try:
124 for lineno, reason in check_file(repo, rel, slug_cache):
125 results.append((rel, lineno, reason))
126 except OSError:
127 continue
128 return results
129
130
131 def main(argv: Optional[Sequence[str]] = None) -> int:
132 ap = argparse.ArgumentParser(
133 description="Check internal markdown links in LIVE docs (excludes
134 docs/archives/)."
135)
136 ap.add_argument(
137 "--root",
138 type=Path,
139 default=Path(__file__).resolve().parents[1],
140 help="repo root (default: the parent of tools/)",
141)
142 args = ap.parse_args(list(argv) if argv is not None else None)
143 results = scan_repo(args.root)
144 if not results:
145 print("[check-md-links] clean ? no broken internal links in live docs.")
146 return 0
147 print(f"[check-md-links] {len(results)} broken internal link(s):", file=sys.stderr)
148 for rel, lineno, reason in results:
149 print(f" {rel}:{lineno}: {reason}", file=sys.stderr)
150 return 1
151
152 if __name__ == "__main__":
153 raise SystemExit(main())
154

```

#### 95. assistant (2026-07-08T06:04:48.322Z)

`check\_md\_links` flags relative `[text](./PHASE\_PLACEMENT.md)` links, and that file only exists on #521's unmerged branch ? so those links break CI on my master-based branch. External `https://` links and backticked paths are skipped. Let me find and fix every such link.

#### 96. user (2026-07-08T06:04:50.834Z)

6:> **Relation to existing docs:** extends [COMPUTE\_DECOUPLED\_SERVING/README.md](./README.md) (ADR-014) and [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521); it does **not** supersede them ? it answers the *topology* question they left open and sequences the migration on top of `phase\_placement`.

30: **Read to get current:** [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521 ? the `phase\_placement` knob this sequences on), [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

219: **Internal docs:** [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521), [README.md](./README.md) (ADR-014, D4/D7/D16), `runlogs/DEPLOY\_REPORT\_cloud-serving\_validation-

latency\_2026-07-07.md` §9 (evidence).

#### 97. user (2026-07-08T06:05:04.681Z)

6:> **\*\*Relation to existing docs:\*\*** extends [COMPUTE\_DECOUPLED\_SERVING/README.md](./README.md) (ADR-014) and [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521); it does **\*\*not\*\*** supersede them ? it answers the **\*topology\*** question they left open and sequences the migration on top of `phase\_placement`.  
30:**\*\*Read to get current:\*\*** [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521 ? the `phase\_placement` knob this sequences on), [README.md](./README.md) §2 decisions **\*\*D4/D7/D16\*\*** (the ratified constraints that bound the options), and the runlog §9 (the evidence).  
219:**\*\*Internal docs:\*\*** [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md) (#521), [README.md](./README.md) (ADR-014, D4/D7/D16), `runlogs/DEPLOY\_REPORT\_cloud-serving\_validation-latency\_2026-07-07.md` §9 (evidence).

#### 98. user (2026-07-08T06:05:12.695Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 99. user (2026-07-08T06:05:17.662Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 100. user (2026-07-08T06:05:27.087Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 101. assistant (2026-07-08T06:05:29.833Z)

Now let me add the doc to the two indexes (COMPUTE\_DECOUPLED\_SERVING/README.md and docs/README.md) and the DOC\_STATUS register. Let me check how those are structured.

#### 102. user (2026-07-08T06:05:36.680Z)

=== CDS/README doc-refs to PHASE/runlogs (how #521 indexed) ===  
115:| M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **\*\*?**  
APPROVED 2026-06-21**\*\*** |  
129:**\*\*Testing strategy is a first-class deliverable\*\*** ?  
[`TESTING\_STRATEGY.md`](TESTING\_STRATEGY.md). **\*\*Run logs / sample runs for posterity\*\*** ?  
[`runlogs/`](runlogs/).  
169:- 2026-06-21 ? **\*\*M5 code landed\*\*** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`\_enforce\_startup\_or\_exit` ? exit 1) + the worker (`cmd\_infer`/`cmd\_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **\*\*L4 profile-parity test\*\*** (`tests/test\_profile\_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. **\*\*?** M5 ? ? the real-GPU truly-local runlog (L5) is owner-side**\*\*** (template pre-staged in `runlogs/`).  
=== #521 README diff (how it added its doc) ===  
9:> **\*\*#521 (2026-07-08):\*\*** phase?machine placement is now **\*\*config-driven\*\*** and compile/stitch is **\*\*relocatable to the L4 worker\*\*** (NVENC) ? see [PHASE\_PLACEMENT.md](./PHASE\_PLACEMENT.md). This refines **\*\*D16(b)\*\*** (NVENC-encode deferred behind a config knob) now that the live CUJ (§9 of the validation-latency runlog) supplies the cost data D16(b) said to revisit on.

#### 103. user (2026-07-08T06:05:37.651Z)

```

=== docs/README.md COMPUTE_DECOUPLED refs ===
20:- [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked
project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to
DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never
branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend`
(what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU,
in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run
GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI).
Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY
proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a
[runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage
ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). $7 M0?M10 tracker is the source
of truth.
36:- [data_pipeline/](data_pipeline/README.md) ? **the isolated data pillar**: how footage is
recorded (`VIDEO_RECORDING_GUIDELINES`), golden sets are created + which videos
(`GOLDEN_SET_IMPLEMENTATION` · `GOLDEN_SET_PHASE1_VIDEOS` · `GOLDEN_VIDEOS`), and data is
shared/ingested (`GOLDEN_DATA_SHARING`). Deliberately **no segmentation/stitching/model** detail
? that's the ML pillar (top level + `COMPUTE_DECOUPLED_SERVING/`). The **annotation-vendor
(Roora)** engagement is archived under `archives/roora-vendor/`.
=== DOC_STATUS register section ===
9:## 0. How to use this
10:- **Reviewing now?** Read $4 (the open findings/actions) ? that's what needs attention. $3 is
the full register.
12:- **This register does NOT duplicate completeness detail** ? it indexes lifecycle + archive-
readiness and points to each tracked doc's $7.
14:## 1. Lifecycle legend
17:## 2. Archival due-diligence checklist ? *the repeatable process (do NOT just `git mv`)*
28:## 3. The register
30:### 3a. Tracked projects ? lifecycle docs (archive when DONE; each doc's $7 is the
completeness source-of-truth)
34:| `COMPUTE_DECOUPLED_SERVING/` | **IN PROGRESS** ? M0 approved + **M1?M9 LARGELY SHIPPED**
(#279?#290; M5 #286, M6 #287, M8 #288, M9-auth #290 all merged, default-OFF); Gen-0 weights
pushed (`weights/0.1.0-l4/`) | No | M11 flywheel + M12 gdrive-api (owner residual:
counsel/prices/OAuth/real-run within the D17 cap) |
57:### 3b. Foundational / strategy references (LIVING ? archive only if **superseded**, not when
"done")
60:| `PLATFORM_ARCHITECTURE.md` | Strategy/research | The `ComputeBackend`/`StorageBackend`
registries ? realized by COMPUTE_DECOUPLED_SERVING |
69:### 3c. Reports & terminal records (archive candidates once their track wraps)
78:### 3d. Conventions / guidelines / templates / runbooks (permanent ? do not archive)
81:### 3e. Already archived (`docs/archives/`)
84:### 3f. ADR log ? `docs/archives/decisions/DECISIONS.md`
85:**ADR-001 ? ADR-014** (14, all ratified). The compute/serving lineage: **ADR-005 ? 008 ? 009
? 010 ? 011 ? 012 ? 013 ? 014**. ADR-014 was **RATIFIED 2026-06-21** (owner M0 sign-off for
COMPUTE_DECOUPLED_SERVING).
87:## 4. Open due-diligence findings / actions (2026-06-21 review)
95:6. **? RESOLVED (2026-06-21) ? `COMPUTE_DECOUPLED_SERVING` M0 approved + M1?M4 SHIPPED**
(#279/#280/#282/#283); all $8 gating Qs locked (D7?D17); M5+ default-OFF code batch authorized
(owner residual: counsel ToS/DPA, real GCP prices, OAuth app, real cloud run within the D17
$100/?10k cap).
97:## 5. Maintenance
98:Update a row when its doc changes lifecycle; run $2 before any archive; re-review $4 at each
clean-slate. This register + the $2 checklist are the operational form of the `CLAUDE.md`
archive policy and [[working-agreement-tracked-project-docs]].

```

#### 104. assistant (2026-07-08T06:06:01.709Z)

The task isn't a terminal-state archival flip, so DOC\_STATUS/docs/README changes aren't required ? but a discoverability pointer in the CDS README (mirroring how #521 added its own) is the right convention. Let me read the README top to place it with minimal merge-conflict surface with #521.

#### 105. user (2026-07-08T06:06:03.062Z)

```

1 # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage

```

```

2
3 > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
Owner: Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4 > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5 > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6 > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7 > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
compute-decoupled serving because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8 > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10 ---
11
12 > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user
confirms (D15).
13
14 ## 0. TL;DR

```

#### 106. user (2026-07-08T06:06:19.197Z)

The file  
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\README.md has  
been updated successfully. (file state is current in your context ? no need to Read it back)

#### 107. assistant (2026-07-08T06:06:29.735Z)

Design doc complete and indexed. Let me update tasks and move to win #2 implementation. First,  
read `DeploymentConfig` to place the flag idiomatically.

#### 108. user (2026-07-08T06:06:30.243Z)

Updated task #3 status

#### 109. user (2026-07-08T06:06:30.754Z)

Updated task #5 status

#### 110. user (2026-07-08T06:06:31.521Z)

Updated task #4 status

#### 111. user (2026-07-08T06:06:32.166Z)

```

454 class DeploymentConfig(BaseModel):
455 """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456 knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458 **Pure / additive:** the empty default profile (````) applies NO preset, so the
pipeline
459 behaves exactly as before this section existed. A profile is opt-in (``config.json``
460 ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461 **suggests** one (D15 ? cloud spend is never entered silently). Nothing in the core

```

```

reads
462 these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463 (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465 Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467 # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468 profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469 # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470 # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471 compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472 # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473 # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474 # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475 # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476 # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477 max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478 # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479 # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480 # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481 # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482 # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483 remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486 class BillingConfig(BaseModel):
487 """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489 **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490 today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491 ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492 version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493 inserts a real, calibrated version and points ``pricebook_version`` at it.

```

## 112. assistant (2026-07-08T06:06:34.669Z)

Now I'll add the `trust\_submit\_video\_id` flag to `DeploymentConfig`:

## 113. user (2026-07-08T06:06:41.991Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

## 114. assistant (2026-07-08T06:06:44.816Z)

Now the orchestration change. Let me read the exact `\_execute\_processing` signature/shim and hash block once more to edit precisely.

### 115. user (2026-07-08T06:06:45.875Z)

```
708 def _execute_processing(
709 config=None, db=None, req=None, progress=None
710) -> Dict[str, Any]:
711 """The full hash ? validate ? segment ? report pipeline for one video.
712
713 Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714 module-level ``global_config``/``db_instance`` (backward-compatible with existing
715 tests that pass only ``req``). Route handlers always pass explicit params; older
716 test code that calls ``_execute_processing(req)`` still works.
717
718 ``progress`` is an optional callable(stage: str) used to surface coarse job
719 progress.
720
721 Raises on unexpected internal errors ? the caller records those as a job failure
722 """
723 import backend.main as _main # call-time seam: main's globals/model stay patchable
724
725 # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
726 # shift positional args ? req arrives in position 0 (config), progress in position 1
727 (db).
728 if config is not None and not isinstance(config, AppConfig):
729 # Old signature: _execute_processing(req, progress=None)
730 if req is None and db is not None and isinstance(db, _main.ProcessRequest):
731 req = db
732 db = None
733 elif req is None and isinstance(config, _main.ProcessRequest):
734 req = config
735 config = None
736 # Resolve config/db from module globals when not passed explicitly
737 if config is None:
738 config = _main.global_config
739 if db is None:
740 db = _main.db_instance
741
742 notify = progress or (lambda stage: None)
743
744 notify("hashing")
745 # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
746 (identity ?
747 # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
748 original
749 # req.video_path (the source URI) is kept for the DB record + report; only the file-
750 reading
751 # consumers (hash / validators / segmenter) use the resolved local path.
752 local_video = ""
753 try:
754 local_video = storage.acquire_local_input(
755 req.video_path, getattr(config, "storage", None)
756)
757 video_id = _main.compute_video_id(local_video)
758 except Exception:
```

### 116. assistant (2026-07-08T06:06:58.721Z)

I need to confirm the format of `submit\_time\_video\_id`'s value ? if the GCS backend returns a base64 md5 rather than hex, it wouldn't equal `compute\_video\_id` and my guard must catch that. Let me check the gcs `stat().md5` implementation.

### 117. user (2026-07-08T06:07:00.801Z)

```
24: def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25: """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB key)
26: have no md5Hash ? None; a malformed value is treated as absent, never raised."""
27: if not md5_b64:
```

```
31: raw = base64.b64decode(md5_b64, validate=True)
83: md5=_md5_hex(getattr(b, "md5_hash", None)),
```

#### 118. user (2026-07-08T06:07:06.567Z)

```
24 def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25 """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26 is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27 have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28 if not md5_b64:
29 return None
30 try:
31 raw = base64.b64decode(md5_b64, validate=True)
32 except (binascii.Error, ValueError):
33 return None
34 return raw.hex() if len(raw) == 16 else None
35
36
37 class GCSStorage(StorageBackend):
38 def __init__(
39 self,
40 *,
41 client: Any = None,
42 project: Optional[str] = None,
43 signed_url_ttl: int = 3600,
44 **_ignored,
45) -> None:
46 self._ttl = signed_url_ttl
47 if client is not None:
48 self._client = client
49 return
50 storage_module: Any = import_module("google.cloud.storage")
51 self._client = storage_module.Client(project=project)
52
53 def _blob(self, ref: StorageRef):
54 return self._client.bucket(ref.bucket).blob(ref.key)
55
56 def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57 if dst is None:
58 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59 import os
60
61 dst = str(dst)
62 if os.path.exists(dst) and not overwrite:
63 return dst
```

#### 119. assistant (2026-07-08T06:07:23.115Z)

Confirmed: `\_md5\_hex` converts GCS's base64 md5 to a \*\*32-char hex\*\* digest "so the two are directly comparable (#484)", returning None for composite objects. So `submit\_time\_video\_id` is byte-identical to `compute\_video\_id` or None ? my hex-32 guard is exactly right. Let me find all `\_execute\_processing` callers before editing.

#### 120. user (2026-07-08T06:07:25.218Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job_worker.py:9:every cross-
function seam ? `db_instance`, `global_config`, `_execute_processing`,
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job_worker.py:109: is the
segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job_worker.py:137: # v8
compute-picker: the worker runs _execute_processing ? _maybe_dispatch_remote, which
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job_worker.py:171:
```

```

result = _main._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:3:The full hash ->
validate -> segment -> report pipeline (`_execute_processing`), the
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:13:...))` and
`import backend.main as m; m._execute_processing(...)` keep working unchanged.
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:708:def
_execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:716: test code
that calls `_execute_processing(req)` still works.
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:723: # Backward-
compatible shim: if called as _execute_processing(req) (old pattern),
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py:726: # Old
signature: _execute_processing(req, progress=None)
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\main.py:231:# backend.main ? `import
backend.main as m; m._execute_processing(...)` the job worker's
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\main.py:238: _execute_processing,
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\scripts\nightly_reelcount.py:430: #
backend/main.py:._execute_processing) ? play_segments has an FK to videos.id, so a
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:151: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:170:
m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:196:
m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:213:
m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:230:
m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:249:
m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:262: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:281: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:307: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:352: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:376: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:417: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:446: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:460:
out = m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:501: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:544: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:575: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:593:
out = m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:756: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:786: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:807: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:838: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:857: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:884: out

```

```

= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:909: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:929: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:951: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:975: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:1003: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:1031: out
= m._execute_processing(m.ProcessRequest(video_path=str(video)))
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py:1088: out
= m._execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:167: """Item 5
positive: direct CLI-style path (m._execute_processing) vs job-worker path
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:181: direct_out =
m._execute_processing(m.global_config, m.db_instance, req)
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:215: direct_f =
m._execute_processing(m.global_config, m.db_instance, req_f)
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:234: de =
m._execute_processing(m.global_config, m.db_instance, req_e)
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:499: """The
claimed-job dispatcher routes kind='compile' to _execute_compile (NOT _execute_processing),
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:515:
_execute_processing",
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_async_jobs.py:516: lambda
*a, **k: pytest.fail("a compile job must not hit _execute_processing"),
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_base.py:135: Exercise the exact
duck-typing patterns used in main.py (both the _execute_processing
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_base.py:181: This allows the
except blocks in main.py (_execute_processing:193-198 and the
C:\Users\avidu\AppData\Local\Temp\claude\wt524-
969\tests\test_job_worker_module_seam.py:6: (`_arm_wake_timer`, `_get_executor`, `db_instance`,
_execute_processing`)
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_main_module_budget.py:13:
(`import backend.main as m; m._execute_processing` and
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_main_module_budget.py:45:
_execute_processing",
C:\Users\avidu\AppData\Local\Temp\claude\wt524-
969\tests\test_validation_cache.py:5:configuration, ``_execute_processing`` reuses the stored
verdict instead of re-running the
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_validation_cache.py:10:These tests
drive ``_execute_processing`` with a stubbed validator suite (a spy that COUNTS how many
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_validation_cache.py:104: return
m._execute_processing(m.ProcessRequest(video_path=video, segmenter="motion", **kw))

```

## 121. assistant (2026-07-08T06:07:36.445Z)

Only one production caller (`job\_worker.py:171`); the rest are tests using positional args (unaffected by a new defaulted trailing kwarg). Now the orchestration edits ? signature param, a hex guard helper, and the hash-block change:

## 122. user (2026-07-08T06:07:42.706Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

## 123. user (2026-07-08T06:07:53.269Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 124. user (2026-07-08T06:08:02.351Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 125. assistant (2026-07-08T06:08:04.357Z)

Now wire the trusted id from the job row at the drain call site:

#### 126. user (2026-07-08T06:08:08.637Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job\_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 127. assistant (2026-07-08T06:08:26.108Z)

Now let me check the `\_maybe\_dispatch\_remote` video\_id-divergence guard (to confirm no new divergence) and read an existing `\_execute\_processing` test harness to mirror for win #2 tests.

#### 128. user (2026-07-08T06:08:27.110Z)

```
1 """Validation-result cache keyed by video_id (task 4585aa00).
2
3 ``video_id`` is the MD5 of the file bytes (backend/utils/hashing.py), so an identical-
content
4 REUPLOAD keys the same cache row. When that content was already validated under the SAME
validator
5 configuration, ``_execute_processing`` reuses the stored verdict instead of re-running
the
6 (minutes-long on a heavy 4K clip) validator suite. This is distinct from the pre-
existing
7 whole-pipeline cache, which only short-circuits on a prior SUCCESSFUL segmentation ? so
a reupload
8 after a SEGMENTATION FAILURE used to re-pay the full validation from scratch (the
2026-07-07 CUJ).
9
10 These tests drive ``_execute_processing`` with a stubbed validator suite (a spy that
COUNTS how many
11 times it actually runs) + a stubbed segmenter, so the assertions are on validation-run
count, not on
12 real decoding. Cardinal guarantees exercised: same content + same config ? validators
run ONCE across
13 two uploads; a validation-config change forces a re-run; ``force=true`` bypasses; a
cached FAIL is
14 replayed without a re-decode; a different content never reuses another video's verdict.
15 """
16
17 import logging
18
19 import backend.main as m
20 from backend.config.models import AppConfig
21 from backend.infrastructure.database import Database
22 from backend.pipeline.validators.base import ValidationResult, registry
23 from backend.results import Failure
24
25 _ORCH_LOGGER = "backend.orchestration"
26
27
28 def _pass():
29 return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
30
31
32 def _fail():
33 return [
```

```

34 ValidationResult(
35 validator_name="blur_check",
36 passed=False,
37 message="too blurry",
38 details={},
39)
40]
41
42
43 class _ValSpy:
44 """Stands in for ``registry.run_all_with_context`` and counts real invocations.
45
46 A cache HIT must NOT call this ? ``calls`` staying flat across a second upload is
the proof that
47 validation was skipped (the whole point of the cache)."""
48
49 def __init__(self, results):
50 self._results = results
51 self.calls = 0
52
53 def __call__(self, *a, **k):
54 self.calls += 1
55 return list(self._results), {}
56
57
58 class _FailSeg:
59 """A segmenter whose detection FAILS (Failure result) ? reproduces "validation
passed, detection
60 failed ? no cached segments", the exact state the whole-pipeline cache can't short-
circuit."""
61
62 def __init__(self, db_, cfg_):
63 pass
64
65 def process_video(self, *a, **k):
66 return Failure("detector crashed")
67
68
69 class _OkSeg:
70 """A segmenter that writes one play row (never reached when validation fails
first)."""
71
72 def __init__(self, db_, cfg_):
73 self.db = db_
74
75 def process_video(self, vid, path, *a, **k):
76 sid = self.db.add_play_segment(vid, 0.0, 5.0)
77 return [
78 {
79 "id": sid,
80 "start_time": 0.0,
81 "end_time": 5.0,
82 "duration": 5.0,
83 "metadata": {},
84 }
85]
86
87
88 def _setup(tmp_path, monkeypatch, *, results, seg, cfg=None, content=b"identical-
bytes"):
89 """Wire the module globals + a real (temp-file) DB + stubbed validator/segmenter,
and return
90 ``(db, spy, video_path)``. The video file's BYTES fix its video_id, so writing the
same bytes
91 twice (a 'reupload') hits the same cache key."""

```

```

92 db = Database(str(tmp_path / "t.db"))
93 monkeypatch.setattr(m, "db_instance", db)
94 monkeypatch.setattr(m, "global_config", cfg or AppConfig())
95 spy = _ValSpy(results)
96 monkeypatch.setattr(m.registry, "run_all_with_context", spy)
97 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: seg)
98 video = tmp_path / "v.mp4"
99 video.write_bytes(content)
100 return db, spy, str(video)
101
102
103 def _process(video, **kw):
104 return m._execute_processing(m.ProcessRequest(video_path=video, segmenter="motion",
105**kw))
106
107 # --- the CUJ: reupload after a segmentation failure must NOT re-validate
108 -----
109
110 def test_reupload_after_segmentation_failure_skips_validation(tmp_path, monkeypatch,
111caplog):
112 """Validation PASSES but detection FAILS ? no segments, so the whole-pipeline cache
113misses on the
114reupload. The validation cache must still short-circuit: validators run exactly ONCE
115across the two
116uploads (the second reuses the cached PASS and goes straight to segmentation)."""
117 db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
118

```

#### 129. assistant (2026-07-08T06:08:47.797Z)

Good harness to mirror. Let me look at the job-worker drain test setup to write a wiring test too.

#### 130. user (2026-07-08T06:08:49.171Z)

```

490 params = json.loads(job["params"])
491 assert params["segment_ids"] == [1, 2, 3]
492 assert params["output_filename"] == "reel.mp4"
493 assert params["locale"] == "kn"
494 db.create_job("pj", "/v.mp4") # a process job
495 assert db.get_job("pj")["kind"] is None
496
497
498 def test_worker_dispatches_compile_kind_to_execute_compile(env):
499 """The claimed-job dispatcher routes kind='compile' to _execute_compile (NOT
500_execute_processing),
501records the result as 'done', and skips the process-only cost/flywheel hooks."""
502 _client, db, _tmp_path, mp = env
503 seen = {}
504
505 def _fake_compile(cfg, db, job, progress=None):
506 seen["id"] = job["id"]
507 return {
508 "status": "success",
509 "video_id": "vidX",
510 "download_url": "/api/highlights/vidX/r.mp4",
511 }
512
513 mp.setattr(m, "_execute_compile", _fake_compile)
514 mp.setattr(
515 m,
516 "_execute_processing",
517 lambda *a, **k: pytest.fail("a compile job must not hit _execute_processing"),
518)

```

```

517)
518 db.create_compile_job("cj2", "vidX", "/v.mp4", [1], "r.mp4")
519 m._run_claimed_job(db.get_job("cj2"), m.global_config, m.db_instance)
520 assert seen["id"] == "cj2"
521 done = db.get_job("cj2")
522 assert done["status"] == "done"
523 assert done["result"]["download_url"] == "/api/highlights/vidX/r.mp4"
524
525
526 def test_create_job_persists_and_reconstructs_compute(tmp_path):
527 """v8: the compute-picker choice must round-trip create_job ? get_job ?
528 _job_to_request so the
529 worker honours it across the async submit?reconstruct boundary. NULL ? server
530 default (the
531 pre-picker default-OFF behaviour). Regression guard for the dropped-across-boundary
532 bug."""
533 db = Database(str(tmp_path / "jobs.db"))
534 db.create_job("j-cloud", "gs://b/clip.mp4", compute="cloud")
535 db.create_job("j-default", "/v.mp4") # no compute ? NULL ? server default
536 assert db.get_job("j-cloud")["compute"] == "cloud"
537 assert db.get_job("j-default")["compute"] is None
538 assert m._job_to_request(db.get_job("j-cloud")).compute == "cloud"
539 assert m._job_to_request(db.get_job("j-default")).compute is None
540
541 # --- crash recovery -----
542
543 def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
544 client, db, tmp_path, mp = env
545 db.create_job("q1", "/v.mp4")
546 db.create_job("r1", "/v.mp4")
547 db.mark_job_running("r1")
548 db.create_job("d1", "/v.mp4")
549 db.mark_job_done("d1", {"status": "success"})

```

### 131. assistant (2026-07-08T06:09:03.369Z)

The compile-dispatch test gives me the exact wiring-test pattern. Let me confirm  
`create\_job`/`set\_job\_video\_id` signatures, then write a self-contained test module.

### 132. user (2026-07-08T06:09:05.423Z)

```

backend/infrastructure/database_jobs.py:23: def create_job(
backend/infrastructure/database_jobs.py:120: def set_job_video_id(self, job_id: str,
video_id: str) -> bool:
backend/infrastructure/database_jobs.py:146: def get_job(self, job_id: str) ->
Optional[Dict[str, Any]]:

```

### 133. user (2026-07-08T06:09:08.141Z)

```

23 def create_job(
24 self,
25 job_id: str,
26 video_path: str,
27 segmenter: Optional[str] = None,
28 player_pool: Optional[List[str]] = None,
29 max_frames: Optional[int] = None,
30 policy: Optional[Dict[str, Any]] = None,
31 owner_id: Optional[str] = None,
32 locale: Optional[str] = None,
33 compute: Optional[str] = None,
34 force: bool = False,

```

```

35) -> bool:
36 """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
37 (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
38 ``owner_id`` (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
39 ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
40 ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
41 honours; NULL ? the server default (byte-identical to pre-picker behaviour).
42 ``force`` opts into deliberate reprocessing; false leaves params NULL for old-row
compatibility."""
43 params = json.dumps({"force": True}) if force else None
44 return self._execute_write(
45 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
46 "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?,
?, ?, ?)",
47 (
48 job_id,
49 video_path,
50 segmenter,
51 json.dumps(player_pool) if player_pool is not None else None,
52 max_frames,
53 json.dumps(policy) if policy is not None else None,
54 owner_id,
55 locale,
56 compute,
57 params,
58),
59 f"Failed to create job {job_id}",
60)
61
62 def create_compile_job(
63 self,
64 job_id: str,
65 video_id: str,
66 video_path: str,
67 segment_ids: List[int],
68 output_filename: str,
69 locale: Optional[str] = None,
70 owner_id: Optional[str] = None,
71) -> bool:
72 """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch

```

#### 134. assistant (2026-07-08T06:09:35.992Z)

Now I'll write a self-contained test module for win #2, mirroring the established  
`\_execute\_processing` harness.

#### 135. assistant (2026-07-08T06:10:11.822Z)

```

"""#524 win #2 ? trust the submit-time ``video_id`` instead of re-hashing on the drain.

```

The chunked upload streams the whole-file MD5 while it uploads (``backend/api/uploads.py``), and the submit hook records that same md5 on the job row (``submit\_time\_video\_id``, #484 ? one gs:// metadata stat, converted to the identical HEX form ``compute\_video\_id`` produces). So when the drain fetches the clip to scratch it already KNOWS the video\_id and need not re-hash the (often multi-GB)

file.

These tests pin the behaviour: with a well-formed trusted id and the knob on (default), the drain SKIPS ``compute\_video\_id`` and uses the trusted id verbatim; with the id absent, malformed, or the knob off, it hashes exactly as before. A spy on ``compute\_video\_id`` (counting invocations) is the proof ? mirrors ``tests/test\_validation\_cache.py``'s stubbed-suite pattern (fully offline, no decode).

"""

```
import backend.main as m
from backend.config.models import AppConfig, DeploymentConfig
from backend.infrastructure.database import Database
from backend.orchestration import _looks_like_video_id
from backend.pipeline.validators.base import ValidationResult

_VALID_ID = "a" * 32 # a well-formed 32-char lowercase-hex video_id
_HASH_SENTINEL = "b" * 32 # what a real compute_video_id() would return (spied)

class _HashSpy:
 """Stands in for ``compute_video_id`` and counts real invocations. A TRUST hit must NOT call
 this
 ? ``calls == 0`` is the proof the drain skipped the re-hash."""

 def __init__(self, ret=_HASH_SENTINEL):
 self._ret = ret
 self.calls = 0

 def __call__(self, *a, **k):
 self.calls += 1
 return self._ret

class _OkSeg:
 """A segmenter that writes one play row (validation is stubbed to pass, so it's reached)."""

 def __init__(self, db_, cfg_):
 self.db = db_

 def process_video(self, vid, path, *a, **k):
 sid = self.db.add_play_segment(vid, 0.0, 5.0)
 return [
 {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata": {}}
]

def _pass_validators(*a, **k):
 return [ValidationResult(validator_name="x", passed=True, message="ok", details={})], {}

def _setup(tmp_path, monkeypatch, *, cfg=None):
 """Wire module globals + a temp-file DB + a hash spy + stubbed validators/segmenter.
 Returns ``(hash_spy, video_path)``. A bare/local ``video_path`` makes
 ``acquire_local_input`` an
 identity (no fetch), so the ONLY video_id source is the trusted id or the spied hash."""
 db = Database(str(tmp_path / "t.db"))
 monkeypatch.setattr(m, "db_instance", db)
 monkeypatch.setattr(m, "global_config", cfg or AppConfig())
 spy = _HashSpy()
 monkeypatch.setattr(m, "compute_video_id", spy)
 monkeypatch.setattr(m.registry, "run_all_with_context", _pass_validators)
 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _OkSeg)
```

```

video = tmp_path / "v.mp4"
video.write_bytes(b"some-bytes")
return spy, str(video)

```

```

def _process(video, *, trusted, cfg=None):
 return m._execute_processing(
 m.ProcessRequest(video_path=video, segmenter="motion"),
 trusted_video_id=trusted,
)

```

## --- \_looks\_like\_video\_id (the format guard) -----

```

def test_looks_like_video_id_accepts_only_32_char_lowercase_hex():
 assert _looks_like_video_id("a" * 32)
 assert _looks_like_video_id("0123456789abcdef0123456789abcdef")
 # rejects: wrong length, uppercase, non-hex, non-str, empty, None
 assert not _looks_like_video_id("a" * 31)
 assert not _looks_like_video_id("a" * 33)
 assert not _looks_like_video_id("A" * 32) # helper is strict-lowercase (callers .lower()
first)
 assert not _looks_like_video_id("g" * 32)
 assert not _looks_like_video_id("not-a-hash")
 assert not _looks_like_video_id("")
 assert not _looks_like_video_id(None)
 assert not _looks_like_video_id(12345678901234567890123456789012) # int, not str

```

## --- the drain: trust vs re-hash -----

```

def test_trusts_valid_submit_id_and_skips_rehash(tmp_path, monkeypatch, caplog):
 """A well-formed trusted id + the default knob ? the drain reuses it and does NOT call the
hash."""
 spy, video = _setup(tmp_path, monkeypatch)
 with caplog.at_level("INFO", logger="backend.orchestration"):
 out = _process(video, trusted=_VALID_ID)
 assert out["video_id"] == _VALID_ID
 assert spy.calls == 0 # the whole point: no re-hash
 assert any("trusting submit-time video_id" in r.message for r in caplog.records)

```

```

def test_hashes_when_no_trusted_id(tmp_path, monkeypatch):
 """No trusted id (local upload / non-gcs URI ? submit_time_video_id was None) ? hash as
before."""
 spy, video = _setup(tmp_path, monkeypatch)
 out = _process(video, trusted=None)
 assert out["video_id"] == _HASH_SENTINEL
 assert spy.calls == 1

```

```

def test_ignores_trusted_id_when_knob_off(tmp_path, monkeypatch):
 """deployment.trust_submit_video_id=False forces the re-hash even with a valid trusted id
(belt-and-suspenders rollback, no redeploy)."""
 cfg = AppConfig(deployment=DeploymentConfig(trust_submit_video_id=False))
 spy, video = _setup(tmp_path, monkeypatch, cfg=cfg)
 out = _process(video, trusted=_VALID_ID, cfg=cfg)
 assert out["video_id"] == _HASH_SENTINEL
 assert spy.calls == 1

```

```

def test_ignores_malformed_trusted_id_and_falls_back_to_hash(tmp_path, monkeypatch):
 """A malformed / wrong-length / wrong-encoding trusted id can never become the DB key ?
 it degrades to a real hash."""
 spy, video = _setup(tmp_path, monkeypatch)
 for bad in ("not-a-valid-id", "a" * 31, "A" * 32, ""):
 spy.calls = 0
 out = _process(video, trusted=bad)
 assert out["video_id"] == _HASH_SENTINEL, bad
 assert spy.calls == 1, bad

def test_uppercase_hex_trusted_id_is_normalized_then_trusted(tmp_path, monkeypatch):
 """The drain lower-cases before the guard, so an upper-case-hex id (a defensive edge) is
 normalized and trusted ? it is still an authoritative md5, just cased differently."""
 spy, video = _setup(tmp_path, monkeypatch)
 out = _process(video, trusted="A" * 32)
 assert out["video_id"] == _VALID_ID # normalized to lowercase
 assert spy.calls == 0

```

### --- the wiring: the drain hands `_execute_processing` the job row's `video_id` -----

```

def test_run_claimed_job_threads_job_video_id_as_trusted(tmp_path, monkeypatch):
 """`_run_claimed_job` must pass the job row's persisted video_id (submit_time_video_id,
#484) to
 `_execute_processing` as `trusted_video_id` ? the wiring that makes win #2 fire on the
 drain."""
 db = Database(str(tmp_path / "jobs.db"))
 monkeypatch.setattr(m, "db_instance", db)
 monkeypatch.setattr(m, "global_config", AppConfig())
 seen = {}

 def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
 seen["trusted"] = trusted_video_id
 return {"status": "success", "video_id": _VALID_ID, "play_segments": []}

 monkeypatch.setattr(m, "_execute_processing", _spy_exec)
 db.create_job("pj", "gs://b/clip.mp4")
 db.set_job_video_id("pj", _VALID_ID)
 m._run_claimed_job(db.get_job("pj"), m.global_config, m.db_instance)
 assert seen["trusted"] == _VALID_ID

def test_run_claimed_job_passes_none_when_job_has_no_video_id(tmp_path, monkeypatch):
 """A job with no known video_id (local upload) threads None ? the drain hashes
 (fallback)."""
 db = Database(str(tmp_path / "jobs.db"))
 monkeypatch.setattr(m, "db_instance", db)
 monkeypatch.setattr(m, "global_config", AppConfig())
 seen = {}

 def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
 seen["trusted"] = trusted_video_id
 return {"status": "success", "video_id": "x", "play_segments": []}

 monkeypatch.setattr(m, "_execute_processing", _spy_exec)
 db.create_job("pj2", "/local/v.mp4") # no video_id set
 m._run_claimed_job(db.get_job("pj2"), m.global_config, m.db_instance)
 assert seen["trusted"] is None

```

136. user (2026-07-08T06:10:11.861Z)

File created successfully at:

C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_video\_id\_trust.py (file state is current in your context ? no need to Read it back)

**137. assistant (2026-07-08T06:10:38.325Z)**

Let me fix a dead param in the test and verify the orchestration logger name for the caplog assertion.

**138. user (2026-07-08T06:10:40.608Z)**

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_video\_id\_trust.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**139. user (2026-07-08T06:10:45.518Z)**

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_video\_id\_trust.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**140. user (2026-07-08T06:10:53.345Z)**

```
45:logger = logging.getLogger(__name__)
```

**141. assistant (2026-07-08T06:10:59.579Z)**

Logger name is `backend.orchestration` ? matches my caplog assertion. Let me run the new test module.

**142. user (2026-07-08T06:11:08.212Z)**

```
....F... [100%]
===== FAILURES =====
_____ test_ignores_malformed_trusted_id_and_falls_back_to_hash _____

tmp_path = WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-avidu/pytest-1612/test_ignores_malformed_trusted0')
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x0000022EFFADA690>

def test_ignores_malformed_trusted_id_and_falls_back_to_hash(tmp_path, monkeypatch):
 """A malformed / wrong-length / wrong-encoding trusted id can never become the DB key ?
 it degrades to a real hash."""
 spy, video = _setup(tmp_path, monkeypatch)
 for bad in ("not-a-valid-id", "a" * 31, "A" * 32, ""):
 spy.calls = 0
 out = _process(video, trusted=bad)
> assert out["video_id"] == _HASH_SENTINEL, bad
E AssertionError: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
E assert 'aaaaaaaaaaaaa...aaaaaaaaaaaaa' == 'bbbbbbbbbbbbbb...bbbbbbbbbbbbbb'
E
E - bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
E + aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

tests\test_video_id_trust.py:133: AssertionError
===== short test summary info =====
FAILED tests/test_video_id_trust.py::test_ignores_malformed_trusted_id_and_falls_back_to_hash
1 failed, 7 passed in 5.05s
```

**143. assistant (2026-07-08T06:11:37.522Z)**

My own test caught a real design inconsistency: I was lowercasing (normalizing) uppercase hex, which contradicts treating a non-canonical id as "not what the drain would produce." The safe, predictable choice is **strict**: trust only an exact 32-char lowercase-hex id (what `submit\_time\_video\_id`/`compute\_video\_id` always produce); anything else falls back to hashing. Let me remove the normalization.

#### 144. user (2026-07-08T06:11:41.834Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 145. user (2026-07-08T06:11:47.746Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_video\_id\_trust.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 146. user (2026-07-08T06:11:58.063Z)

```
..... [100%]
7 passed in 3.95s
```

#### 147. assistant (2026-07-08T06:12:00.611Z)

All 7 win #2 tests pass. Now run the config tests and the touched-area suites to catch regressions early, then the full gate suite.

#### 148. user (2026-07-08T06:12:54.093Z)

E Use -v to get more diff

```
tests\test_config_deployment.py:395: AssertionError
===== warnings summary =====
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED tests/test_config_deployment.py::test_default_config_dump_only_adds_deployment_section
FAILED tests/test_config_deployment.py::test_save_default_config_includes_deployment
2 failed, 110 passed, 8 warnings in 47.04s
```

#### 149. assistant (2026-07-08T06:12:58.046Z)

Expected ? two config tests pin the exact serialized default deployment section, which my new field changes. Let me read and update them.

#### 150. user (2026-07-08T06:12:58.852Z)

```
375 # Helpers + plumbing.
376 # -----
377 def test_is_known_profile():
378 assert is_known_profile("cpu-mock") is True
379 assert is_known_profile("") is False
380 assert is_known_profile("banana") is False
381
382
383 def test_preset_for_returns_copy():
384 a = preset_for("cpu-mock")
```

```

385 a["indexing"]["detector_impl"] = "mutated"
386 b = preset_for("cpu-mock")
387 assert b["indexing"]["detector_impl"] == "stub" # module table not mutated
388 assert preset_for("") == {}
389 assert preset_for("banana") == {}
390
391
392 def test_save_default_config_includes_deployment(tmp_path):
393 p = save_default_config(str(tmp_path / "out.json"))
394 saved = json.loads(p.read_text())
395 assert saved["deployment"] == {
396 "profile": "",
397 "compute_target": "",
398 "max_concurrent_remote_jobs": 4,
399 "remote_poll_max_wait_sec": 3900.0,
400 }
401 cfg = load_config(str(p))
402 assert cfg.deployment.profile == ""
403
404
405 def test_repo_config_json_has_no_profile():
406 """The shipped config.json must not silently opt into a profile (default path stays
today's
407 behaviour for the real deployment)."""
408 repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
409 cfg_path = os.path.join(repo_root, "config.json")
410 if not os.path.exists(cfg_path):
411 pytest.skip("no repo config.json")
412 cfg = load_config(cfg_path)
413 assert cfg.deployment.profile == ""
414
415
416 def test_remote_poll_max_wait_default_and_validation():
417 """CP-rebuild $4.3: the dispatch poll budget must outlast the worker Job's
418 --task-timeout (3600s) by default, and a non-positive budget is a config error."""
419 from pydantic import ValidationError
420
421 from backend.config.models import DeploymentConfig
422
423 assert DeploymentConfig().remote_poll_max_wait_sec == 3900.0
424 with pytest.raises(ValidationError):
425 DeploymentConfig(remote_poll_max_wait_sec=0)
426 with pytest.raises(ValidationError):
427 DeploymentConfig(remote_poll_max_wait_sec=-1)
428

```

#### 151. user (2026-07-08T06:13:06.171Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_config\_deployment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 152. assistant (2026-07-08T06:13:14.983Z)

Now the other failing test ? `test\_default\_config\_dump\_only\_adds\_deployment\_section`:

#### 153. user (2026-07-08T06:13:16.495Z)

```

63: def test_default_config_dump_only_adds_deployment_section():
64- """Adding the section must not perturb any other default ? the dump equals a fresh
65- AppConfig()'s dump, and the new section is present with empty values."""
66- dumped = load_config(NONEXISTENT).model_dump(mode="json")
67- assert dumped == AppConfig().model_dump(mode="json")
68- assert dumped["deployment"] == {
69- "profile": "",

```

```

70- "compute_target": "",
71- "max_concurrent_remote_jobs": 4,
72- "remote_poll_max_wait_sec": 3900.0,
73- }
74-
75-
76-# -----
77-# Parity: the model Literal must match the presets table (no drift).
78-# -----
79-def test_profile_literal_matches_presets():
80- import typing
81-
82- literal_profiles = set(
83- typing.get_args(DeploymentConfig.model_fields["profile"].annotation)
84-)
85- assert literal_profiles == {"", *PROFILES}
86- assert set(PROFILE_PRESETS) == set(PROFILES)
87- assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)
88-
89-
90-def test_compute_target_literal_matches_presets():
91- import typing
92-
93- literal_targets = set(
94- typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation)
95-)
96- assert literal_targets == {"", *COMPUTE_TARGETS}
97- assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)
98-

```

#### 154. user (2026-07-08T06:13:21.829Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_config\_deployment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 155. assistant (2026-07-08T06:13:27.337Z)

Let me check for any other tests pinning the deployment-section shape or full-config schema, so I catch all of them before the full run.

#### 156. user (2026-07-08T06:13:29.220Z)

Found 25 files

```

C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_config_deployment.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_video_id_trust.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\job_worker.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\models.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_startup_checks.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_restart_recovery.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_profile_parity.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_p2b_install.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_drain_concurrency.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_compute_backend.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_config_wasb.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_deploy_manifests.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_app_launcher.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_cloud_dispatch.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test_api_endpoints.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\deploy\cloudrun\gen_service_yaml.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\worker_main_.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\main.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\startup.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\presets.py

```

```
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\loader.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\compute\cloud_run.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\compute__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\compute\base.py
```

### 157. assistant (2026-07-08T06:13:38.121Z)

Most are consumers, not shape-pinner. Let me check the CI coverage floor and how the suite runs, then run the full gate suite (which authoritatively catches any remaining shape assertions).

### 158. user (2026-07-08T06:13:40.977Z)

```
.github/workflows/ci.yml:146: run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:141: run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70
=== run_all_tests head ===
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.
```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```
python run_all_tests.py # full suite, quiet
python run_all_tests.py -v # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
```

Anything after the script name is passed straight through to pytest.

Notes

-----

- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport` package isn't installed (they use `pytest.importorskip`). To exercise them too:  

```
pip install -e ../sports-obsreport --no-build-isolation
```
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).

"""

```
from __future__ import annotations
```

```
import importlib.util
import subprocess
import sys
```

```
def _installed(module: str) -> bool:
 try:
 return importlib.util.find_spec(module) is not None
 except (ImportError, ValueError):
 return False
```

```
def main() -> int:
 args = list(sys.argv[1:])
 # Default to quiet unless the caller chose a verbosity.
 if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
```

### 159. assistant (2026-07-08T06:13:47.570Z)

Coverage floor is **70%**. Let me run the full suite with coverage exactly as CI does. This is the authoritative gate.

160. user (2026-07-08T06:22:15.337Z)

backend\reporting\harvest.py	111	8	93%
backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\server_runtime.py	57	2	96%
backend\sports\__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\storage\__init__.py	85	3	96%
backend\storage\base.py	16	1	94%
backend\storage\capabilities.py	59	10	83%
backend\storage\gcs.py	80	3	96%
backend\storage\gdrive.py	61	3	95%
backend\storage\gdrive_api.py	105	6	94%
backend\storage\local.py	49	3	94%
backend\storage\ref.py	49	0	100%
backend\storage\s3.py	47	5	89%
backend\tools\__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	105	47	55%
backend\utils\__init__.py	0	0	100%
backend\utils\annotation_proxy.py	64	11	83%
backend\utils\app_home.py	40	0	100%
backend\utils\ffmpeg.py	47	3	94%
backend\utils\freespace.py	32	2	94%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	44	3	93%
backend\utils\redact.py	16	0	100%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\single_instance.py	43	11	74%
backend\utils\video.py	34	3	91%
backend\utils\workspace.py	49	0	100%
backend\worker\__init__.py	0	0	100%
backend\worker\__main__.py	319	29	91%
training\__init__.py	0	0	100%
training\gen0\__init__.py	0	0	100%
training\gen0\harness.py	130	9	93%
-----			
TOTAL	12460	1781	86%
Required test coverage of 70% reached. Total coverage: 85.71%			
1854 passed, 8 skipped, 25 warnings in 496.17s (0:08:16)			
=====			
RESULT: PASS - all tests green			
=====			

161. assistant (2026-07-08T06:22:19.356Z)

Full suite green: \*\*1854 passed, 8 skipped, coverage 85.71%\*\* (floor 70%). Now the remaining gates ? ruff, mypy, leak\_scan, check\_md\_links:

162. user (2026-07-08T06:22:35.188Z)

```
=== RUFF ===
All checks passed!
=== LEAK_SCAN ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
=== CHECK_MD_LINKS ===
[check-md-links] clean ? no broken internal links in live docs.
```

163. user (2026-07-08T06:24:34.240Z)

```

66: # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
68: # up. Typed core deps are installed so mypy sees their real signatures
79: - name: Install mypy + typed core deps
=== running mypy as CI does ===
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 149 source files

```

#### 164. assistant (2026-07-08T06:24:43.764Z)

All gates green: full suite 1854 passed / coverage 85.71%, ruff clean, leak\_scan clean, check\_md\_links clean, mypy clean. Let me update tasks and verify the branch/diff before committing.

#### 165. user (2026-07-08T06:24:44.397Z)

Updated task #4 status

#### 166. user (2026-07-08T06:24:45.210Z)

Updated task #6 status

#### 167. user (2026-07-08T06:24:45.765Z)

Updated task #7 status

#### 168. user (2026-07-08T06:24:50.794Z)

```

feat/524-l4-primary
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
 (§9) (#522)
=== branch base vs origin/master (should be empty = no sibling commits) ===
=== changed files ===
 M backend/api/job_worker.py
 M backend/config/models.py
 M backend/orchestration.py
 M docs/COMPUTE_DECOUPLED_SERVING/README.md
 M tests/test_config_deployment.py
?? docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
?? tests/test_video_id_trust.py

```

#### 169. assistant (2026-07-08T06:25:01.039Z)

Branch is clean ? HEAD at `origin/master`, no sibling commits, exactly my 7 intended files. Before committing, let me verify one interaction the map flagged: `\_maybe\_dispatch\_remote`'s video\_id-divergence guard (the worker re-hashes and compares). Trusting the submit-time id must not break it.

#### 170. user (2026-07-08T06:25:21.566Z)

```

661: if result.video_id and result.video_id != video_id:
662: # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
667: result.video_id,
=== context around dispatch video_id handling ===
309: existing = db.get_video(video_id)
313: and db.query_play_segments(video_id, {})
322: report_uri = f"{output_root}/outputs/{video_id}/report.json"

```

```

329: "[API] cache probe failed for %s ? treating as a miss", video_id, exc_info=True
335: db, cfg, video_id: str, video_path: str, requested_compute=None
346: per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via the
350: with _rehydrate_lock(video_id):
351: existing = db.get_video(video_id)
353: segs = db.query_play_segments(video_id, {})
356: video_id,
367: outputs_uri = f"{output_root}/outputs/{video_id}"
381: f"{output_root}/videos/{video_id}/", sc
383: play_wm, cand_wm = db.segment_watermarks(video_id)
384: db.add_video(video_id, video_path, "processed", [])
385: n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
390: for s in db.query_play_segments(video_id, {}):
393: _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
397: "[API] re-hydrated %d segments for video_id=%s from %s (submit-time cache
hit)",
399: video_id,
402: return _cached_process_payload(video_id, [], segments, requested_compute)
406: video_id,
423: def read_path_rehydrate(db, cfg, video_id: str) -> bool:
429: vid = (video_id or "").strip().lower()
441: video_id: str,
462: db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
469: "video_id": video_id,

```

#### 171. user (2026-07-08T06:25:25.529Z)

```

658 return _failed(f"cloud dispatch error: {e}", True)
659 if result.returncode != 0:
660 return _failed(f"cloud job failed (returncode {result.returncode}).", True)
661 if result.video_id and result.video_id != video_id:
662 # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
663 # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
664 logger.warning(
665 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
666 "and the worker hash; ingesting under the local id.",
667 result.video_id,
668 video_id,
669)
670
671 notify("ingesting (cloud_run)")
672 storage_dict = (
673 storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
674 if hasattr(storage_cfg, "model_dump")
675 else {}
676)
677 try:

```

#### 172. assistant (2026-07-08T06:25:41.397Z)

The divergence guard is safe: the trusted submit-time md5 **is** the md5 of the bucket object (what the worker re-hashes), so `result.video_id == video_id` still holds normally; if the object changed between submit and worker-pull, the guard still fires. No regression ? arguably a wider detection window. Let me capture the diff and run an adversarial multi-agent review before committing.

#### 173. user (2026-07-08T06:25:49.131Z)

warning: in the working copy of 'docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test\_video\_id\_trust.py', LF will be replaced by CRLF the next time Git touches it

```
backend/api/job_worker.py | 5 +
backend/config/models.py | 10 +
backend/orchestration.py | 51 ++++
.../L4_PRIMARY_TOPOLOGY.md | 225 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py | 2 +
tests/test_video_id_trust.py | 173 +++++
7 files changed, 466 insertions(+), 2 deletions(-)
```

=== FULL DIFF (code only, excl docs) ===

warning: in the working copy of 'tests/test\_video\_id\_trust.py', LF will be replaced by CRLF the next time Git touches it

319 C:/Users/avidu/AppData/Local/Temp/claude/wt524-diff.txt

#### 174. assistant (2026-07-08T06:26:02.898Z)

Diff is compact (466 insertions across 7 files). The LF?CRLF warnings are just git's normalization notice (harmless). Let me run an adversarial multi-agent review ? correctness, security/trust-boundary, doc-accuracy, and convention/discipline ? verifying findings before I act on them.

#### 175. user (2026-07-08T06:26:58.571Z)

Workflow launched in background. Task ID: wd2d30h22

Summary: Adversarial review of the #524 branch diff (win #2 + design doc) across 4 dimensions, each finding verified before it survives

Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-

indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf\_dd531439-5e0

Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\workflows\scripts\review-524-diff-wf\_dd531439-5e0.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\workflows\scripts\review-524-diff-wf\_dd531439-5e0.js"} to iterate without resending the script.)

Run ID: wf\_dd531439-5e0

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\workflows\scripts\review-524-diff-wf\_dd531439-5e0.js", resumeFromRunId: "wf\_dd531439-5e0"}) ? completed agents return cached results (cached results may themselves be empty ? inspect journal.jsonl before assuming there is something to recover).

You will be notified when it completes. Use /workflows to watch live progress.

#### 176. assistant (2026-07-08T06:27:13.881Z)

While the review runs, I'll respect the concurrent-session guardrail and \*\*not\*\* touch the main checkout's `SESSION\_HANDOFF.md` (it's on a different branch with the #523 session's WIP). Let me draft the PR body in scratch while waiting.

#### 177. assistant (2026-07-08T06:27:38.935Z)

### What & why

Epic \*\*#524\*\* (collapse the pipeline onto the L4 worker; shrink the control-plane to a thin router) is \*\*investigation-first\*\*. This PR lands the two lowest-risk, reversible pieces and defers the risky rearchitecture behind a reviewed design:

1. \*\*Design doc\*\* ? `docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md` resolves the compute-topology question (\*\*A\*\* Cloud Run Service-with-GPU / \*\*B\*\* GCE L4 VM / \*\*C\*\* hybrid presigned-upload) with a cost model, and \*\*recommends against the epic's proposed "upload

directly onto the L4\*\*\*:

- A same-region GCS?Cloud Run transfer is \*\*LAN-fast and not billed as internet egress\*\*  
`[researched]`, so the round-trip that upload-onto-L4 saves is worth ~seconds and ~\$0, while keeping an L4 up to \*receive\* a \*\*minutes-long\*\* upload burns \*\*~175 s of idle GPU/run (~\$0.08?0.16)\*\* and \*opens\* a durability window instead of closing one.
- The cheaper, more durable, already-ratified path is the opposite: \*\*keep upload ? bucket\*\* (the #480 design; the durable copy exists before the L4 starts) and \*\*collapse validate+detect+stitch onto the scale-to-zero Cloud Run L4 Job\*\* via #521's `phase\_placement`. The residual ~30 s gap dies not by moving the upload but by the control-plane \*\*never downloading the file\*\* (delegate validation per #521 + trust the md5 per win #2). Net movement: \*\*3x ? the floor (1 client upload + 1 LAN pull)\*\*; zero idle-GPU cost, zero exposure window.
- Persistent-GPU (VM / Service-with-min-instances) is \*\*rejected at current volume\*\* (breakeven ? 51 % on-demand / 31 % spot duty cycle ? 2.6?4.3k runs/mo) and left as an owner-gated future ADR. Honors ADR-014 \*\*D4/D7/D16\*\*.

2. \*\*Win #2 ? drop the redundant control-plane re-hash.\*\* The drain re-hashed the whole (often multi-GB) clip it just fetched (`orchestration.py` `compute\_video\_id`), even though the submit hook already recorded the file's md5 on the job row (`submit\_time\_video\_id`, #484 ? a GCS-metadata md5 converted to the identical hex form; \*\*already trusted for the submit-time cache probe\*\*). Now the drain reuses that \*\*server-derived, byte-identical\*\* id and skips the re-hash; absent / malformed ? it hashes exactly as before. Gated by `deployment.trust\_submit\_video\_id` (default on) and guarded by `\_looks\_like\_video\_id` (strict 32-char lowercase hex).

\*\*Win #1 (validation-on-L4) is intentionally NOT here\*\* ? it is delivered by the separate open branch `feat/521-compile-on-l4` (`phase\_placement.validation=cloud\_run\_l4` + its guards). Per the coordination note in #524, this PR does \*\*not\*\* duplicate that knob; the design doc credits #521 and specifies the download-avoidance follow-up (which composes #521's delegation with this PR's win #2) plus the verdict-serialization gap #521 leaves open.

## Coordination with #521

- Branched from the \*\*latest `origin/master`\*\*; #521 is still open. Win #2 touches the top-of-`\_execute\_processing` hash block, which \*\*#521 does not modify\*\* ? independent, non-conflicting.
- The new `deployment.trust\_submit\_video\_id` field is a \*\*non-overlapping addition\*\* to `DeploymentConfig` (#521 adds `phase\_placement`/`cloud\_stitch\_encoder` elsewhere in the class).

## Reversibility

Every change is off-by-config: `deployment.trust\_submit\_video\_id=false` reverts win #2 to a byte-identical re-hash with no redeploy. When no trusted id is present (local/appliance upload, non-gcs direct-URI submit) the drain hashes as before. The resulting `video\_id` is byte-identical to today's re-hash (same bytes, same MD5), so no cache/DB key changes.

## Verification (all local, gates match CI)

- `python run\_all\_tests.py --cov=backend --cov=training --cov-fail-under=70` ? \*\*1854 passed, 8 skipped, coverage 85.71%\*\* (floor 70%).
- `ruff check .` clean · `mypy backend training` clean (149 files) · `python -m tools.leak\_scan` clean · `python -m tools.check\_md\_links` clean.
- New `tests/test\_video\_id\_trust.py` proves: trusted id ? `compute\_video\_id` \*\*not\*\* called; absent/malformed/knob-off ? re-hash; and the `\_run\_claimed\_job` wiring threads `jobs.video\_id` through.
- Not verifiable locally (owner-side, D17 budget): the live CUJ confirming the ~30 s gap is gone + movement is 2x on rev ? `00020`. Design + code + tests only ? \*\*no deploy\*\* in this PR.

Refs #524, #521, #520.

? Generated with [Claude Code](https://claude.com/claude-code)

178. user (2026-07-08T06:27:39.019Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\wt524-pr-body.md (file

state is current in your context ? no need to Read it back)

**179. assistant (2026-07-08T06:27:41.619Z)**

The PR body is drafted. Let me wait for the adversarial review to complete before committing so I can fold in any confirmed fixes.

**180. user (2026-07-08T06:27:45.236Z)**

review running in background (wd2d30h22); waiting for completion notification

**181. assistant (2026-07-08T06:27:59.416Z)**

The full suite already passed (1854 tests), which authoritatively confirms no other config-shape assertions were missed. I'll wait for the adversarial review to complete before committing, so any confirmed fixes fold into the single commit.

**182. user (2026-07-08T06:28:02.055Z)**

waiting for review workflow wd2d30h22

**183. assistant (2026-07-08T06:28:21.636Z)**

I've completed the substantive work and am waiting on the adversarial review before committing. Here's where things stand:

## Progress so far

**\*\*Investigation complete\*\*** ? 8-agent subsystem map + live GCP pricing pulled from the Cloud Billing Catalog API, all verified against the actual code and the 2026-07-08 runlog.

**\*\*Key scoping discovery:\*\*** #521's open branch `feat/521-compile-on-l4` **\*\*already implements win #1\*\*** (validation-on-L4 via `phase\_placement.validation` + guards). Per the coordination note, I did **\*\*not\*\*** duplicate that knob ? my PR credits #521 and delivers the complementary pieces.

**\*\*Deliverable 1 ? design doc\*\*** (`docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md`): resolves the A/B/C topology question and **\*\*recommends against the epic's "upload onto the L4" with a cost model.** The central quantified finding: uploading onto the GPU box trades a **\*\*~free, seconds-fast intra-region LAN pull\*\*** for **\*\*~175 s of idle GPU per run\*\***, and **\*\*opens\*\*** a durability window. The cheaper/more-durable path (co-located scale-to-zero Jobs + bucket upload) kills the ~30 s gap by the control-plane **\*\*never downloading the file\*\*** ? composing #521's validation-delegation with win #2. Includes the minimal control-plane surface, the async-copy durability spec, and a phased migration on #521's `phase\_placement`.

**\*\*Deliverable 2 ? win #2\*\*** (drop the redundant re-hash): the drain now trusts the **\*\*server-derived\*\*** submit-time md5 (`jobs.video\_id` ? `submit\_time\_video\_id`, #484) instead of re-hashing the fetched clip. Gated by `deployment.trust\_submit\_video\_id` (default on), guarded by a strict hex format check, byte-identical id, fully reversible.

**\*\*All gates green (CI-equivalent, local):\*\***

- Full suite: **\*\*1854 passed, 8 skipped, coverage 85.71%\*\*** (floor 70%)
- ruff clean · mypy clean (149 files) · leak\_scan clean · check\_md\_links clean

**\*\*Branching safety:\*\*** isolated worktree, branched from latest `origin/master`, exactly 7 intended files, no sibling commits, no stray files staged.

An adversarial 4-dimension review (correctness · security/trust-boundary · doc-accuracy · conventions), with each finding independently verified, is running now. I'll fold any **\*\*confirmed\*\*** findings into the single commit, then open the PR against `master` referencing #524/#521/#520. Waiting on the review to complete.

**184. user (2026-07-08T06:41:05.374Z)**

```

<task-notification>
<task-id>wd2d30h22</task-id>
<tool-use-id>toolu_01J17DFTw7dD5R85EVBSctxYi</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\tasks\wd2d30h22.output</output-
file>
<status>completed</status>
<summary>Dynamic workflow "Adversarial review of the #524 branch diff (win #2 + design doc)
across 4 dimensions, each finding verified before it survives" completed</summary>
<result>{"reviews":[{"dimension":"CORRECTNESS of win #2 (trust path + wiring:
_execute_processing trusted-vs-rehash branch, _looks_like_video_id, job_worker trusted_video_id
source, divergence-guard & cache interaction, caller
regressions)","clean":true,"findings":[]},{dimension:"SECURITY / TRUST BOUNDARY","clean":false
,"findings":[{"severity":"low","file":"backend/orchestration.py","line":791,"summary":"Win #2
drops the only end-to-end check that the bytes the drain processed hash to the identity they're
stored under: when trusting, there is no post-fetch recheck and the fetch is not generation-
pinned, so the DB/output key rests on submit-time metadata + an unstated object-immutability
assumption rather than on the code itself.","failure":"submit_time_video_id() stats the gs://
object at submit (orchestration.py:231) and persists md5Hash=X on jobs.video_id. The drain later
fetches the LIVE object via acquire_local_input -& GCSStorage.fetch_to_local -&
download_to_filename(dst) (gcs.py:65) with NO if_generation_match; download verifies the fetched
bytes against the CURRENT object's md5Hash, not against X. If the object at that gs:// path is
overwritten with different-hash content Y between the submit-time stat and the drain fetch, the
trusted branch (orchestration.py:791-796) takes video_id=X with NO recheck, and this run's
detection outputs + DB rows for content Y get written under key X -- silently poisoning the
GLOBAL, cross-tenant content cache for future reads of X. Pre-#524 the drain re-hashed the
fetched bytes (compute_video_id), so the key always matched the processed bytes and any mid-
flight overwrite self-corrected. NOTE reachability: not client-triggerable in the current hosted
config -- uploads are server-mediated (clients hold no direct GCS write creds), upload paths are
unique uuid4 dirs (uploads/&uid&uid&uid;), and direct-URI submits are restricted to input_root
-- so this is defense-in-depth today, but it becomes client-reachable the moment any future
presigned direct-to-bucket upload (the doc's rejected hybrid C direction) or in-place bucket
rewrite/lifecycle op lands. (MD5-collision keying is a separate, pre-existing property of
compute_video_id, unchanged by this PR).","fix":"Make the trust self-verifying instead of
assumption-based: carry the object generation from the submit-time stat onto the job row and
fetch with if_generation_match=&gen&gen; (fail the job on generation mismatch); or, cheaper,
re-stat the object at fetch time and compare its md5 to the trusted id before taking the trusted
branch (one extra metadata GET, no multi-GB re-hash). Either closes the TOCTOU window while
preserving the perf win."},{severity":"nit","file":"docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_T
OPOLOGY.md","line":41,"summary":"The L6 'byte-identical video_id' claim silently omits its
precondition (the bucket object is immutable in-flight), which is the load-bearing assumption
for the whole win #2 trust argument.","failure":"Line 41 states win #2 'produces a byte-
identical video_id (the GCS md5Hash over the same bytes = compute_video_id)'. This equality only
holds while the gs:// object is not overwritten between the submit-time stat and the drain
fetch. The doc presents it as unconditional, so a future reader wiring a presigned direct-to-
bucket upload (contemplated in the hybrid-C discussion) could reasonably assume win #2 stays
safe when it would no longer be.","fix":"Add one clause to L6 / the §Boundaries notes: the byte-
identity holds because the input object is immutable in-flight in the current topology (server-
mediated uploads, no client bucket-write creds, unique uuid upload paths); a future client-
writable/presigned upload path MUST add generation-pinning (or a fetch-side md5 recheck) before
enabling trust_submit_video_id, since the drain no longer re-hashes."}]},{dimension:"DESIGN-
DOC ACCURACY (docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md)","clean":false,"findings":[{"severity":"medium","file":"docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md","line":144
,"summary":"§4.4 cites a [verified] code anchor `backend/main.py::get_highlights` that does not
exist.","failure":"`grep -rn get_highlights backend/` returns nothing (exit 1). The
/api/highlights 307 signed-URL handler is actually `download_highlight` at backend/main.py:1042
(route `@app.get(\"/api/highlights/{video_id}/{filename}\")`, `RedirectResponse(signed,
status_code=307)` at ~:1059). An owner verifying the [verified] tag finds no such
function.","fix":"Change the anchor to `backend/main.py::download_highlight` (backend/main.py:10
42)."}]},{severity":"low","file":"docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md","line":
68,"summary":"The ~30s 'Accept?validate' gap is attributed (as [verified]) to a CP scratch re-
download + re-hash, but the cited runlog labels that same phase differently.","failure":"§4.1
(line 68) / §1 finding (b) / §0 say the ~30s = 'CP re-download to scratch (acquire_local_input +
re-hash)', tagged [verified] against runlog §9. But runlog line 118 labels the ~30s 'Accept ?
validation start' phase 'stage upload to serving bucket'. The two sources give different causes

```

for the same measured number, and the doc calls it 'unlogged' while the runlog does label it. (Code actually backs the doc: uploads.py streams straight into `<input_root>/uploads/?`` in the serving bucket with no bucket-to-bucket stage-copy ? so the runlog label is the imprecise one ? but the [verified] tag still conflicts with its own cited evidence.)", "fix": "Reconcile the two: either correct the runlog phase label, or retag §4.1's causal claim [researched]/[design] and note the runlog labels it as a stage-copy. Optionally clarify that win #2 removes only the (fast) re-hash and it is P2b ? not this PR ? that removes the download bulk of the 30s."}, {"severity": "low", "file": "docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md", "line": "13", "summary": "§0's '~175 s of idle GPU per run (~\$0.08?0.16)' upper bound is not derivable from the §4.2 cost model.", "failure": "175 s at the doc's own L4 Job rate (\$0.000474/s) = \$0.083; at the warm-GPU-Service rate (\$0.000349/s) = \$0.061; at GPU-only (\$0.00022404/s) = \$0.039. The full upload-onto-L4 topology delta in §4.2 is only +\$0.13. No stated rate × 175 s reaches \$0.16, so the upper bound is unsupported by the model.", "fix": "Narrow to ~\$0.08?0.13, or show the derivation (e.g. state the rate and duration that produce \$0.16)."}, {"severity": "nit", "file": "docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md", "line": "23", "summary": "orchestration.py line anchors are stale in the committed tree because this same PR shifts those lines.", "failure": "§4.1 (line 23) cites `backend/orchestration.py:748` (acquire\_local\_input) and `:751` (re-hash compute\_video\_id); §10 (line 218) cites `:741-763` for '(re-hash)'. Those were the pre-diff (HEAD) positions, but this PR's own orchestration.py edit moves them: in the committed worktree acquire\_local\_input is at :771 and compute\_video\_id is at :798, and :741-763 now spans the docstring/config-resolution, not the re-hash. Anchors land on the wrong lines on the merged branch.", "fix": "Update to :771 (acquire\_local\_input) / :798 (re-hash), or drop the line numbers and keep the function-name anchors only."}, {"severity": "nit", "file": "docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md", "line": "191", "summary": "Win #1 (P2) status is marked Done in §7 but Open/unchecked in §5 and §9 ? internal inconsistency.", "failure": "§7 P2 row (line 191) shows status '? delivered by #521', while §5 P2b (line 167) marks it '#521 (open)' and §9 DoD (line 212) leaves '[ ] Win #1 confirmed delivered by #521' unchecked. #521 is an unmerged open branch, so the same deliverable reads as Done in one table and Pending in two others.", "fix": "Change §7 P2 status to ??? 'pending #521 merge' (or reword the ? to clearly mean 'scoped to #521, not this PR') so it matches §5 and §9."}], {"dimension": "REPO CONVENTIONS / DISCIPLINE", ... (truncated 14440 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\tasks\wd2d30h22.output)</result>  
<diagnostics>Per-agent results: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf\_dd531439-5e0\journal.jsonl ? one {"type": "result", ...} line per completed agent with its full return value.  
If the result above is empty or unexpected, Read this file BEFORE diagnosing ? do not assume agents returned non-empty results.  
To re-run with edited post-processing: Workflow({scriptPath: 'C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\workflows\scripts\review-524-diff-wf\_dd531439-5e0.js', resumeFromRunId: 'wf\_dd531439-5e0'}) ? agents whose (prompt, opts) are unchanged replay from cache.</diagnostics>  
<usage><agent\_count>14</agent\_count><agents\_done>14</agents\_done><agents\_error>0</agents\_error><agents\_skipped>0</agents\_skipped><agents\_empty\_result>0</agents\_empty\_result><subagent\_tokens>950461</subagent\_tokens><tool\_uses>179</tool\_uses><duration\_ms>846749</duration\_ms></usage>  
</task-notification>

### 185. assistant (2026-07-08T06:41:17.343Z)

Excellent, high-signal findings. Let me read the full output including the conventions dimension and the verify verdicts.

### 186. user (2026-07-08T06:41:23.894Z)

VERIFY: CONFIRMED | must\_fix= True | docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md:144 cites a `[verified]`-tagged code anchor `backend/main.py::get\_highlights` for the "Signed-URL download (`/api/highlights` 307)" row. `grep -rn "get\_highlights" backend/` returns noth  
fix: In docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md:144 change the anchor `backend/main.py::get\_highlights` to `backend/main.py::download\_highlight` (backend/main.py:1042).

VERIFY: REFUTED | must\_fix= False | Literal claim is true but it is not a defect. DOC\_STATUS.md §3 (register, lines 28-85, verified by full read) has no row/sub-note for

L4\_PRIMARY\_TOPOLOGY.md, and the PR does not touch DOC\_STATUS.md (git diff --stat lists 7 files;  
DOC\_STATUS

fix:

VERIFY: CONFIRMED | must\_fix= False | Real internal inconsistency in  
docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md. The \$7 tracker ? self-labeled "? source  
of truth" (line 183) ? marks Win #1 / validation-on-L4 as Done: line 191 Status = "? \*\*delivered  
by #521\*\* ? not d

fix: In \$7, line 191, change the P2 (Win #1) Status cell from "? \*\*delivered by #521\*\* ? not  
duplicated here" to a not-done marker consistent with \$5/\$9, e.g. "? pending #521 merge (scoped  
to #521, not thi

VERIFY: CONFIRMED | must\_fix= False | Verified against the worktree at  
C:/Users/avidu/AppData/Local/Temp/claude/wt524-969.

HEAD (pre-diff), via `git show HEAD:backend/orchestration.py`: acquire\_local\_input in  
\_execute\_processing = line 748; compute\_video\_id re-hash = line 751

fix: Update the anchors in docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md to the  
committed-tree positions: line 23 ? `backend/orchestration.py:771` (acquire\_local\_input) and  
`:798` (re-hash); line 2

VERIFY: CONFIRMED | must\_fix= False | The doc's [verified]-tagged absolute line anchors point at  
unrelated code in the SAME commit, because this PR inserts ~27 lines of win-#2 logic ahead of  
the re-hash and the doc's anchors were written against the pre-insertion positions.

Pr

fix: In L4\_PRIMARY\_TOPOLOGY.md \$0 (line 23): change acquire\_local\_input anchor from `:748`  
to `:771` and the re-hash anchor from `:751` to `:798`. In \$10 (line 218): change the re-hash  
range `:741-763` to

VERIFY: CONFIRMED | must\_fix= False | docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md:13  
(\$0) states keeping an L4 up burns "~175 s of idle GPU per run (~\$0.08?0.16)". No rate in the  
\$4.2 cost model (lines 83?92) yields the \$0.16 upper bound at 175 s. Computed against eve

fix: Narrow the \$0 range to what the \$4.2 model supports ? the marginal idle tax vs the  
recommended #521 path is \$0.083, and the delta vs today is +\$0.13, so "~\$0.08?0.13" is  
defensible. Alternatively, sta

VERIFY: REFUTED | must\_fix= False | The L6 claim at  
docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md:41 ("byte-identical video\_id: GCS md5Hash  
over the same bytes = compute\_video\_id") is TRUE for the shipped topology, and no present code  
path makes it diverge ? so there

fix:

VERIFY: REFUTED | must\_fix= False | The described code mechanic is real but the TOCTOU  
precondition is unreachable in the actual current code, so this is defense-in-depth hardening,  
not a must-fix defect.

VERIFIED MECHANIC (accurate):

- orchestration.py:791-796 ? trusted bra

fix: No fix required for this PR. If/when a future presigned direct-to-bucket upload path or  
in-place bucket rewrite/lifecycle op is added (the doc's rejected hybrid-C direction), close the  
window then by

VERIFY: REFUTED | must\_fix= False | The finding flags that L4\_PRIMARY\_TOPOLOGY.md \$4.1 (line 68)  
attributes the ~30s "Accept?validate" gap to a CP re-download+re-hash under a [verified]/"From  
runlog \$9" header, while runlog \$9 line 118 labels the same phase "stage upload to s

fix: No required fix. The doc's causal claim is code-accurate; the runlog's "stage upload to  
serving bucket" label is the imprecise one. Optional-only clarity nicety: add a one-line  
footnote at \$4.1 noting

VERIFY: REFUTED | must\_fix= False | The finding's MECHANICAL claim is factually TRUE and now  
guaranteed (not hypothetical): origin/master has already advanced from the worktree HEAD bf97700  
to dac6736 = "#521 ... config-driven phase?machine placement (#521) (#525)", so #521 i

fix: No code change to #524. At merge/rebase onto current origin/master (which already  
contains #521 as #525), resolve the DeploymentConfig conflict by keeping BOTH field additions

(phase\_placement/cloud\_s

#### 187. assistant (2026-07-08T06:41:41.485Z)

**\*\*Major finding from the review: #521 has now MERGED to `origin/master`\*\*** (as PR #525). My branch is based on the pre-merge `bf97700`. Per CLAUDE.md ("if #521's PR has merged, build on its `phase\_placement`"), I should rebase onto current master. Let me confirm, and read the conventions dimension + full findings list.

#### 188. user (2026-07-08T06:41:48.395Z)

```
=== origin/master now at ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
=== is #521/#525 on master? ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
1358521 review: address #42 nits ? dynamic section scan, shim comment, scalar-default test
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config
=== does PHASE_PLACEMENT.md exist on master now? ===
YES exists on master
```

#### 189. user (2026-07-08T06:41:50.688Z)

CONVENTIONS clean= False

- low docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md 23 :: Doc's own [verified]-tagged code anchors point at the wrong lines because THIS PR shifts them ? a same-commit honesty defect that violates the doc's stated tagging convention.  
fix: Update \$0 to acquire\_local\_input at :771 and the re-hash at :798, and \$10 to :798 for the re-hash (or drop brittle absolute line numbers in favor of symbol names, which is what the doc already does fo
- nit backend/config/models.py 493 :: trust\_submit\_video\_id is appended at the tail of DeploymentConfig, the same insertion point #521 uses to extend DeploymentConfig ? a likely (trivial) textual merge conflict.  
fix: No code change needed ? flag for whoever merges second: keep both field additions (distinct names, both pure additions). Optionally rebase-order so one lands first and the other re-appends cleanly.
- nit docs/DOC\_STATUS.md 34 :: New DRAFT design doc L4\_PRIMARY\_TOPOLOGY.md is not registered in the DOC\_STATUS \$3 lifecycle register.  
fix: Add a one-line sub-note (or row) under the COMPUTE\_DECOUPLED\_SERVING/ register entry noting L4\_PRIMARY\_TOPOLOGY.md is DRAFT/owner-review, archive-on-DONE. Optional; the doc's own status header + the u

#### 190. assistant (2026-07-08T06:43:03.397Z)

Confirmed: **\*\*#521 merged to master as `dac6736` (#525)\*\***; `PHASE\_PLACEMENT.md` now exists there. This changes my base. Per CLAUDE.md ("if #521's PR has merged, build on its `phase\_placement`"), I'll rebase onto current `origin/master`.

Confirmed review findings to fix (I'll apply them against the **\*\*rebased\*\*** tree so line numbers are final):

- **\*\*must-fix:\*\*** \$4.4 `get\_highlights` ? `download\_highlight` (real, nonexistent symbol)
- reconcile Win#1 status across \$5/\$7/\$9 (now genuinely *\*done\** since #521 merged)
- drop brittle absolute line-anchors in favor of symbol names
- narrow \$0 cost range `\$0.08?0.16` ? `\$0.08?0.13`
- update "#521 open" ? "#521 merged (#525)"; switch PHASE\_PLACEMENT links to relative
- add an honest immutability/TOCTOU caveat to \$6 (defense-in-depth, correctly *\*not\** coding it now)

Let me set up the rebase. First, commit the WIP with explicit paths, then rebase.

**191. user (2026-07-08T06:43:06.406Z)**

Updated task #7 subject, description

**192. user (2026-07-08T06:43:09.638Z)**

```
feat/524-l4-primary
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the
next time Git touches it
M backend/api/job_worker.py
M backend/config/models.py
M backend/orchestration.py
A docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
M docs/COMPUTE_DECOUPLED_SERVING/README.md
M tests/test_config_deployment.py
A tests/test_video_id_trust.py
```

**193. assistant (2026-07-08T06:43:11.776Z)**

Exactly 7 files staged. Let me commit, then rebase onto the merged master.

**194. user (2026-07-08T06:43:18.892Z)**

0d49c6f @ feat(serving): L4-primary topology design + trust submit-time video\_id (#524)

**195. user (2026-07-08T06:43:22.977Z)**

```
Auto-merging backend/config/models.py
CONFLICT (content): Merge conflict in backend/config/models.py
Auto-merging backend/orchestration.py
Auto-merging docs/COMPUTE_DECOUPLED_SERVING/README.md
Auto-merging tests/test_config_deployment.py
CONFLICT (content): Merge conflict in tests/test_config_deployment.py
Rebasing (1/1)
error: could not apply 0d49c6f... @
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config advice.mergeConflict false"
Could not apply 0d49c6f... @ feat(serving): L4-primary topology design + trust submit-time
video_id (#524)
```

**196. assistant (2026-07-08T06:43:34.334Z)**

Conflicts are exactly as expected ? both #521 and my change extend `DeploymentConfig` and the config-shape tests. Resolution: keep all fields. Let me view the exact conflict regions.

**197. user (2026-07-08T06:43:37.512Z)**

```
=== models.py conflicts ===
507:<<<<<< HEAD
537:=====
548:>>>>>> 0d49c6f (@)
=== test conflicts ===
73:<<<<<< HEAD
77:=====
79:>>>>>> 0d49c6f (@)
407:<<<<<< HEAD
410:=====
412:>>>>>> 0d49c6f (@)
```

## 198. user (2026-07-08T06:43:40.664Z)

```
483 ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
484 *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
485 these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
486 (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
487
488 Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
489
490 # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
491 profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
492 # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
493 # (recorded for observability; consumed by the ComputeBackend seam from M2+).
494 compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
495 # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
496 # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
497 # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
498 # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
499 # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
500 max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
501 # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
502 # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
503 # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
504 # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
505 # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
506 remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
507 <<<<<<< HEAD
508 # Per-phase machine placement (#521). Unset ? today's placement (validation +
compile on the
509 # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
510 # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read
the sub-fields
511 # directly ? the resolver folds in the legacy compute_target default).
512 phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
513 # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for
the L4 stitch
514 # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
515 # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is placed
on the L4;
516 # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
517 cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
518
519 @model_validator(mode="after")
520 def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
521 # A clip can only be validated ON the worker if the worker also DETECTS it there
? the worker
522 # is only launched for the segment(detect) phase. So validation=cloud_run_l4
with segment on
```

```

523 # the control-plane would leave NOBODY validating (the control-plane skips, and
no worker runs)
524 # ? reject that footgun at config load rather than ship a silently-unvalidated
clip.
525 if self.phase_placement.validation == "cloud_run_l4":
526 seg = self.phase_placement.segment or (
527 "cloud_run_l4" if self.compute_target == "cloud_run" else
"control_plane"
528)
529 if seg != "cloud_run_l4":
530 raise ValueError(
531 "deployment.phase_placement.validation='cloud_run_l4' requires
segment to also "
532 "run on the L4 ? the worker can only validate a clip it also
detects. Set "
533 "phase_placement.segment='cloud_run_l4' (or
compute_target='cloud_run'), or keep "
534 "validation on the control_plane."
535)
536 return self
537 =====
538 # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain
(#524 win #2).
539 # The chunked upload already streams the whole-file md5
(``backend/api/uploads.py``), and the
540 # submit hook records it on the job row (``submit_time_video_id``, one gs://
metadata stat ? #484);
541 # that value is BYTE-IDENTICAL to ``compute_video_id`` (same bytes, same MD5) and is
ALREADY
542 # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True
(default) the drain
543 # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when
the id is absent
544 # (a local/appliance upload or a non-gcs direct-URI submit ?
``submit_time_video_id`` returns None)
545 # the drain still hashes, exactly as before. Set False to force a re-hash
unconditionally
546 # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
547 trust_submit_video_id: bool = True
548 >>>>>> 0d49c6f (@)
549
550
551 class BillingConfig(BaseModel):
552 """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).

```

#### 199. assistant (2026-07-08T06:43:46.019Z)

Resolving the `models.py` conflict ? keep #521's  
`phase\_placement`/`cloud\_stitch\_encoder`/validator **\*\*and\*\*** my `trust\_submit\_video\_id`, with my  
field declared before the validator:

#### 200. user (2026-07-08T06:44:07.772Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\models.py has been  
updated successfully. (file state is current in your context ? no need to Read it back)

#### 201. assistant (2026-07-08T06:44:10.191Z)

Now resolve the two test-shape conflicts ? keep all fields:

#### 202. user (2026-07-08T06:44:11.387Z)

```

66 dumped = load_config(NONEXISTENT).model_dump(mode="json")
67 assert dumped == AppConfig().model_dump(mode="json")

```

```

68 assert dumped["deployment"] == {
69 "profile": "",
70 "compute_target": "",
71 "max_concurrent_remote_jobs": 4,
72 "remote_poll_max_wait_sec": 3900.0,
73 <<<<<<< HEAD
74 # #521: per-phase placement (unset ? legacy default) + the L4 stitch encoder.
75 "phase_placement": {"validation": None, "segment": None, "compile": None},
76 "cloud_stitch_encoder": "h264_nvenc",
77 =====
78 "trust_submit_video_id": True,
79 >>>>>>> 0d49c6f (@)
80 }
81
82
83 # -----

```

### 203. user (2026-07-08T06:44:17.571Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_config\_deployment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 204. user (2026-07-08T06:44:20.818Z)

```

398 saved = json.loads(p.read_text())
399 assert saved["deployment"] == {
400 "profile": "",
401 "compute_target": "",
402 "max_concurrent_remote_jobs": 4,
403 "remote_poll_max_wait_sec": 3900.0,
404 <<<<<<< HEAD
405 "phase_placement": {"validation": None, "segment": None, "compile": None},
406 "cloud_stitch_encoder": "h264_nvenc",
407 =====
408 "trust_submit_video_id": True,
409 >>>>>>> 0d49c6f (@)
410 }
411 cfg = load_config(str(p))
412 assert cfg.deployment.profile == ""
413
414
415 def test_repo_config_json_has_no_profile():

```

### 205. user (2026-07-08T06:44:29.016Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\tests\test\_config\_deployment.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 206. assistant (2026-07-08T06:44:34.896Z)

Both test conflicts resolved. README and orchestration auto-merged. Let me verify no conflict markers remain anywhere and that my orchestration changes survived alongside #521's:

### 207. user (2026-07-08T06:44:39.759Z)

```

=== any remaining conflict markers? ===
Binary file backend/__pycache__/main.cpython-313.pyc matches
docs/EVALUATION.md:74:=====
docs/EVALUATION.md:77:=====
docs/EVALUATION.md:82:=====
(none above = clean)
=== my orchestration changes present? ===
269: def _looks_like_video_id(value: Optional[str]) -> bool:
766: config=None, db=None, req=None, progress=None, trusted_video_id=None

```

```

777: ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for
this
822: # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
833: candidate = trusted_video_id or ""
834: if trust_submitted and _looks_like_video_id(candidate):
837: "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
=== #521 changes present in orchestration? ===
8

```

#### 208. assistant (2026-07-08T06:44:49.713Z)

Clean merge ? both changesets coexist. Let me verify my `job\_worker.py` change survived and stage the resolved files to continue the rebase.

#### 209. user (2026-07-08T06:44:51.954Z)

```

180: trusted_video_id=job.get("video_id") or None,
=== stage resolved + continue rebase ===
[detached HEAD 6d528f1] @ feat(serving): L4-primary topology design + trust submit-time video_id
(#524)
7 files changed, 466 insertions(+), 2 deletions(-)
create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
create mode 100644 tests/test_video_id_trust.py

```

Successfully rebased and updated refs/heads/feat/524-l4-primary.

#### 210. assistant (2026-07-08T06:44:54.593Z)

Rebase complete. Now verify the branch state and confirm mpyy/models parse correctly before the doc fixes:

#### 211. user (2026-07-08T06:44:58.946Z)

```

6d528f1 @ feat(serving): L4-primary topology design + trust submit-time video_id (#524)
=== only my commit above (base now dac6736) ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
=== quick import sanity ===
phase_placement= None | cloud_stitch_encoder= h264_nvenc | trust_submit_video_id= True

```

#### 212. assistant (2026-07-08T06:45:10.345Z)

Clean rebase ? exactly one commit (6d528f1) on top of `dac6736` (which includes #521/#525), config parses with all three fields. Now the doc fixes against the final tree. Let me get accurate current anchors and confirm the `download\_highlight` symbol.

#### 213. user (2026-07-08T06:45:14.266Z)

```

=== orchestration anchors (post-rebase) ===
247: def submit_time_video_id(input_ref, cfg) -> Optional[str]:
814: local_video = storage.acquire_local_input(
833: candidate = trusted_video_id or ""
841: video_id = _main.compute_video_id(local_video)
1276: local_src = storage.acquire_local_input(
=== highlights handler ===
513: Mirrors /api/highlights' safety (bare-name + containment checks)."""
626: # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
1041: @app.get("/api/highlights/{video_id}/{filename}")
1042: def download_highlight(video_id: str, filename: str, request: Request):
1059: return RedirectResponse(signed, status_code=307)
=== PHASE_PLACEMENT.md on master? ===
present in worktree

```

## 214. assistant (2026-07-08T06:45:35.569Z)

Accurate anchors confirmed: `acquire\_local\_input`:814, re-hash:841, `download\_highlight`:1042, PHASE\_PLACEMENT.md now on master. Let me find all the doc strings needing updates so my edits match exactly.

## 215. user (2026-07-08T06:45:38.569Z)

```
6:> **Relation to existing docs:** extends [COMPUTE_DECOUPLED_SERVING/README.md](./README.md)
(ADR-014) and [PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does
not supersede them ? it answers the *topology* question they left open and sequences the
migration on top of `phase_placement`.
13:The 2026-07-08 live CUJ moved one 1.83 GiB clip **3x** (upload?bucket, bucket?control-plane
to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for **~16
min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by
uploading directly onto the L4 and async-copying to the bucket. **This design recommends
against that direction** and quantifies why: a same-region bucket?compute transfer inside GCP
is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-
onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a **minutes-
long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a
durability exposure window instead of closing one.
17:**Actionable now (this PR):** the design doc + **win #2** (drop the redundant control-plane
re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ?
this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is
rejected at current volume on a quantified cost basis and left as an owner-gated future ADR.
23:1. **Data is moved 3x per run** `[verified]`: (a) upload streams straight to
`gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix,
`backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane
re-downloads the whole clip to scratch to validate ? `_execute_processing` ?
`storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-
hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect
(`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
30:**Read to get current:** [PHASE_PLACEMENT.md`
(#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the `phase_placement` knob this
sequences on, [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that
bound the options), and the runlog §9 (the evidence).
144:| **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP just
signs | `backend/main.py::get_highlights` |
165:| **P1** | **Stitch ? L4** | `phase_placement.compile=cloud_run_l4` (+
`cloud_stitch_encoder`) | in-process stitch | **#521 (open)** |
167:| **P2b** | **Validation ? L4** (CP stops running validators) |
`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment) | CP validates |
#521 (open) |
168:| **P2 net** | **CP never downloads the file** ? the ~30 s gap dies; movement 3x?2x | P2a ?
P2b together | either half alone degrades gracefully | needs #521 **** win #2 merged |
191:| P2 | **Win #1** ? validation-on-L4 (`phase_placement.validation`) | #521 | No | ?
delivered by #521 ? not duplicated here | #521 |
212:- [] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
218:**Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219:**Internal docs:** [PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-
highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
```

## 216. assistant (2026-07-08T06:46:05.398Z)

Now I'll apply all the confirmed fixes. Since #521 is merged, PHASE\_PLACEMENT links become relative and Win#1 status becomes genuinely done. Let me batch the edits:

217. user (2026-07-08T06:46:10.233Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

218. user (2026-07-08T06:46:15.567Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

219. user (2026-07-08T06:46:23.633Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

220. user (2026-07-08T06:46:32.633Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

221. user (2026-07-08T06:46:41.134Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

222. user (2026-07-08T06:46:46.016Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

223. assistant (2026-07-08T06:46:48.678Z)

Now the \$5 migration table rows and \$7/\$9 status reconciliation (Win#1 now genuinely done on master):

224. user (2026-07-08T06:46:52.082Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

225. user (2026-07-08T06:47:00.099Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

226. user (2026-07-08T06:47:04.513Z)

185        Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from  
186        `origin/master`, no stacking.

187

	ID	Deliverable	Depends on	Gated?	Status	PR
188	----	-----	-----	-----	-----	----

```

189 | P0 | **This design doc** (resolves topology A/B/C; recommends A?; specs durability +
phased migration) | ? | owner-review | ? (this PR) | #524 |
190 | P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5;
`deployment.trust_submit_video_id` (default on), reversible | ? | No | ? (this PR) | #524 |
191 | P2 | **Win #1** ? validation-on-L4 (`phase_placement.validation`) | #521 | No | ?
delivered by #521 ? not duplicated here | #521 |
192 | P3 | **CP-download-avoidance measured live** ? confirm the ~30 s gap is gone +
movement is 2x on rev ? `00020` | #521 + P1 merged | ? owner-side (live spend, D17) | ? | ? |
193 | P4 | **Worker validation-verdict serialization** ($6 gap) so P2b keeps rich SPA
verdict + #518 cache | #521 | No | ? follow-up | ? |
194 | P5 | **Persistent-GPU re-evaluation** only if load crosses $4.2 breakeven | volume | ?
new ADR | ? | ? |
195
196 ## 8. Open questions ? owner / external
197
198 **Blocking (owner sign-off):**
199 1. **Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it;
this doc recommends against it with the $4.2/$4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers
the same "collapse onto the L4" outcome, cheaper and more durable, via config. *If you still
want upload-onto-L4* (e.g. for a future no-bucket edge deployment), $4.5 specs its durability
model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).
200 2. **Flip P2b (validation?L4) before or after P3 (verdict serialization)?** P2b alone
offloads CPU but degrades the reject-UX + skips #518 cache-warm ($6). Recommend: land P3 first,
then P2b.
201
202 **Nice-to-know:**
203 3. Is pilot volume expected to cross the $4.2 breakeven within the planning horizon?
(Decides whether P5 is ever live.)
204
205 ## 9. Definition of done
206
207 - [x] Design doc resolves the topology question (A/B/C scored) with a clear
recommendation (A?) + quantified cost/latency/reliability. `[this PR]`
208 - [x] Minimal control-plane surface defined ($4.4). `[this PR]`
209 - [x] Async copy-to-bucket durability model specified with failure/resume + exposure
window ($4.5). `[this PR]`
210 - [x] Phased migration laid out on #521's `phase_placement`, each step a config flip
with a bucket-mediated fallback ($5). `[this PR]`
211 - [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
212 - [] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
213 - [] Live CUJ confirms the gap is gone + movement is 2x (P3, owner-side). `[gated]`
214 - [] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.
215
216 ## 10. References

```

## 227. user (2026-07-08T06:47:12.906Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

## 228. user (2026-07-08T06:47:18.688Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

## 229. assistant (2026-07-08T06:47:23.476Z)

Now the \$10 references (anchors + PHASE\_PLACEMENT link) and the \$6 immutability/TOCTOU honest caveat. Let me read \$6 first.

## 230. user (2026-07-08T06:47:25.265Z)

171  
172     \*\*The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's  
knob + this PR's win #2.\*\* No rearchitecture, no new infra, fully reversible.  
173  
174     ## 6. Honest limits ? what this does NOT do / does NOT prove  
175  
176     - \*\*A green suite does not prove the cloud path\*\* ? the tests are fully-mocked/offline  
(a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are  
validated \*\*in the cost model + code paths\*\*, not on live infra. A live CUJ on rev ? `00020`  
(owner-side, D17 budget) is required to \*confirm\* the gap is gone and the movement is 2x.  
177     - \*\*Network-egress "free intra-region" is `[researched]`, not live-verified\*\* ? general  
web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress  
rests on documented GCP behavior, not a fetched invoice. If it were \*not\* free, A? is \*still\*  
cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.  
178     - \*\*Win #2 changes nothing when no trusted id is present\*\* ? local/appliance uploads and  
non-GCS direct-URI submits (`submit\_time\_video\_id` ? `None`) still re-hash. The win applies to  
the GCS cloud-serving path (the CUJ case).  
179     - \*\*P3 (worker validation-verdict serialization) is a real gap #521 leaves open\*\*  
`[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-  
validator `ValidationResult` list (`result['results']` the SPA renders) and the #518  
`validation\_cache` warm are \*\*not\*\* reconstructed on the CP. Until P3, moving validation to the  
L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't  
flipped on blind.  
180     - \*\*SQLite is the residual CP scaling limit\*\* ? the thin router is stateless \*except\*  
the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).  
181     - \*\*The #513 asymmetry reverses under P2b\*\* `[verified]`: worker-side validators run at  
model-default thresholds unless the CP forwards `validation.\*` as `--set`. #521's validation-  
delegation path must forward those (or re-inherit the divergence that caused the #513 incident).  
Flagged for the #521 review, not owned here.  
182  
183     ## 7. Deliverables & progress tracker ? source of truth

### 231. user (2026-07-08T06:47:39.424Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 232. assistant (2026-07-08T06:47:43.561Z)

Now the §10 references ? fix the orchestration anchors and the PHASE\_PLACEMENT link:

### 233. user (2026-07-08T06:47:54.196Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 234. assistant (2026-07-08T06:48:04.024Z)

Let me sweep the doc for any remaining stale references before re-running gates:

### 235. user (2026-07-08T06:48:08.336Z)

=== stale refs remaining? ===  
=== '(open)' or 'still open' anywhere ===  
(empty above = all clean)

### 236. assistant (2026-07-08T06:48:15.986Z)

All stale refs cleaned. The base changed (rebased onto #521), so I must re-run the \*\*full\*\* gate suite ? my changes now interact with #521's merged code. Let me run everything, and check the DOC\_STATUS precedent in parallel.

### 237. user (2026-07-08T06:48:19.724Z)

```
=== is #521's PHASE_PLACEMENT.md registered in DOC_STATUS? (precedent check) ===
0
=== check_md_links (PHASE_PLACEMENT relative links now resolve?) ===
[check-md-links] clean ? no broken internal links in live docs.
```

### 238. user (2026-07-08T06:52:11.743Z)

```
training\gen0\harness.py 130 9 93%

TOTAL 12671 1810 86%
Required test coverage of 70% reached. Total coverage: 85.72%
1882 passed, 8 skipped, 25 warnings in 226.26s (0:03:46)
=====
RESULT: PASS - all tests green
=====
```

### 239. assistant (2026-07-08T06:52:19.252Z)

Post-rebase: **\*\*1882 passed, 8 skipped, coverage 85.72%\*\***; check\_md\_links clean (relative PHASE\_PLACEMENT links now resolve). DOC\_STATUS precedent confirmed ? #521's PHASE\_PLACEMENT.md has **\*\*no\*\*** DOC\_STATUS row either, so sub-docs of the CDS project aren't individually registered; I'll match that (the reviewer marked it optional). Now the remaining gates post-rebase:

### 240. user (2026-07-08T06:52:27.397Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 150 source files
=== LEAK_SCAN ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
```

### 241. assistant (2026-07-08T06:52:36.289Z)

All gates green post-rebase (suite 1882 / coverage 85.72%, ruff/mypy/leak\_scan/md\_links clean). Now let me fold the doc fixes into the single commit (unpushed, so amending keeps a clean one-commit history) and verify the final state.

### 242. user (2026-07-08T06:52:39.367Z)

```
M docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
=== stage the doc fix (only file changed since rebase) ===
M docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
```

### 243. assistant (2026-07-08T06:52:56.487Z)

Only the design doc changed since the rebase. Per the "prefer a new commit over amending" convention, I'll make a second commit capturing the review-fix pass. Let me re-verify branch safety and commit.

### 244. user (2026-07-08T06:53:05.280Z)

```
feat/524-l4-primary
=== commits ahead of origin/master (must be ONLY mine) ===
6d528f1 @ feat(serving): L4-primary topology design + trust submit-time video_id (#524)
=== final commits ===
3a043fc @ docs(524): address adversarial-review findings; reconcile now-merged #521
6d528f1 @ feat(serving): L4-primary topology design + trust submit-time video_id (#524)
```

#### 245. user (2026-07-08T06:53:19.331Z)

=== HEAD commit full message ===

@

docs(524): address adversarial-review findings; reconcile now-merged #521

- fix a [verified] anchor that named a nonexistent symbol (backend/main.py::get\_highlights -> ::download\_highlight)
- refresh orchestration.py line anchors shifted by this PR + the #521 rebase; prefer durable symbol-name anchors over brittle absolute line numbers
- #521 merged to master as #525: switch PHASE\_PLACEMENT.md links to relative, update phased-migration statuses (P1/P2b delivered), tick the win #1 DoD box
- narrow the \$0 idle-GPU upper bound to what the \$4.2 cost model supports (\$0.08-0.13)
- add an honest \$6 caveat: win #2 byte-identity rests on in-flight object immutability (safe today); a future presigned/client-writable upload path must generation-pin before enabling trust\_submit\_video\_id
- note the runlog phase label vs the code-accurate cause of the ~30s gap

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

@

=== HEAD~1 full message ===

@

feat(serving): L4-primary topology design + trust submit-time video\_id (#524)

Design doc (docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md) resolves the compute-topology question and recommends AGAINST uploading onto the L4 (a cost model shows it trades a ~free intra-region LAN pull for minutes of idle GPU); instead collapse validate+detect+stitch onto the scale-to-zero Cloud Run L4 Job via #521 phase\_placement + bucket upload, honoring ADR-014 D4/D7/D16.

Win #2: the control-plane drain trusts the server-derived submit-time md5 (jobs.video\_id <- submit\_time\_video\_id, #484 - byte-identical to compute\_video\_id, already trusted for the submit-time cache probe) instead of re-hashing the fetched clip. Gated by deployment.trust\_submit\_video\_id (default on), guarded by a strict 32-char lowercase-hex check; absent/malformed -> re-hash as before. Fully reversible.

Refs #524, #521, #520.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

@

#### 246. assistant (2026-07-08T06:53:37.174Z)

Found a real bug: I used PowerShell here-string syntax (`@'...'@`) inside the **Bash** tool (POSIX sh), so both commit **subjects** are literally `@` with the real subject on line 2 and a trailing `@`. I'll fix by soft-resetting and re-committing once, cleanly, using a message file to avoid quoting pitfalls.

#### 247. assistant (2026-07-08T06:53:49.583Z)

feat(serving): L4-primary topology design + trust submit-time video\_id (#524)

Epic #524 is investigation-first. This lands the two low-risk, reversible pieces and defers the risky rearchitecture behind a reviewed design.

Design doc (docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md) resolves the compute-topology question (Cloud Run Service-with-GPU vs GCE L4 VM vs hybrid) and recommends AGAINST the epic's "upload directly onto the L4": a same-region GCS->Cloud Run pull is LAN-fast and not billed as internet egress, so uploading onto the GPU box trades a ~free, seconds-fast pull for ~175s of idle GPU per run and opens a durability window. Instead collapse validate+detect+stitch onto the scale-to-zero Cloud Run L4 Job via #521's phase\_placement (merged as #525) + bucket upload, honoring ADR-014 D4/D7/D16. Persistent-GPU is rejected at current

volume on a cost basis (breakeven ~51%/31% duty cycle) and left as a future ADR.

Win #2: the control-plane drain now trusts the SERVER-DERIVED submit-time md5 (jobs.video\_id <- submit\_time\_video\_id, #484 - the GCS metadata md5 in hex, byte-identical to compute\_video\_id and already trusted for the submit-time cache probe) instead of re-hashing the multi-GB clip it just fetched. Gated by deployment.trust\_submit\_video\_id (default on), guarded by a strict 32-char lowercase-hex check; absent/malformed -> re-hash exactly as before. Fully reversible; the resulting video\_id is byte-identical so no cache/DB key changes.

Win #1 (validation-on-L4) is delivered by #521/#525, not duplicated here.

Verified locally (gates match CI): python run\_all\_tests.py --cov-fail-under=70 -> 1882 passed, 8 skipped, coverage 85.72%; ruff, mypy (150 files), leak\_scan, check\_md\_links all clean. New tests/test\_video\_id\_trust.py proves trust-vs-rehash and the drain wiring. Not verifiable locally (owner-side, D17): the live CUJ confirming the ~30s gap is gone. Design + code + tests only; no deploy.

Adversarial multi-agent review folded (wf\_dd531439): fixed a doc anchor naming a nonexistent symbol, reconciled statuses now that #521 merged, and added an honest caveat that win #2's byte-identity rests on in-flight object immutability (safe in today's server-mediated-upload topology; a future presigned/client-writable upload path must generation-pin before enabling the trust).

Refs #524, #521, #520.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

#### 248. user (2026-07-08T06:53:49.645Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\wt524-msg.txt (file state is current in your context ? no need to Read it back)

#### 249. user (2026-07-08T06:53:55.828Z)

```
=== staged files after soft reset (should be all 7) ===
backend/api/job_worker.py
backend/config/models.py
backend/orchestration.py
backend/pipeline/stitcher/compiler.py
backend/worker/__main__.py
docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
docs/COMPUTE_DECOUPLED_SERVING/README.md
tests/test_compile_dispatch.py
tests/test_compiler.py
tests/test_config_deployment.py
tests/test_video_id_trust.py
tests/test_worker.py
=== final commit ===
f7030b2 feat(serving): L4-primary topology design + trust submit-time video_id (#524)
=== subject line check ===
feat(serving): L4-primary topology design + trust submit-time video_id (#524)
```

#### 250. assistant (2026-07-08T06:54:24.158Z)

Wait ? the soft-reset staged **\*\*#521's files too\*\*** (compiler.py, worker/\_\_main\_\_.py, PHASE\_PLACEMENT.md, test\_compile\_dispatch.py, etc.). That means my commit's diff-vs-`origin/master` is wrong. This is exactly the kind of concurrent-checkout hazard the guardrails warn about. Let me diagnose the actual branch topology before doing anything else.

#### 251. user (2026-07-08T06:54:27.624Z)

```
=== HEAD and its parent ===
```

```
f7030b2 feat(serving): L4-primary topology design + trust submit-time video_id (#524)
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
=== parent of HEAD ===
65ee4c64dcab41c74f63613c973e9069d0167b8d
=== origin/master ===
65ee4c64dcab41c74f63613c973e9069d0167b8d
=== merge-base(HEAD, origin/master) ===
65ee4c64dcab41c74f63613c973e9069d0167b8d
=== does origin/master contain #521? ===
origin/HEAD -> origin/master
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
```

## 252. assistant (2026-07-08T06:55:17.933Z)

This is the sibling-work-revert hazard the guardrails warn about: `origin/master` advanced again (dac6736 ? \*\*65ee4c6\*\*, adding #527/#528 follow-ups) and my soft-reset re-based my commit onto it, so my tree carries the \*old\* versions of #527/#528's files ? my commit would \*\*revert them\*\*. I must fix this. Let me first see exactly which files #527/#528 touched so I know which to restore.

## 253. user (2026-07-08T06:55:21.494Z)

```
=== what my commit f7030b2 currently changes vs origin/master (MUST be only my 7 files) ===
backend/api/job_worker.py | 5 +
backend/config/models.py | 10 +
backend/orchestration.py | 100 ++++-----
backend/pipeline/stitcher/compiler.py | 19 +-
backend/worker/__main__.py | 3 -
.../L4_PRIMARY_TOPOLOGY.md | 226 ++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 8 +-
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_compile_dispatch.py | 85 +-----
tests/test_compiler.py | 62 -----
tests/test_config_deployment.py | 2 +
tests/test_video_id_trust.py | 173 ++++++
tests/test_worker.py | 74 +-----
13 files changed, 480 insertions(+), 289 deletions(-)
```

```
=== commits origin/master gained since dac6736 (the base I rebased onto) ===
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
=== files those commits touched ===
backend/orchestration.py | 49 ++++++--
backend/pipeline/stitcher/compiler.py | 19 +++-
backend/worker/__main__.py | 3 +
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 8 +-
tests/test_compile_dispatch.py | 85 ++++++
tests/test_compiler.py | 62 ++++++
tests/test_worker.py | 74 ++++++
7 files changed, 287 insertions(+), 13 deletions(-)
```

## 254. assistant (2026-07-08T06:56:04.250Z)

Now it's clear: `origin/master` gained #527 (`65ee4c6`) and #528 (`008a5d4`) ? both touching `compiler.py`, `worker/\_\_main\_\_.py`, `orchestration.py`, `PHASE\_PLACEMENT.md`, and 3 test files. My dac6736-based commit would \*\*revert all of those\*\*. I must restore those 6 sibling files to `origin/master` and re-merge my win#2 edits into the current `orchestration.py`. Let me uncommit, restore the sibling files cleanly, and reset orchestration.py to re-apply my edits.

#### 255. user (2026-07-08T06:56:08.077Z)

```
=== restore the 6 sibling files + orchestration.py to clean origin/master ===
=== staged now (should be my 6 non-orchestration files; orchestration cleaned) ===
backend/api/job_worker.py | 5 +
backend/config/models.py | 10 +
.../L4_PRIMARY_TOPOLOGY.md | 226 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py | 2 +
tests/test_video_id_trust.py | 173 +++++
6 files changed, 418 insertions(+)
```

#### 256. assistant (2026-07-08T06:56:14.460Z)

Staged changes are now exactly my 6 non-orchestration files. `orchestration.py` is clean (current master). Now I re-apply my 3 win#2 edits to it. Let me confirm the anchors still match in the current version.

#### 257. user (2026-07-08T06:56:16.967Z)

```
269: def _cached_process_payload(
752: config=None, db=None, req=None, progress=None
794: video_id = _main.compute_video_id(local_video)
```

#### 258. assistant (2026-07-08T06:56:19.908Z)

All three anchors present in the clean version. Let me read the exact surrounding context for the signature and hash block to make my edits match precisely (line numbers shifted slightly from #527/#528).

#### 259. user (2026-07-08T06:56:20.738Z)

```
752 config=None, db=None, req=None, progress=None
753) -> Dict[str, Any]:
754 """The full hash ? validate ? segment ? report pipeline for one video.
755
756 Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
757 module-level ``global_config``/``db_instance`` (backward-compatible with existing
758 tests that pass only ``req``). Route handlers always pass explicit params; older
759 test code that calls ``_execute_processing(req)`` still works.
760
761 ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
762
763 Raises on unexpected internal errors ? the caller records those as a job failure
764 """
765 import backend.main as _main # call-time seam: main's globals/model stay patchable
766
767 # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
768 # shift positional args ? req arrives in position 0 (config), progress in position 1
769 (db).
770 if config is not None and not isinstance(config, AppConfig):
771 # Old signature: _execute_processing(req, progress=None)
772 if req is None and db is not None and isinstance(db, _main.ProcessRequest):
773 req = db
774 db = None
775 elif req is None and isinstance(config, _main.ProcessRequest):
776 req = config
777 config = None
778 # Resolve config/db from module globals when not passed explicitly
779 if config is None:
780 config = _main.global_config
781 if db is None:
782 db = _main.db_instance
```

```

782 notify = progress or (lambda stage: None)
783
784 notify("hashing")
785 # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
786 # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
787 # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
788 # consumers (hash / validators / segmenter) use the resolved local path.
789 local_video = ""
790 try:
791 local_video = storage.acquire_local_input(
792 req.video_path, getattr(config, "storage", None)
793)
794 video_id = _main.compute_video_id(local_video)
795 except Exception:
796 storage.cleanup_acquired_local_input(

```

#### 260. assistant (2026-07-08T06:56:25.005Z)

Identical context ? #527/#528 didn't touch this region. Re-applying my three win#2 edits:

#### 261. user (2026-07-08T06:56:29.737Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 262. user (2026-07-08T06:56:39.858Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 263. user (2026-07-08T06:56:49.153Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 264. assistant (2026-07-08T06:56:55.019Z)

Now stage orchestration.py and verify the diff vs `origin/master` is **exactly** my 7 files (no sibling reverts):

#### 265. user (2026-07-08T06:56:59.550Z)

```

=== FULL diff vs origin/master (MUST be exactly my 7 files) ===
backend/api/job_worker.py | 5 +
backend/config/models.py | 10 +
backend/orchestration.py | 51 ++++
.../L4_PRIMARY_TOPOLOGY.md | 226 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py | 2 +
tests/test_video_id_trust.py | 173 +++++
7 files changed, 467 insertions(+), 2 deletions(-)

```

=== confirm NO revert of #527/#528: compiler/worker/PHASE\_PLACEMENT/compile-tests must NOT appear ===

GOOD: none of #527/#528's files touched

#### 266. assistant (2026-07-08T06:57:07.175Z)

Diff is now exactly my 7 files (`orchestration.py` +51 = win#2 only, no #527/#528 revert). Base moved and orchestration.py was re-merged, so I must re-run the **full** gate suite as the authoritative final check.

#### 267. user (2026-07-08T07:01:27.199Z)

```
TOTAL 12678 1796 86%
Required test coverage of 70% reached. Total coverage: 85.83%
1889 passed, 8 skipped, 25 warnings in 246.48s (0:04:06)
=====
RESULT: PASS - all tests green
=====
=== RUFF ===
All checks passed!
=== MYPY ===
Success: no issues found in 150 source files
=== LEAK ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
=== MDLINKS ===
[check-md-links] clean ? no broken internal links in live docs.
```

#### 268. assistant (2026-07-08T07:01:32.957Z)

All gates green on the correctly-based tree (1889 passed, coverage 85.83%). Let me update the commit message with the accurate figures and commit.

#### 269. user (2026-07-08T07:01:38.379Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-msg.txt has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 270. user (2026-07-08T07:01:46.542Z)

```
Exit code 128
=== branch + base safety re-check BEFORE commit ===
feat/524-l4-primary
fatal: Needed a single revision
```

#### 271. user (2026-07-08T07:01:58.509Z)

```
feat/524-l4-primary
HEAD:
65ee4c6
origin/master:
65ee4c6
=== committing ===
97f9551 feat(serving): L4-primary topology design + trust submit-time video_id (#524)
```

#### 272. assistant (2026-07-08T07:02:04.625Z)

Clean commit `97f9551` on top of `origin/master` (65ee4c6), only my commit ahead. Final safety verification before pushing:

#### 273. user (2026-07-08T07:02:08.727Z)

```
=== subject (no @ artifact) ===
feat(serving): L4-primary topology design + trust submit-time video_id (#524)
=== diff vs origin/master = exactly my 7 files ===
 backend/api/job_worker.py | 5 +
 backend/config/models.py | 10 +
 backend/orchestration.py | 51 ++++
 .../L4_PRIMARY_TOPOLOGY.md | 226 +++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
 tests/test_config_deployment.py | 2 +
 tests/test_video_id_trust.py | 173 +++++
 7 files changed, 467 insertions(+), 2 deletions(-)
=== no leftover conflict markers in my changed files ===
(empty = clean)
```

=== working tree clean? ===

#### 274. assistant (2026-07-08T07:02:12.129Z)

Everything verified: clean subject, exactly my 7 files, no conflict markers, clean working tree. Now push and open the PR against `master`.

#### 275. user (2026-07-08T07:02:16.161Z)

```
remote:
remote: Create a pull request for 'feat/524-l4-primary' on GitHub by visiting:
remote: https://github.com/Khelsutra/badminton-highlight-
indexer/pull/new/feat/524-l4-primary
remote:
branch 'feat/524-l4-primary' set up to track 'origin/feat/524-l4-primary'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch] feat/524-l4-primary -> feat/524-l4-primary
```

#### 276. assistant (2026-07-08T07:02:22.692Z)

Pushed. Let me update the PR body to reflect the #521-merged reality and accurate figures, then open the PR.

#### 277. assistant (2026-07-08T07:02:46.012Z)

### What & why

Epic **#524** (collapse the pipeline onto the L4 worker; shrink the control-plane to a thin router) is **investigation-first**. This PR lands the two lowest-risk, reversible pieces and defers the risky rearchitecture behind a reviewed design.

#### 1. Design doc ? `docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md`

Resolves the compute-topology question (**A** Cloud Run Service-with-GPU / **B** GCE L4 VM / **C** hybrid presigned-upload) with a cost model, and **recommends against the epic's proposed "upload directly onto the L4"**:

- A same-region GCS?Cloud Run transfer is **LAN-fast** and not billed as internet egress **[researched]**, so the round-trip that upload-onto-L4 saves is worth ~seconds and ~\$0, while keeping an L4 up to **receive** a **minutes-long** upload burns **~175 s** of idle GPU/run (~\$0.08?0.13) and **opens** a durability window instead of closing one.
- The cheaper, more durable, already-ratified path is the opposite: **keep upload ? bucket** (the #480 design; the durable copy exists before the L4 starts) and **collapse validate+detect+stitch** onto the scale-to-zero Cloud Run L4 Job **via #521's `phase\_placement`**. The residual ~30 s gap dies not by moving the upload but by the control-plane **never** downloading the file (delegate validation per #521 + trust the md5 per win #2). Net movement: **3x ? the floor** (1 client upload + 1 LAN pull), zero idle-GPU cost, zero exposure window.
- Persistent-GPU (VM / Service-with-min-instances) is **rejected at current volume** (breakeven ? 51 % on-demand / 31 % spot duty cycle ? 2.6?4.3k runs/mo) and left as an owner-gated future ADR. Honors ADR-014 **D4/D7/D16**.

The doc also specifies the minimal control-plane surface, the async copy-to-bucket durability model (failure/resume + exposure window), and a phased migration where each move is a config flip with a bucket-mediated fallback.

#### 2. Win #2 ? drop the redundant control-plane re-hash

The drain re-hashed the whole (often multi-GB) clip it just fetched, even though the submit hook already recorded the file's md5 on the job row (`submit_time_video_id`, #484 ? the GCS-metadata md5 in hex, **already trusted** for the submit-time cache probe). Now the drain reuses that **server-derived, byte-identical** id and skips the re-hash; absent / malformed ? it hashes exactly as before. Gated by `deployment.trust_submit_video_id` (default on) and guarded by `_looks_like_video_id` (strict 32-char lowercase hex).

**\*\*Win #1 (validation-on-L4) is intentionally NOT here\*\*** ? it is delivered by **\*\*#521, merged to master as #525\*\*** (``phase_placement.validation=cloud_run_l4`` + its guards). Per the coordination note in #524, this PR does **\*\*not\*\*** duplicate that knob; the design doc credits #521 and specifies the download-avoidance follow-up (which composes #521's delegation with this PR's win #2) plus the verdict-serialization gap #521 leaves open.

## Coordination / rebase safety

- Rebased onto the **\*\*latest `origin/master`\*\***, which now includes #521 (#525) **\*\*and\*\*** its #527/#528 follow-ups. The diff is **\*\*exactly 7 files\*\*** ? no sibling work reverted (verified ``git diff --stat origin/master``).
- ``deployment.trust_submit_video_id`` is a **\*\*non-overlapping addition\*\*** to ``DeploymentConfig`` alongside #521's ``phase_placement`/`cloud_stitch_encoder``.

## Reversibility

Off-by-config: ``deployment.trust_submit_video_id=false`` reverts win #2 to a byte-identical re-hash, no redeploy. When no trusted id is present (local/appliance upload, non-gcs direct-URI submit) the drain hashes as before. The resulting ``video_id`` is byte-identical (same bytes, same MD5), so no cache/DB key changes.

## Verification (all local, gates match CI)

- ``python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`` ? **\*\*1889 passed, 8 skipped, coverage 85.83%\*\*** (floor 70%).
- ``ruff check .`` clean · ``mypy backend training`` clean (150 files) · ``python -m tools.leak_scan`` clean · ``python -m tools.check_md_links`` clean.
- New ``tests/test_video_id_trust.py`` proves: trusted id ? ``compute_video_id`` **\*\*not\*\*** called; absent/malformed/knob-off ? re-hash; and the ``_run_claimed_job`` wiring threads ``jobs.video_id`` through.
- **\*\*Not verifiable locally\*\*** (owner-side, D17 budget): the live CUJ confirming the ~30 s gap is gone + movement is 2x on rev ? ``00020``. Design + code + tests only ? **\*\*no deploy\*\*** in this PR.

## Review

An adversarial 4-dimension multi-agent review (correctness · security/trust-boundary · doc-accuracy · conventions), each finding independently verified, was folded in: fixed a doc anchor naming a nonexistent symbol, reconciled statuses now that #521 merged, narrowed an unsupported cost bound, and added an honest \$6 caveat that win #2's byte-identity rests on in-flight object immutability (safe in today's server-mediated-upload topology; a future presigned/client-writable upload path must generation-pin before enabling the trust). Correctness dimension was clean; the trust-boundary TOCTOU was verified **\*\*unreachable\*\*** in the current topology (documented, not coded, correctly scoped out).

Refs #524, #521, #520.

? Generated with [Claude Code](https://claude.com/claude-code)

### 278. user (2026-07-08T07:02:46.060Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\wt524-pr-body.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 279. user (2026-07-08T07:02:54.317Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/530>

### 280. assistant (2026-07-08T07:03:02.531Z)

PR **\*\*#530\*\*** opened. Let me verify it targets ``master`` correctly and check the CI status.

### 281. user (2026-07-08T07:03:16.456Z)

PR #530 [OPEN] feat/524-l4-primary -> master  
feat(serving): L4-primary topology design + trust submit-time video\_id (#524)  
<https://github.com/Khelsutra/badminton-highlight-indexer/pull/530>  
=== CI checks (may be pending) ===  
Golden fixtures cross-OS (macos-latest) pending 0 <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617500>  
Golden fixtures cross-OS (windows-latest) pending 0  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617534>  
Syntax sweep + import smoke (no GPU stack) pending 0  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617438>  
Leak scan (PII / attribution guard) pass 9s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617379>  
Markdown link check (live docs) pass 6s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617384>  
Ruff lint pass 10s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617452>  
Built wheel smoke (install + launch) pending 0 <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617424>  
Dependency license scan (no copleft in runtime deps) pending 0  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617477>  
Full test suite (offline, mocked) pending 0 <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617401>  
Golden fixtures cross-OS (ubuntu-latest) pending 0  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617462>  
Mypy (lenient baseline) pending 0 <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617370>

## 282. assistant (2026-07-08T07:03:37.283Z)

PR #530 is open and targets `master`. CI's fast gates already **pass** on CI (leak-scan, markdown-link-check, ruff) ? corroborating my local runs. The heavier checks (full suite, mypy, cross-OS, wheel, license) are running. Let me mark the task done, clean up scratch, and watch CI to green.

## 283. user (2026-07-08T07:03:37.791Z)

Updated task #7 status

## 284. user (2026-07-08T07:06:16.337Z)

scratch cleaned  
Golden fixtures cross-OS (ubuntu-latest) pass 20s  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617462>  
Golden fixtures cross-OS (windows-latest) pass 1m7s  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617534>  
Leak scan (PII / attribution guard) pass 9s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617379>  
Markdown link check (live docs) pass 6s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617384>  
Mypy (lenient baseline) pass 35s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617370>  
Ruff lint pass 10s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617452>  
Syntax sweep + import smoke (no GPU stack) pass 23s  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617438>  
Built wheel smoke (install + launch) pass 1m9s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924085426/job/85807617424>

```

Dependency license scan (no copyleft in runtime deps) pass 32s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617477
Full test suite (offline, mocked) pass 2m45s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28924085426/job/85807617401
Golden fixtures cross-OS (macos-latest) pass 23s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28924085426/job/85807617500
Golden fixtures cross-OS (ubuntu-latest) pass 20s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617462
Golden fixtures cross-OS (windows-latest) pass 1m7s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617534
Leak scan (PII / attribution guard) pass 9s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28924085426/job/85807617379
Markdown link check (live docs) pass 6s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28924085426/job/85807617384
Mypy (lenient baseline) pass 35s https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617370
Ruff lint pass 10s https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617452
Syntax sweep + import smoke (no GPU stack) pass 23s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28924085426/job/85807617438

```

## 285. assistant (2026-07-08T07:06:38.298Z)

**\*\*All CI checks pass.\*\*** PR #530 is green across the board ? full suite, mypy, cross-OS golden fixtures (macOS/Windows/Ubuntu), wheel smoke, license scan, leak scan, markdown link check, ruff, import smoke.

Done. Here's the complete picture.

## Summary

**\*\*PR [#530](https://github.com/Khelsutra/badminton-highlight-indexer/pull/530)\*\*** ?  
`feat/524-l4-primary` ? `master`, **\*\*all CI green.\*\***

## What shipped (2 of 3 deliverables; #3 was correctly a "don't ship yet")

**\*\*1. Design doc\*\*** ? [``docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md``](docs/COMPUTE\_DECOUPLED\_SERVING/L4\_PRIMARY\_TOPOLOGY.md). Resolves the A/B/C topology question with a cost model pulled live from the GCP Billing Catalog API. **\*\*Recommends \*against\* the epic's "upload onto the L4"\*\*: the central quantified finding is that uploading onto the GPU box trades a \*\*~free, seconds-fast intra-region LAN pull for ~175 s of idle GPU per run\*\***, and **\*\*opens\* a durability window. The cheaper/more-durable path ? co-located scale-to-zero Jobs + bucket upload ? kills the ~30 s gap by the control-plane \*\*never downloading the file\*\* (composing #521's validation-delegation with win #2). Persistent-GPU rejected on a breakeven analysis. Includes the minimal control-plane surface, async-copy durability spec, and a config-flip phased migration.**

**\*\*2. Win #2 (re-hash)\*\*** ? the drain now trusts the **\*\*server-derived\*\*** submit-time md5 (``submit_time_video_id``, #484 ? already trusted for the cache probe) instead of re-hashing the fetched clip. Gated by ``deployment.trust_submit_video_id`` (default on), strict hex-format guard, byte-identical id, fully reversible. New ``tests/test_video_id_trust.py`` proves trust-vs-rehash + the drain wiring.

**\*\*Win #1 (validation-on-L4)\*\*** ? **\*\*not duplicated\*\***; delivered by #521 (merged as #525). The doc credits it and specifies the follow-ups (#521 leaves the worker verdict-serialization gap open).

## Rigor applied

- 8-agent subsystem map + adversarial 4-dimension review (each finding independently verified). Fixed a doc anchor naming a nonexistent symbol, narrowed an unsupported cost bound, added an honest TOCTOU/immutability caveat. Correctness dimension was clean.
- **\*\*All gates green locally and on CI\*\***: full suite 1889 passed / coverage 85.83% (floor 70%),

ruff, mypy, leak\_scan, check\_md\_links.

### One hazard caught and corrected

`origin/master` advanced twice mid-session (#521 ? #527/#528). My first commit would have **reverted #527/#528** (the exact "sibling-work absorbed" incident the guardrails warn about). I detected it via `git diff --stat origin/master`, restored the 6 sibling files to master, re-merged only my win#2 edits into the current `orchestration.py`, and reverified ? the final diff is **exactly my 7 files, no sibling reverts**.

### Two things left for you

- **\$8 blocking question:** do you accept rejecting "upload directly onto the L4" (the doc's L5 decision)? My recommendation is against it, with the numbers ? but it's your call since it contradicts the epic's proposed direction.
- **Owner-side (D17 budget):** a live CUJ on rev ? `00020` to confirm the ~30 s gap is gone and movement is 2x. No deploy was in scope here.

I did **not** touch the main checkout's `SESSION\_HANDOFF.md` ? it's on the `codex/issue-523` branch with another session's WIP, and the concurrent-session guardrail says not to clobber it.

### 286. user (2026-07-08T07:07:01.082Z)

is the win in this PR enabled by default?

### 287. assistant (2026-07-08T07:07:27.636Z)

Yes ? **on by default**, with one practical qualifier.

**The knob:** `deployment.trust\_submit\_video\_id` defaults to `True`. So the trust path is active out of the box (no config change needed), and it's a one-line rollback (`false`) if it ever needs disabling.

**Where it actually fires:** the win only takes effect when a trusted `video\_id` is present on the job row, which requires the input to be a **GCS object** ? `submit\_time\_video\_id` gets it from one gs:// metadata stat (#484). For **local/appliance uploads** and **non-GCS direct-URI submits** it returns `None`, so the drain re-hashes exactly as before. In other words:

Path	Default behavior with this PR
---	---
GCS cloud-serving upload (the CUJ case)	<b>skips the re-hash</b> ? the win
Local / appliance upload	re-hashes (unchanged)
Non-GCS direct URI (s3/gdrive)	re-hashes (unchanged)
Knob set to `false`	re-hashes everywhere (unchanged)

**Why it's safe to default on** (all verified, and the adversarial review's correctness dimension was clean):

- The trusted id is **server-derived**, not client-supplied ? a caller's `/api/process` body can't inject it (the submit hook recomputes the authoritative id).
- The result is **byte-identical** to today's re-hash (same bytes, same MD5), so no cache key, DB key, or output path changes.
- It's the **same value** the submit-time cache probe already trusts to complete jobs with zero re-hash ? this PR just extends that established trust to the cache-miss drain path.
- A strict 32-char-lowercase-hex guard means any malformed/absent id silently falls back to a real hash.

The one honest caveat I flagged in the doc (§6): the byte-identity rests on the bucket object being immutable in-flight, which holds in today's server-mediated-upload topology. If a future presigned/client-writable upload path is ever added, it must generation-pin before relying on this trust ? but that's not reachable today.

### 288. user (2026-07-08T07:08:39.507Z)

merged

### 289. assistant (2026-07-08T07:08:50.449Z)

Let me confirm the merge landed and clean up the isolated worktree/branch.

**290. user (2026-07-08T07:08:53.861Z)**

```
=== origin/master head ===
304af7b feat(serving): L4-primary topology design + trust submit-time video_id (#524) (#530)
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
=== is my #524 commit in origin/master? ===
304af7b feat(serving): L4-primary topology design + trust submit-time video_id (#524) (#530)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
d5a524f feat: integrate hybrid YOLOv8 + Gemini rally segmenter and extensive test suite
```

**291. assistant (2026-07-08T07:08:57.345Z)**

Confirmed merged ? `origin/master` is now at `304af7b` (#530). Let me clean up the isolated worktree and the local branch.

**292. user (2026-07-08T07:09:01.362Z)**

```
error: failed to delete '.git/worktrees/wt524-969': Permission denied
=== remaining worktrees mentioning 524 ===
none (clean)
scratch cleaned
```

**293. assistant (2026-07-08T07:09:13.595Z)**

The worktree was de-registered (no longer listed) but a metadata dir deletion hit a Windows file-lock (same benign issue as the other stale worktrees), and the `&&` chain skipped the branch delete. Let me finish cleanup independently.

**294. user (2026-07-08T07:09:22.805Z)**

```
Deleted branch feat/524-l4-primary (was 97f9551).
=== worktree dir still on disk? ===
dir gone
=== final: local branches mentioning 524 ===
none (clean)
=== current branch (unchanged main checkout) ===
```

**295. assistant (2026-07-08T07:09:35.127Z)**

Branch deleted, worktree gone, metadata pruned. Let me confirm the main checkout is still in its original state (I never switched its branch, but the empty `show-current` warrants a quick check).

**296. user (2026-07-08T07:09:37.181Z)**

**HEAD (no branch)**

```
M README.md
M backend/config/models.py
HEAD
```

**297. assistant (2026-07-08T07:10:08.362Z)**

The main checkout is in a **detached-HEAD** state with staged changes to `README.md` and `backend/config/models.py` ? this is **not** how it started (it began on `codex/issue-523-sd-reel-plan`), and those staged changes are **not mine** (my work was fully isolated in the now-removed worktree). Let me read the reflog (read-only) to understand what moved it, without touching anything.

**298. user (2026-07-08T07:10:11.735Z)**

```

=== HEAD reflog (read-only) ===
3598082 HEAD@{0}: rebase (finish): returning to refs/heads/codex/issue-523-sd-reel-plan
3598082 HEAD@{1}: rebase (pick): feat(compile): add opt-in SD reel rendition
304af7b HEAD@{2}: rebase (start): checkout origin/master
ad9923e HEAD@{3}: rebase (finish): returning to refs/heads/codex/issue-523-sd-reel-plan
ad9923e HEAD@{4}: commit: feat(compile): add opt-in SD reel rendition
65ee4c6 HEAD@{5}: rebase (start): checkout origin/master
ee3ec6c HEAD@{6}: reset: moving to HEAD
ee3ec6c HEAD@{7}: commit: feat(compile): add opt-in SD reel rendition
=== what HEAD points at ===
3598082 feat(compile): add opt-in SD reel rendition
=== does codex/issue-523 branch still exist? ===
* codex/issue-523-sd-reel-plan
+ codex/issue-523-sd-reel-review

```

## 299. assistant (2026-07-08T07:10:37.183Z)

Resolved ? and it was **not** my doing. The reflog shows the concurrent **#523** session was mid-rebase\*\* onto the updated `origin/master` (the one that now includes my **#524**) exactly when I ran `git status`. A rebase detaches HEAD and stages its picked files transiently; between that call and the next, the rebase **finished** and HEAD re-attached:

```

...
3598082 HEAD@{0}: rebase (finish): returning to refs/heads/codex/issue-523-sd-reel-plan
304af7b HEAD@{2}: rebase (start): checkout origin/master ? rebasing onto master-with-#524
...

```

`git branch` now confirms the checkout is cleanly back on `* codex/issue-523-sd-reel-plan`. This is precisely the "git state changes under you between two commands" race the guardrails warn about ? I correctly left the shared checkout untouched.

## Final state ? all clean

- **PR [#530](https://github.com/Khelsutra/badminton-highlight-indexer/pull/530)** merged ? `origin/master` at `304af7b`. All CI was green.
- **Win #2** is live on master\*\*, enabled by default (`deployment.trust_submit_video_id=True`), firing on the GCS cloud-serving path, byte-identical result, one-flag reversible.
- **Design doc** (`L4_PRIMARY_TOPOLOGY.md`) is on master with the topology recommendation for your sign-off.
- **My isolated worktree + local branch removed**, metadata pruned, scratch cleaned. The shared main checkout is healthy on its own branch (a sibling session's rebase, left untouched).

Two things still sit with you, unchanged from before: the **\$8 decision** on rejecting upload-onto-L4, and the **owner-side live CUJ** (D17 budget) to confirm the ~30 s gap is gone. Nothing else is outstanding on my end.